



UNIVERSITÀ DI PISA

LARGE-SCALE AND MULTI-STRUCTURED DATABASES

Class Notes 2023/2024

Saverio Mosti

Gabriele Pianigiani

Francesco Taverna

Sommario

BIG DATA	7
THE 5 V's	7
Volume	7
Veracity	7
Velocity	7
Variety	7
Value	8
The main Big Data computing issues	8
DISTRIBUTED SYSTEMS	8
DATA LOCALITY	9
MAPREDUCE	9
The MapReduce Workflow	10
HADOOP	12
Properties of Hadoop	12
Limits of Hadoop	12
SPARK	12
Properties of Spark	12
How Spark works	13
THE DATABASES REVOLUTIONS	14
THE FIRST DATABASE REVOLUTION – THE DBMS	14
The navigational models and their issues	14
THE SECOND DATABASE REVOLUTION	15
Relational Theory	15
ACID Transactions	16
OBJECT-ORIENTED DBMS - OODBMS	16
Object-oriented Programming vs RDBMS	16
Massive web-scale applications - MWSAs	17
The problems of the relational database	17
The needs of modern companies	18
NoSQL Databases	18
PROPERTIES OF DISTRIBUTED SYSTEMS	19
DATA AVAILABILITY	20
Two-Phase Commit	20
"TOO ACID!"	20
NETWORK PARTITIONING	20
THE CAP THEOREM - "BREWER'S THEOREM"	21
BASE PROPERTIES	22
CONSISTENCY MANAGEMENT	23
Strict Consistency	23
Eventual Consistency	24
Read-Your-Writes Consistency	24
Session consistency	24
Monotonic read consistency	25
Monotonic write consistency	25
Causal consistency	26
PARTITIONING E SHARDING	27
Vertical Partitioning	27
Horizontal Partitioning - Sharding	27
KEY-VALUE DATABASES	29
DICTIONARY	29
NAMESPACE AND BUCKETS	29
WHEN A KVDB SHOULD BE CHOSEN	29
ESSENTIAL FEATURES OF A KVDB	30
Simplicity (schema-less)	30

<i>Forgiving - No Typing for the Values</i>	30
<i>Caching System</i>	30
SCALABILITY OF A KVDB	31
<i>Master-Slave replication</i>	31
<i>Masterless replication</i>	32
RETRIEVE VALUES ON A KVDB	33
<i>Hashing for selecting servers</i>	33
CHARACTERISTICS OF A KVDB	34
<i>KVDB operations and complex queries</i>	34
KVDB AND CLUSTERS	36
<i>Data Partitioning</i>	36
<i>Loosely Coupled Cluster</i>	36
<i>Replication of a KVDB</i>	36
<i>Hash mapping</i>	38
DATA COMPRESSION FOR KVDB	41
DESIGN OF A KVDB	41
<i>Relational to KV</i>	41
<i>KVDB Design – Tips and Tricks</i>	42
CASE STUDY – RESTAURANT	46
DOCUMENT DATABASES	47
XML DOCUMENTS DATABASES	47
<i>XML Ecosystem</i>	48
<i>XML schema Document</i>	49
<i>XML Databases</i>	50
JSON (JAVASCRIPT OBJECT NOTATION) DOCUMENTS	52
<i>Comparison of JSON and XML</i>	52
<i>Features of JSON Databases</i>	53
DOCUMENT DATABASES: DATA MODELLING	54
<i>Schemeless property</i>	54
<i>How to deal with relationships among entities on a JSON Database</i>	54
HOW TO MODEL HIERARCHIES	58
<i>Exercise - Trucks Company</i>	59
PLANNING FOR MUTABLE DOCUMENTS	60
<i>Documents relocation</i>	60
<i>Life cycle of a document</i>	60
INDEXING A DOCUMENT DATABASE	60
<i>Applications and indexes</i>	61
<i>Transactions Processing Systems</i>	62
COLLECTIONS	62
<i>Manipulate a collection with code</i>	63
QUERIES AND ENTITIES	63
COLUMNAR DATABASES	65
OLTP vs OLAP	65
<i>Data Warehouse</i>	66
<i>Optimize analytics - Star Scheme</i>	67
COLUMN STORAGE	68
<i>Solution for the OLTP and OLAP problem</i>	69
COLUMNAR COMPRESSION	69
<i>Delta Store</i>	70
<i>Projection</i>	70
<i>Hybrid Columnar Compression (Oracle)</i>	72
BIGTABLE	72
<i>Row and row key</i>	72
<i>Keyspace</i>	73
<i>Columns</i>	73
<i>Column family</i>	73
	2

COLUMNAR DATABASES — DESIGN TIPS	74
<i>Advantages of a columnar database</i>	74
<i>Typical queries in columnar databases</i>	75
<i>Example: Relational to columnar</i>	75
<i>Differences between Relational and Columnar databases</i>	76
<i>Avoid join operation and the name of the column</i>	77
<i>Model an entity with specialisations</i>	79
<i>How to manage the previous versions of values</i>	79
<i>Avoid complex structures</i>	79
<i>Indexes</i>	80
<i>Translate relationships</i>	81
ARCHITECTURE OF A DISTRIBUTED DATABASE	81
PARTITIONING	81
<i>Partition policy</i>	81
<i>Pros of partitioning</i>	81
SHARDING	81
<i>Sharding policy</i>	82
<i>Pros of sharding</i>	82
HBASE	82
MASTERLESS (CASSANDRA)	83
COMMIT LOG	83
BLOOM FILTER	84
REPLICATION	86
<i>Consistency level</i>	86
<i>Anti-Entropy</i>	86
COMMUNICATION PROTOCOLS	88
<i>Fully Connected Approach</i>	88
<i>Centralised Approach</i>	88
<i>Gossip Approach</i>	88
<i>Cassandra Approach</i>	88
HOTSPOTTING AND LOAD BALANCING	89
GRAPH DATABASES	90
RECAP OF GRAPH'S THEORY	90
<i>Vertices</i>	90
<i>Edges</i>	90
<i>Types of Graphs</i>	90
<i>Path in a graph</i>	90
<i>Loops</i>	91
<i>Union & Intersection of Graphs</i>	91
<i>Graph traversal</i>	92
<i>Isomorphism</i>	92
<i>Properties of a graph</i>	92
<i>Properties of a vertex</i>	92
<i>Betweenness</i>	93
<i>Other types of graphs</i>	93
GRAPH DATABASES INTRODUCTION	94
<i>Graph databases vs Relational databases</i>	94
<i>Relationships in graph databases</i>	95
<i>Typical queries on a graph database</i>	95
<i>Example of graph database: HollywoodDB</i>	96
<i>Example of graph database: Social Network</i>	98
<i>Scalability of graph databases</i>	100
MongoDB	101
MONGODB KEY FEATURES	101
ARCHITECTURE OF MONGODB	101
<i>Replication in MongoDB</i>	101

<i>Sharding in MongoDB</i>	102
CORE PROCESSES OF MONGODB	102
DOCUMENTS IN MONGODB	103
<i>The BSON file</i>	103
<i>The <code>_id</code> field</i>	103
MONGODB OPERATIONS	103
<i>Insert Operation</i>	103
<i>Drop Operations</i>	104
<i>Query operations</i>	104
<i>Update operations</i>	105
<i>Cursors</i>	105
<i>Queries on embedded and nested documents</i>	106
<i>Queries on arrays</i>	107
<i>Projection of fields</i>	108
<i>Queries on the type of the field value</i>	109
<i>Aggregation</i>	111
MONGODB INDEXES	115
<i>Index commands</i>	115
<i>Compound Indexes</i>	116
<i>Index for embedded documents and arrays</i>	117
<i>Advanced Indexing</i>	117
<i>Types of Indexes</i>	118
IN MEMORY DATABASES	119
PERSISTENCY MANAGEMENT	119
<i>Snapshot</i>	119
<i>Transaction log</i>	119
TIMESTEN	120
REDIS	120
HANA DB	121
ORACLE 12C	122
Neo4J	124
NODE	124
LABELS	124
RELATIONSHIPS	124
<i>Properties</i>	124
TRAVERSAL AND PATH	125
INDEXES	126
<i>Index free adjacency</i>	126
GUIDELINES FOR SELECTING A DATABASE	127
WHAT GUIDE US?	127
KEY-VALUE DATABASES	127
DOCUMENT DATABASES	127
SOFTWARE ENGINEERING RECAP	128
FUNCTIONAL REQUIREMENTS	128
NOT FUNCTIONAL REQUIREMENTS	129
USE CASE AND ACTORS	130
<i>Use case diagram</i>	130
DATA MODELLING	131
DATA ANALYSIS	131
JAVA RECAP	133
<i>How is a Java file 'compiled' and executed?</i>	133
<i>Encapsulation</i>	135
<i>Polymorphism</i>	136
<i>Packages</i>	138
<i>References and Primitive data types and Functions</i>	139
<i>Input in Java</i>	142

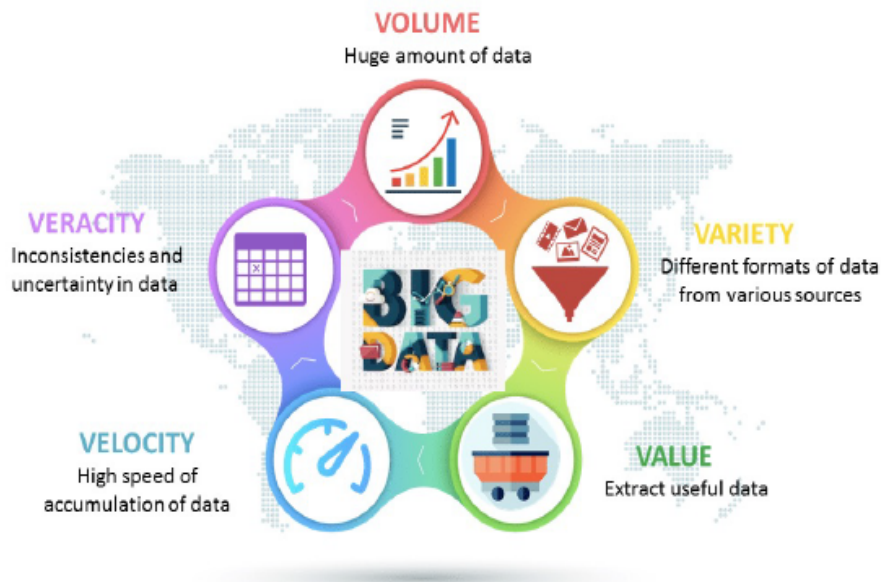
JAVA CODE NOTES	143
<i>Try & Catch & Finally block</i>	143
<i>Throw keyword</i>	145
CONNECTORS	146
<i>Driver Manager</i>	146
<i>Connection to MySQL</i>	147
<i>Perform Queries and Prepare Statements</i>	147
<i>Update Tables</i>	148
MAVEN	150
<i>The POM File</i>	150
JAVA MVC	151
<i>Model</i>	151
<i>View</i>	151
<i>Controller</i>	153
<i>Example of MVC: To-do list application</i>	153
DAO & DTO	154
<i>Model</i>	154
<i>DAO - Data Access Object</i>	154
<i>DTO - Data Transfer Object</i>	155
REDIS	157
<i>Redis Data Types</i>	158
<i>Lists and Sets</i>	158
<i>The Redis Server</i>	159
<i>Transactions in Redis</i>	161
<i>When should you use Redis?</i>	163
JEDIS	164
<i>Connecting to the Redis server</i>	164
<i>Jedis Commands</i>	166
<i>Connection to the database</i>	166
<i>Subscribe / Unsubscribe</i>	166
<i>Publisher / Subscriber</i>	166
SCALING IN REDIS	167
<i>Eviction policies</i>	167
<i>Persistence</i>	168
<i>Replication</i>	168
<i>Keyspace Notification</i>	174
<i>Publisher – Subscriber</i>	175
<i>Task 3 : Ecommerce</i>	177
SOFTWARE LAYERED ARCHITECTURE	177
<i>Package Organization</i>	178
<i>Redis Exercises</i>	178
GOOD PRACTICES IN DEVELOPING JAVA APPLICATION	180
<i>Package structure and naming conventions</i>	180
<i>Git and the collaborative work</i>	184
<i>How to structure applications with Maven</i>	186
<i>API & SPI</i>	193
<i>Code Obfuscation</i>	194
<i>Javadoc – Document your stuff!</i>	195
<i>Testing Java applications</i>	196
JAVA & MONGODB	206
<i>Access to the database</i>	207
<i>Access and handling a collection</i>	208
<i>Create and insert a document</i>	208
<i>Query a collection</i>	210
<i>Update a document</i>	211
<i>Delete a document</i>	211

<i>Pipeline</i>	212
MONGODB REPLICATION	214
<i>The primary node</i>	214
<i>The secondary nodes and replicas</i>	214
<i>Write concern specification</i>	215
<i>Acknowledgment behaviour</i>	215
<i>Replica sets</i>	216
<i>Read preference</i>	219
<i>Replication on MongoDB - shell commands</i>	221
EXAM QUESTIONS (ITALIAN)	228
ESERCIZI PRATICI DOCUMENTDB	243
AGGREGATION PIPELINE	244
ESERCIZI PRATICI KVDB	245
ESERCIZI PRATICI COLUMNARDB	246

Big Data

The word represents a big amount of data received from different sources (*internet*) that continuously generate and send data at different speeds (*often very high speed*).

The 5 V's



Volume

Files are sent continuously on the internet, but you should be aware that even a simple file of a few Megabytes contains millions of bytes.

Veracity

If I collect some data at time T , at time $T+1$ are they still valid or they must be updated/deleted?

Different Sources

In different sources we can find data about the same thing, but they could be inconsistent, they could be different. This is also a problem of security of the sources. What do I have to do if I receive incorrect data or that comes from a not secure source?

Velocity

Every source has a different speed that can depend on many conditions (*network, load of work...*).

Variety

Sources can be everything connected to the internet that produce, collect, and send data like security cameras, laboratories, transactions, mobile devices and so on. Every source can produce many files of different format and size.

Value

Raw data do not have any value; their value (*knowledge*) must be extracted. Every data has value, and nobody gives data for free. Data has to be interpreted, alone they have no meaning, so we have to give them a meaning.

The main Big Data computing issues

Classical data elaboration algorithms, such as the ones for analytics and data mining, cannot directly apply to Big Data:

- New technologies are needed for both storage and computational tasks.
- New paradigms are needed for designing, implementing, and experimenting algorithms for handling big data.

Vertical Scaling

If you need more, then upgrade your machines to increase the elaboration capacity. But, with this technique I can issue:

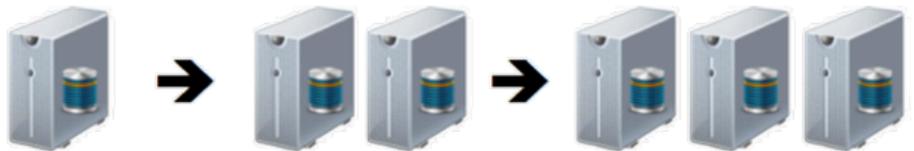
- Single point of failure.
- Data are physically centralised (*too much data in a single storage*).
- Cost of the single machine.



Horizontal Scaling

If you need more, then buy more machines and distribute the incoming jobs.

- Data are physically distributed (*but logically centralised*), they are spread in different databases.
- We need more complexity to manage distributed data (because I want that logically data appear like they are stored in a unique place).



Distributed Systems

Which is the Objective?

To apply an operation to all data. But one machine cannot process or store all data. So, we need a distributed system (*horizontal scaling*).

What is distributed system and Why we need it?

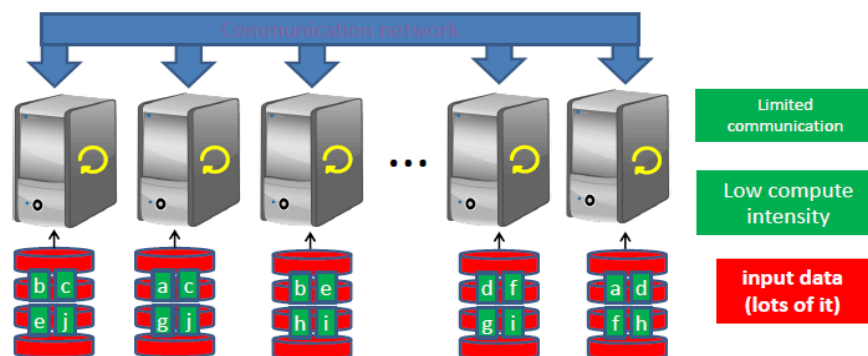
A distributed system is any environment where multiple computers or devices are working on a variety of tasks and components, all spread across a network. Components within

distributed systems split up the work, coordinating efforts to complete a given job more efficiently than if only a single device ran it.

Properties of a distributed system

- Data is distributed in a cluster of nodes.
 - We still have to know where data physically are.
- It does not matter which node executes the operation.
 - It does not matter if the same job is run multiple times in different nodes (*due to failures or straggler nodes*).
- We look for an abstraction of the complexity behind distributed systems.
 - We want that all the architecture appears like a unique computer.

Data Locality



The process of moving computation to the node where that data resides, instead of vice versa.

This technique helps to:

- Minimise network congestion.
- Improve computation throughput.

We store data on disks of the nodes that perform computations on that data.

Note that we should avoid data transfers between machines as much as possible; because moving data from disk to memory (*and vice versa*) is time expensive.

MapReduce

MapReduce is a framework for distributed computation on a huge quantity of data using a large cluster of computers.

Data is distributed among nodes (*distributed file system*), operations are distributed among nodes. It follows the “*divide & conquer*” idea:

Divide (map)

Divide the partition dataset into smaller, independent chunks to be processed in parallel.

Conquer (reduce)

combine, merge, or otherwise aggregate the results from the previous step.

The MapReduce Workflow

MapReduce consist in 6 steps:

Input Reading

We receive and unstructured data input from a source.

Splitting

We divide the input dataset in N smaller and independent chunks to be processed in N parallel nodes.

Map

Mapping is a partial elaboration that every node does on their own. Each node (*of the N nodes that we speak about in the splitting*) will process one of the N chunks. Every node will produce a partial result.

Shuffle

We group the partial results. The grouping operation is based on the type of partial result.

Reduce

In the reduce step we group the N partial results into a unique result, which is part of the final output of the computation.

Result

We merge all the parts, and we finally obtain the final output.

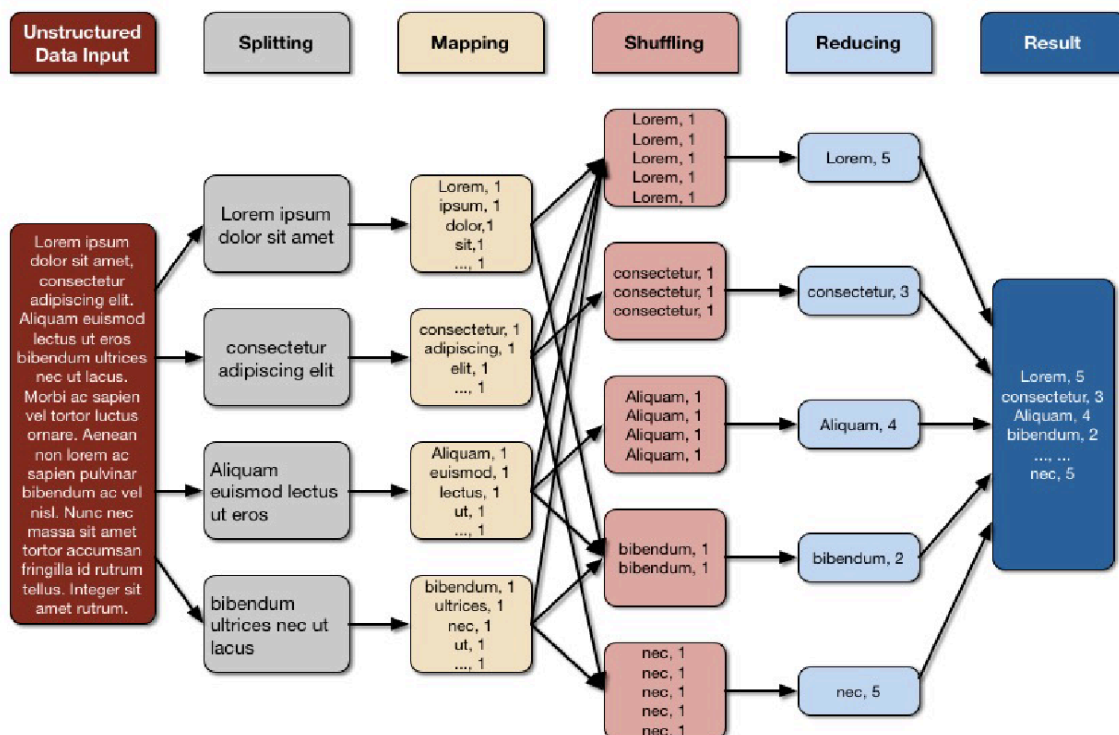
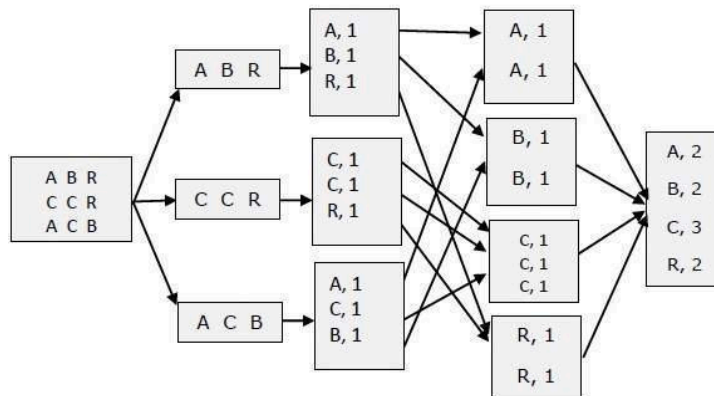


Figure 5- MapReduce workflow

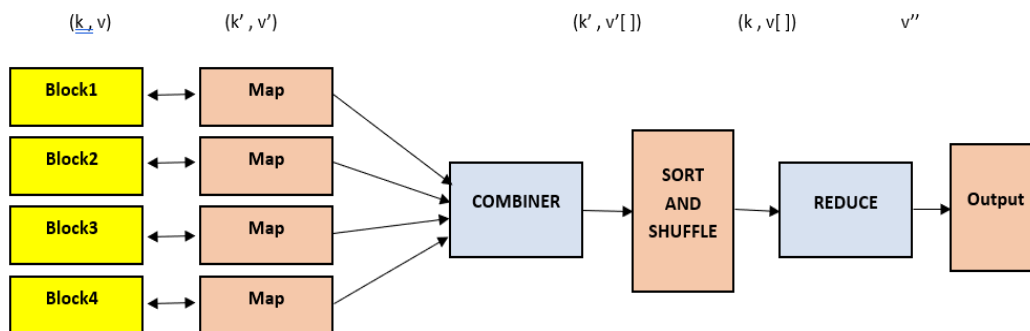
MapReduce - practical explanation

In this example, the computation consists in calculating the number of occurrences of letters in an input text.

1. The initial text is divided into 3 smaller blocks.
2. Each block goes in input at one mapper; each mapper produces a partial result:
 - “For each letter we have the number of that letter in the text”.
3. Then partial results go into a reducer that aggregates them to produce a part of the output:
 - “Calculate the addition of the occurrences of a specific letter”.
4. All the results produced by reducer are merged in the final output.



MapReduce - formal explanation



1. Initially data are divided in chunks.
 - A chunk is something like $\langle K, V \rangle$.
2. Every mapper receives one chunk and produces a partial result.
 - A partial result is something like $\langle K', V' \rangle$.
3. Then the partial results are sorted and shuffled.
 - In this step they are regrouped using the key K' , so the result of the shuffling is the list of map results regrouped by key.
 - The result of the shuffling is something like $\langle K', V[] \rangle$.
4. Then there is the reduce operation that produces the final output by merging the result of the shuffling.

Hadoop

It is an open-source framework (*written in Java*) that we can use for running applications that manage big data. Hadoop uses MapReduce for perform distributed computations. Hadoop consists of several modules, each one implementing a specific functionality: one of those modules is called `HadoopDistributedFileSystem(HDFS)` and it implements a distributed file system (*thanks to that module I can use the distributed storage like a unique big storage*).



Properties of Hadoop

Horizontal Scaling and Data Locality

Hadoop can scale to clusters with thousands of computing nodes, each node will store a part of the data in the system.

Fault Tolerance

Hadoop is robust to faults because it has data replication (a partition of the data can be replicated in more than one node).

Designed for low-cost hardware

Designed for managing big files using low-cost hardware.

Efficient for read operations

Efficient for read and append operations (*random updates are to consider rare*).

Limits of Hadoop

Hadoop is efficient for one-shot (*read and append*) operation, but for operations that include more iteration of Map and Reduce it's not the best choice. Hadoop doesn't have a good memory computation to do that, in fact for each MapReduce operation we have to do an access to the disk for loading data from disc to primary memory. A write operation needs an access to the disk, so it is really slow and represents a bottleneck.

Spark

Appreciated for its enhanced flexibility and efficiency: Spark uses a master/slave architecture i.e., one central coordinator and many distributed workers. Here, the central coordinator is called "*driver*", and the workers are called "*executors*". Each executor is a separate process.



Properties of Spark

Flexibility on the distributed model

Spark allows employing different distributed programming models, such as MapReduce and Pregel. Regards MapReduce, Spark has proved to perform faster than Hadoop, especially in case of iterative and online applications.

In-memory cluster computing

Unlike the disk-based MapReduce paradigm supported by Hadoop, Spark employs the concept of in-memory cluster computing, where datasets are cached in memory to reduce their access latency until the last operation, at the end of the map reducing operation the memory is flushed.

Independent processes

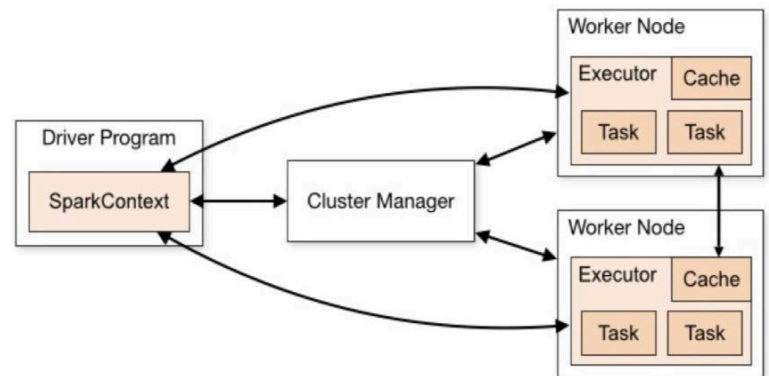
At high level, a Spark application runs as a set of independent processes on the top of the dataset distributed across the machines of the cluster and consists of one driver program and several executors.

How Spark works

The driver program, hosted in the master machine, runs the user's main function, and distributes operations on the cluster by sending several units of work, called tasks, to the executors.

Each executor, hosted in a slave machine, runs tasks in parallel and keeps data in memory or disk storage across them.

The cluster Manager does the resource allocating work.



The databases revolutions

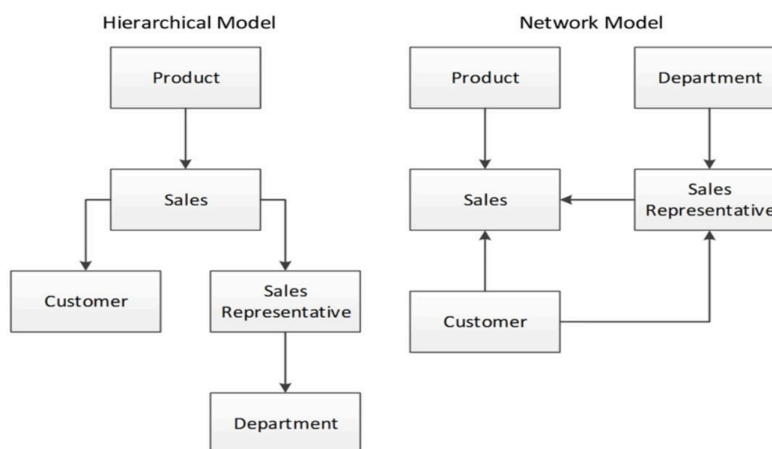
The First Database Revolution – The DBMS

The DBMS represented a new layer that allowed the separation between the database handling logic (*functional integrity, join etc*) and the application itself. As a result, programmer overhead was minimised, and the performance and integrity of data access routines were ensured.

The two Main Features of a DBMS are:

- The definition of a scheme for the data to store in the database.
- Give an access path for navigating among data records.

The navigational models and their issues



Lack of Flexibility

They *ran* exclusively on the *mainframe computer* systems of the day (*largely IBM mainframes*)

Only *queries* that could be *anticipated* during the initial design phase were possible.

Complexity

It was *extremely difficult* to add new data elements to an existing system. It is not clear the logical and physical division of data.

Time-Hungry for application developing

CRUD (Create, Read, Update, Delete) operations oriented: complex analytic queries required hard coding: Indeed, programmers had to write methods for reading, modifying files, tables and in general for interacting with stored data.

The Second Database Revolution

The business demands for *analytic-style reports* were growing rapidly. Existing databases had some problems:

- Were too hard to use.
- Lacked a theoretical foundation.
- Mixed logical and physical implementations.

Relational Theory

Now we will see a recap of the traditional relational theory.

Tuples

A tuple is an unordered set of attribute values. In a database system, a tuple corresponds to a row, and an attribute to a column value of the tuple.

Relations

A relation is a collection of distinct tuples and corresponds to tables in relational database implementations.

Constraints of the Relational Theory

Constraints enforce consistency of the database. *Key constraints* are used to identify tuples and relationships between tuples.

Operation and the Relational Theory

Operations are made on relations (*such as joins, projections, unions*). These operations always return relations. In practice, this means that a query on a table returns data in a tabular format. You must set your queries in advance.

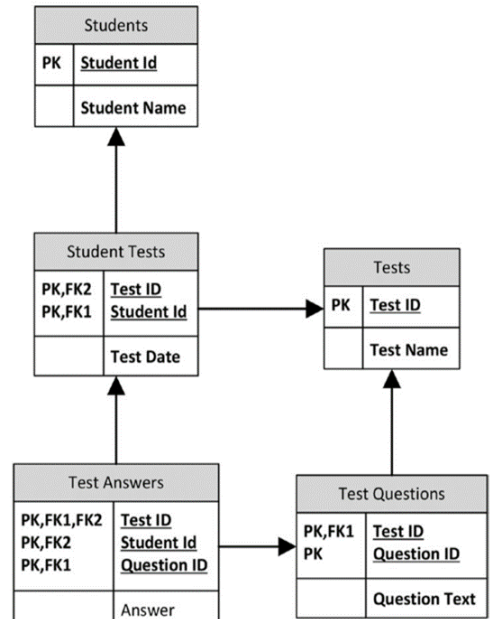
The Third Normal Form

Non-key attributes must be dependent on “the key, the whole key, and nothing but the key”.

Un-normalized data

Test scores	
Student Name	
Test Name	
Test Date	
Answer 1	
Answer 2	
Answer 3	
Answer 4	
Answer 5	
Answer 6	
Answer N	

Normalized data



ACID Transactions

A transaction is a concurrent data change request on a database system. We need rules to ensure consistency and integrity of data.

An ACID transaction should be:

- **Atomic:** The transaction is *indivisible* - either all the statements in the transaction are applied to the database or none are.
- **Consistent:** The database remains in a consistent state *before* and *after* transaction execution.
- **Isolated:** While multiple transactions can be executed by one or more users simultaneously, one transaction should *not see the effects* of other in- progress transactions.
- **Durable:** Once a transaction is saved to the database, its changes are expected to *persist* even if there is a *failure* of the operating system or hardware.

Example of ACID Transaction

Consider the transaction T
T1 and T2:

Transfer of 100 from account X to

Before: X : 500	Y: 200
Transaction T	
T1	T2
Read (X)	Read (Y)
X: = X - 100	Y: = Y + 100
Write (X)	Write (Y)
After: X : 400	Y : 300

following consisting of

account Y.

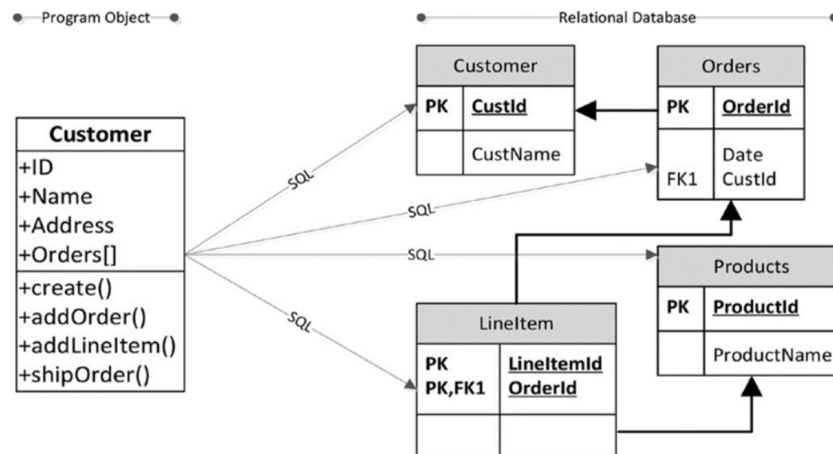
If the transaction fails after completion of T1 but before completion of T2 (after write(X) but before write(Y)), then the amount has been deducted from X but not added to Y. 44

This result leaves the database in an *inconsistent* state.

Object-Oriented DBMS - OODBMS

Object-oriented Programming vs RDBMS

Object-oriented (OO) developers were frustrated by the mismatch between the object-oriented representations of their data within their programs and the relational representation within the database. In the program a whole entity was represented with only one class, but in the database the entity was represented with several tuples in different tables.



“A relational database is like a garage that forces you to take your car apart and store the pieces in little drawers”.

The encapsulation is important: the class also has public and private attributes and methods: people can only access to the field programmers want to, These characteristics are lost in a relational model. When an object was stored into or retrieved from a relational database, multiple SQL operations would be required to convert from the object-oriented representation to the relational representation.

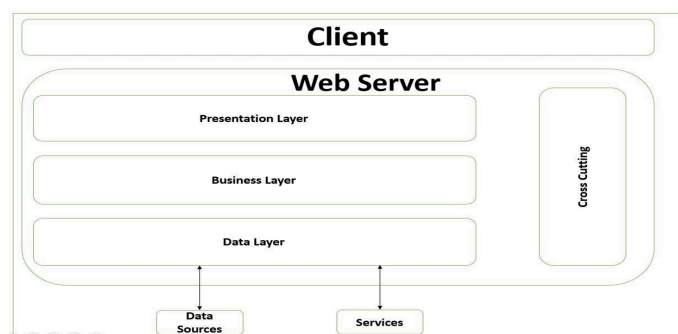
OODBMS - A Solution for OO Programmer Frustration

A new DBMS model, namely Object-Oriented Database Management System (OODBMS). An OODBMS permits you to store objects without normalisation.

- Applications would be able to load and store their object very easily.
- The implementation resembles the “Navigation” models.

However, OODBMS systems had completely failed to gain market share, But Object-Relational Mapping (ORM) frameworks (Hibernate for example) has been introduced to alleviate the pain of OO programmers.

Massive web-scale applications - MWSAs



Big companies started to put pressure on the traditional paradigm because they need:

- Large volume of read and write operations.
- Low latency responses time.
- High Availability.
 - The system should continue to work correctly and should always provide feedback.

The problems of the relational database

Difficult horizontal scaling

The performance of RDBMSs may be improved by *upgrading* CPUs, *adding* memory and *faster* storage devices, but the vertical scaling has some problems:

- *Costly*, in terms of hardware and maintenance.
- There are *limitations* on the number of CPUs and memory dimension supported by a server.

The horizontal scaling has its problems too: It's very hard to manage a distributed relational database.

- *Performance issues* related to some operations that involve tables stored on different nodes.
- *ACID transactions* are hard to implement on multiple servers.

Hard to modify the schema

And finally, in a RDBMS it's *hard* to modify the structure of existing data (*i.e.*, *adding an attribute to an object stored in a table*).

The needs of modern companies

Scalability

We have a spike (*picco*) of traffic on a website, then we add new servers (*scaling-out* or "*temporary horizontal scaling*") that we shut down when the spike ends; but it is hard to adopt this solution with a RDBMS, because we need to manage all the replicas and manage all the updates. We need fast and cheap horizontal scaling.

Flexibility

E-commerce has millions of products, all different, so an object can have some columns that other objects don't need to have. We want a scheme-less model, a model that doesn't give a limit on type and attributes that I want to use. Every object has its model, we shouldn't have to assume the category of an object. In a RDBMS it's very hard to change the scheme once you have data in the database.

Availability

The system should always be available for all users all around the world (*companies lose money if their services are offline*). So, the database must be distributed to reduce the latency at the minimum, so, if a server crashes or has too much work, there should be another server ready to take its job even though the prestation go down. The RDBMS isn't the correct choice because managing tables on a distributed database is very time consuming.

Low Costs

RDBMS have licence costs that often depend on the number of users. A solution could be to use an open-source DBMS, but an enterprise then has to change the entire infrastructure.

NoSQL Databases

"NoSQL" stands for "Not Only SQL".

NoSQL considers a set of data models and software. Most NoSQL solutions ensure: *Scalability, Availability, Flexibility* and often they are *open source*. ACID may not be supported on a NoSQL database. NoSQL rejects the constraint of the relational model, including the strict consistency and schemas. But there exists also NoSQL databases which respect ACID properties and the features of a relational database. SQL Cloud-based solutions are still largely used (*Amazon and Google*).

DBMS and Distributed Systems

General tasks of a DBMS

Databases must allow users to store and retrieve data. To follow this aim, three tasks must be in charge to a DBMS:

- Store data persistently.
- Maintain data consistency.
- Ensure data availability (important non-functional requirement).

Properties of distributed systems

Most of the recent NoSQL DBMSs can be deployed and used on distributed systems, namely on multiple servers rather than a single machine, that can communicate with each other with a network.

Scalability, flexibility, cost control and availability

If you want to consume less power, you can switch off some of the servers and turn on when you need to.

Easy horizontal scaling

It is easier to add or to remove nodes (horizontal scalability) rather than to add memory or to upgrade the CPUs of a single server (vertical scalability)

Easy to improve

Allow the implementation of fault tolerance strategies and spikes management.

I can't have everything

To balance the requirements of data consistency and system availability. Because if we have replicas, we need to ensure the consistency, therefore I must update the information's of all replicas (*time consuming*).

Nodes isolation

To protect themselves from network failures that may leave some nodes isolated.

Data availability

The data availability refers to the fact that data is always available to be read or to be modified. Data stored in a single server DBMS may be not available for several reasons, such as failures of the operating system, voltage drops, disks break down.

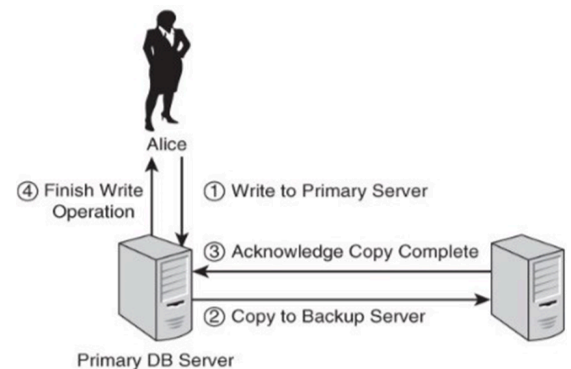
Availability can be achieved through replication. But keeping all the replica consistent is a problem that we have to solve.

Two-Phase Commit

All the transactions on the primary server are replicated on the backup server, and the final commit is sent to the user only when both servers have done the transaction; during the two-phase commit, every other query to the data is blocked.

If the primary server goes down, the backup server takes its place, and the DBMS continues to offer its services to the users.

The two-phase commit is a whole transaction itself, so it should be ACID!



“TOO ACID!”

There are some applications in which it is not acceptable waiting too much time for having concurrently consistent data and highly available systems. In this kind of application, the availability of the system and its fast response is more important than having consistent data on the different servers.

Other applications need consistency first, like every application that concerns the financial world. The system must be always available (*during and ACID transaction the system is not available*), otherwise I will lose money! So, the response time should be minimum.

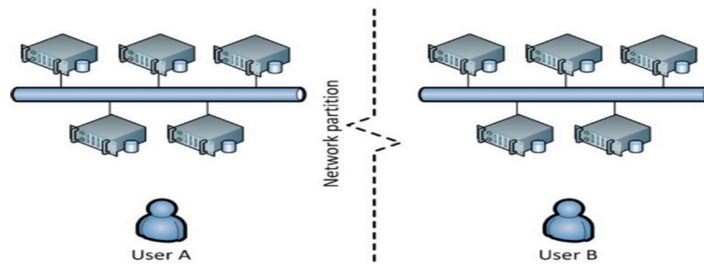
Network partitioning

A network partition is a division of a computer network into relatively independent subnets, either by design, to optimise them separately, or due to the failure of network devices.

Distributed software must be designed to be partition-tolerant, that is, even after the network is partitioned, it still works correctly.

A system has two choices:

1. Show each user a different view of the data (*availability but not consistency*) but keep itself available.
2. Shut down one of the partitions and disconnect one of the users (*consistency but not availability*).



Properties of a network partition

- Each partition is independent, so if one of them fails the other can still work.
- Each partition has its nodes and replicas so it may happen that an entire partition has inconsistent data.
- If a partition fails, some terminals may not be available.
- Some node may can't communicate with some other nodes due to the separation of the subnets.

The CAP theorem - “Brewer’s theorem”

Distributed Databases cannot ensure at the same time:

Consistency (C)

Data is the same across the cluster, so you can read/ write from/to any node and always get the same result.

Availability (A)

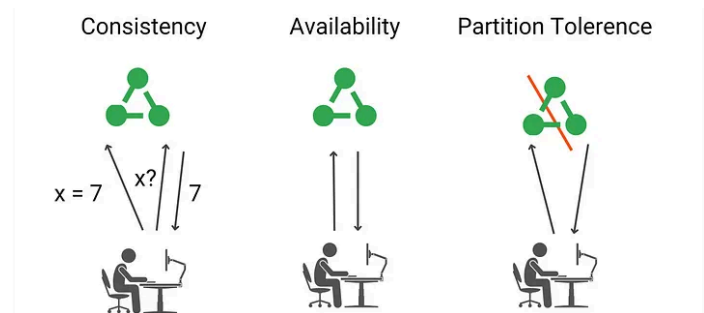
The ability to access the cluster even if a node in the cluster goes down.

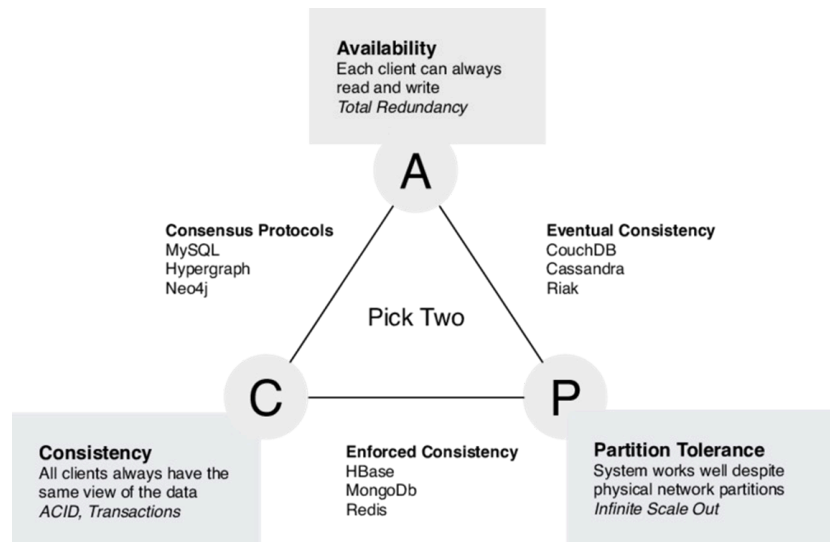
Partition Tolerance (P)

The system’s ability to continue functioning despite the presence of network partitions (“network partitioning”, or simply “partitioning”) or communication breakdowns between nodes.

The CAP theorem thesis and the CAP triangle

At maximum two of the previous three features may be found in a distributed database.





CA Solutions

When a network partition occurs, the system cannot respond in a correct way, but the system will always be available and give consistent responses.

Use cases: Banking and Finance application, system which must have transactions e.g. connected to RDBMS.

AP Solutions

System is still available under partitioning, but some of the data returned may be not updated.

In case of partition the system is still working: write operations can be processed when the partition is resolved.

Use cases: shopping carts, News publishing CMS, etc.

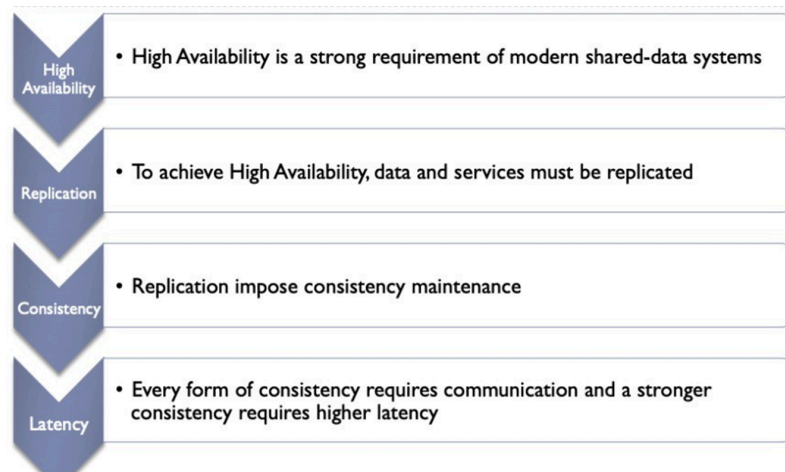
CP Solutions

I don't care about availability (*i can accept that the system is working for someone and for someone else don't*), I have partition tolerance, so probably in another terminal the system is still working.

As suitable for applications which require consistency, but also partition tolerance, while somewhat long response times are acceptable (Bank ATMs).

BASE properties

Base is a list of requirements that some applications may need.



BA - Basically Available

Reading and writing operations are available as much as possible (*using all nodes of a database cluster*) but an operation might not be consistent (*the write might not persist after conflicts are reconciled, and the read might not get the latest write*).

S - Soft state

The state of the database can change over time, even without any explicit user intervention.

Due to the effects of background processes, updates to data, and other factors.

The database should be designed to handle this change gracefully and ensure that it does not lead to data corruption or loss.

E - Eventually consistent

At some point (*I don't know precisely when*) in the future, data in all nodes will converge to the consistent state.

Consistency Management

I have to deal with the update of all replicas, so i must implement a set of functions in charge to maintain the consistency of replicas.

More requests I get, the more latency I will have, because maintaining consistency all over the replicas requires time.

To minimize the latency, we must implement those functions properly (*avoid useless time-consuming operations*).

Strict Consistency

The strongest consistency model: An update of a variable needs to be seen instantaneously by all users.

More requests I get, the more latency I will have, because maintaining consistency requires time.

We want low latency, so we have to implement those functions properly (*avoid time-consuming operations*).

Eventual Consistency

The update will eventually be reflected in all nodes that store the data.

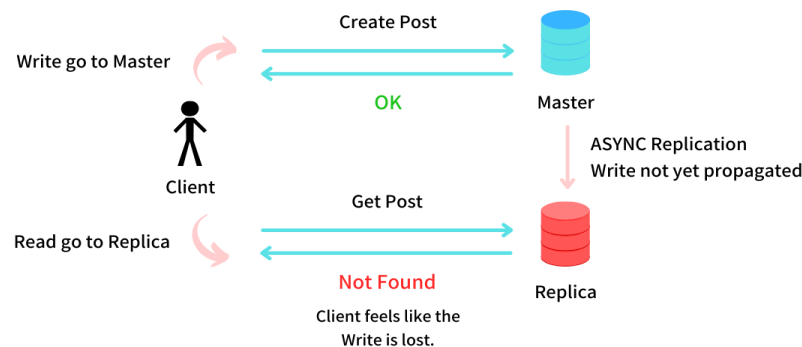
It is not sure that a user will always read the last version of the information, but eventually (*at some time in the future*), the user will read the last version of the value.

Read-Your-Writes Consistency

Once an item on a certain replica has been updated by a client C1, any attempt to read the very same record by C1 will always return the last version of the value.

But the update of the same value on other replicas may not be instantaneous, so any other client that performs the same read operation immediately after the write operation of C1 could get an outdated value (*if the read operation is done in another server, otherwise the client will get the updated one*).

This consistency is meant to reassure the user (*that made the write operation*) that his write operation is successful.



1. Alice updates a customer's outstanding balance to \$1,500.
2. The update is written to one server and the replication process begins updating other copies in the other servers.
3. During the replication process, Alice queries the customer's balance (*so Alice do not end the session previously opened*).
4. She is guaranteed to see \$1,500 when the database supports read-your-writes consistency.

Session consistency

Ensures "*read-your-writes consistency*" but only during a session.



The animated graphic above illustrates the session consistency with musical notes. The "West US 2 writer" and the "East US 2 reader" are using the same session (*session A*) so they both read the same data at the same time.

Whereas the "Australia East" region is using a different session (*session B*) so, it receives the updates later but in the same order.

What if the user changes session?

If the user ends a session and starts another session with the same DBMS, there is no guarantee the server will "*remember*" the writes made by the user in the previous session.

The user could not immediately see his updates, but this paradigm follows the eventual consistency, so *eventually he will be able to see his updates*.

Monotonic read consistency

If a process reads the value of x , any successive read operation on x by the same process will always return that same value or a more recent value.

If another process makes the same query on the same location, it is not guaranteed that he will see the same version of the first user.

1. Alice updates the customer's outstanding balance to \$1,500.
2. Alice updates the customer's outstanding balance to \$2,500.
3. Bob queries the database and sees \$2,500.
4. Bob queries the database and sees \$2,500.

If Bob issues the query again, he will see the balance is \$2,500 even if all the servers with copies of that customer's outstanding balance have not updated to the latest value.

Monotonic write consistency

If a process makes two write operations, they will be executed in the order he issued them, and every other process will see the two operations in the very same order.

This paradigm can't be applied to updates made by different process on the same object.

1. Alice reduces all customers' outstanding balances by 10%. (*sconto del 10%*)
2. Charlie has a \$1,000 outstanding balance.
3. The reduction is applied: Charlie now has a \$900 balance.
4. Alice continues to process orders.
5. Charlie ordered \$1,100 worth of material.
6. Charlie's outstanding balance is now the sum of the previous outstanding balance:
 - $\$900 + \$1100 = \$2000$.

Now consider what would happen if the database performed Alice's operations in a different order.

1. Alice reduces all customers' outstanding balances by 10%. (*sconto del 10%*)
2. Charlie has a \$1,000 outstanding balance.
3. Charlie ordered \$1,100 worth of material.
 - Charlie's balance becomes $\$1000 + \$1100 = \$2100$.
4. The reduction is applied: Charlie's balance is set to \$1890.

The balance of Charlie now is 1890 Instead of \$2,000, so Alice lost \$110. So, the order of the operations is very important.

Causal consistency

Is a particular type of eventual consistency, where we have consistency on the view order of the operations, but only if there is a causal relationship between them.

If an operation causally depends on a preceding operation, there is a causal relationship between the operations. This relationship could create side-effects that that the user must see.

If an action A happens before an action B and there is a causal relationship between them, then the action A must always be executed before the action B on all replicas.

As we can see in the image below: between the post of Pietro and the post of Mike it is a casual relationship because Mike's post is a response to Pietro's post.

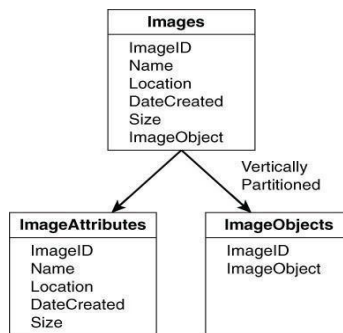


If Pietro would not have written the post, then Mike would not have written the response. In the accepted view order, I maintain the casual relationship.

Partitioning e Sharding

Vertical Partitioning

It's a technique used for improving (*relational*) database performance by separating columns of a relational table into multiple separate tables. So, from a table I obtain two different tables.

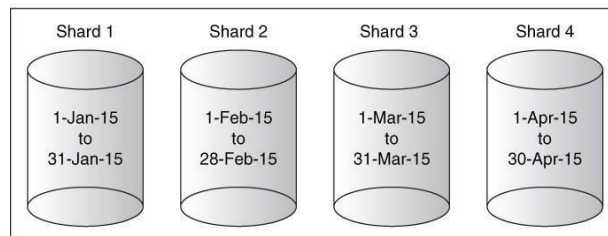


Horizontal Partitioning - Sharding

Sharding or horizontal partitioning is the process of dividing data into blocks or chunks.

Each block, labelled as “shard”, is deployed on a specific physical server of a cluster.

Normally each node can contain only one shard, but in case of data replication, a shard can be hosted by more than one node.



Pros of sharding

- Handling heavy loads and the increase of system users.
- Data may be easily distributed on a variable number of servers that may be added or removed by request.
- Cheaper than vertical scaling.
- Combined with replications, ensures a high availability of the system and fast responses.

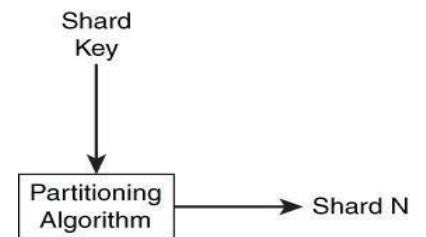
We have to create a shard key which contains some attributes, only an atomic field can be chosen as a shard key. We may use the name, the date etc.

Shard Keys

A shard key is one or more fields that exist in all documents in a collection (*the value of the attribute doesn't have to be unique*), that is used to separate documents.

A shard key must be a field that does not change often in value.

Examples of shared keys may be like (*unique document ID, Name of a person, Date of birth, creation date, Category or type, Geographical region*).



Sharding Algorithms

Range

"All the document created in the 2023 go in the shard 3".

Hashing

The shard is decided by the hash value of the shard key.

List

Let's imagine a product database with several types (*electronics, appliances, household goods, books, and clothes*).

These product types could be used as a shard key to allocate documents across five different servers.

Key-Value databases

Dictionary

Is the data storage paradigm used in these types of databases. Dictionaries contain a collection of objects (*records*) which may have many different fields within them, each one containing data (*for example a JSON object*). These records can be retrieved using a key that uniquely identifies them.

Keys and values can be anything from a simple object to complex compound objects. This type of structure permits to Key-value databases to be highly partitionable and horizontal scalable at levels that other types of databases cannot.

Key	Value
K1	AAA,BBB,CCC
K2	AAA,BBB
K3	AAA,DDD
K4	AAA,2,01/01/2015
K5	3,ZZZ,5623

Namespace and buckets

The namespace is a first level container, and it is used to organize data. Meanwhile buckets are used to organize data within a namespace (*they usually contains highly correlated data*).

The namespace is composed by one or more buckets. If you use more than one bucket, then the key is: Final Key = < Bucket Name , Key > so keys must be unique within the bucket.

In a semantic point of view, a namespace permits dealing with the same value but with multiple different keys. Namespaces are helpful when multiple applications use the same key-value database.

Namespace: products

Bucket: electronic

Key: product123

Value: {"name": "Phone1", "price": 599, "brand": "ABC"}

Key: product124

Value: {"name": "Phone2", "price": 699.99, "brand": "BBC"}

Bucket: clothes

Key: product456

Value: {"name": "Shirt", "price": 79, "brand": "XYZ" }

When a KVDB should be chosen

When you need ease of storage

A key-value database requires a low quantity of memory

When you need a fast retrieval of data

If performance is more important than organising data into more complex data structures, such as tables or networks.

When you do not need “multi-table” features

Practically, when you do not need all the features of the relational models (*joining tables or running queries about multiple entities in the database*).

Essential features of a KVDB

Simplicity (*schema-less*)

Unlike a relational one, we do *not* have to *define* a database *schema* nor to define data *types* for each *attribute*. I only have to define the namespace and the buckets.

If we need to add a new attribute (*maybe because a certain entity needs, it*) we can simply modify the code of our program without the necessity of *modifying* the database.

We can make mistakes assigning a wrong type of data to an attribute or use different types of data for a specific attribute.

The key-value database can also be used instead of a configuration file because it can be easily read by an application.

Forgiving - No Typing for the Values

No typing Check: If I insert something (*like a telephone number*) with an inconsistent value (*like a string without numbers*), nobody will check this. This permits the system to be very fast.

String: '1232 NE River Ave, St. Louis, MO'

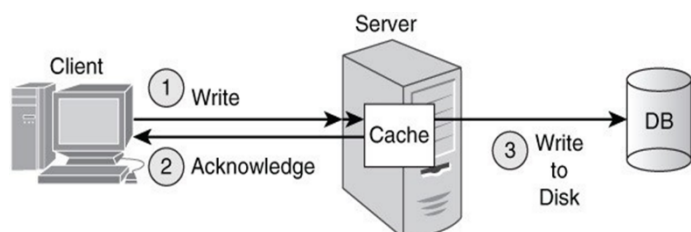
List of strings: ('1232 NE River Ave', 'St. Louis', 'MO')

JSON: {'Street:' : '12 Ave', 'City' : 'NY' , 'State' : 'MO' }

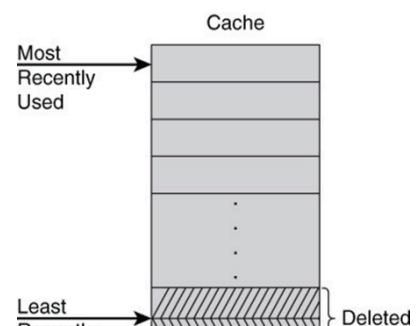
For the key-value database are all the same (*strings*).

Caching System

The last updated version of the value is loaded in the RAM and while the user program is doing something else using the cache the DBMS can persistently write the recently updated values to disk.



This trick permits the user to perform write operations at a very high speed and the disk will be eventually updated (*and eventually in a batch mode*) and it decrease drastically the number of accesses to the disk .



Obviously will use a partial paging strategy to put data in the cache (*because i can't put all the database in the cache*), but if the database is small enough, then it can be entirely loaded in the primary memory.

When the key-value database *uses all the RAM memory* allocated to it, the database will need to *free* some records to make space for the new ones.

Several algorithms exist for selecting the records to remove from the RAM and the *Least Recently Used* (LRU) is the most used one.

Scalability of a KVDB

Capability to add or remove servers from a cluster of servers as needed to accommodate the current load on the system. When dealing with *NoSQL databases*, employed in web and other large-scale applications, it is particularly important to correctly handle operations when the system is horizontally scaled.

I need the replication of the server. In the framework of *key-value databases*, two main approaches are adopted:

- Master-Slave Replication.
- Masterless Replication.

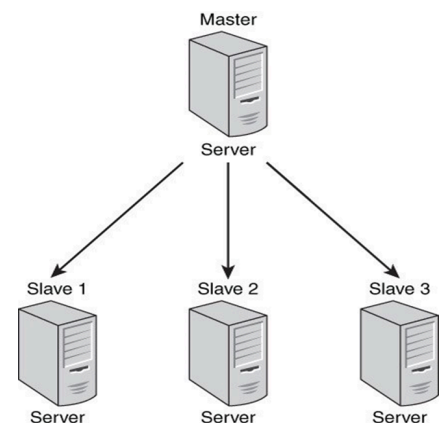
Master-Slave replication

The *master* accepts *write/read* requests, and it oversees the collecting of all the updates of the data for eventually be sent to all the slaves (*for update them too*).

The slaves may only respond to read requests, to do some load balance.

When a write operation is performed on the master node, it is eventually replicated to all the slave nodes to keep the data consistent across the entire system.

This type of architecture is particularly suitable for read-intensive applications, in other words for applications with a growing demand for read operations.



Pros of the Master-Slave replication : Simplicity

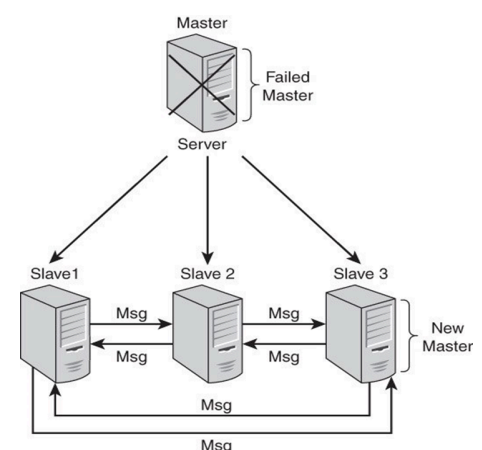
Only the master communicates with the slave servers and the communication is “one-way”. There is no need to coordinate write operations or resolve conflicts between multiple servers accepting write requests, the slave only receives updates from the master.

Cons of the Master-Slave replication : Single Point of Failure

If the master fails, the cluster cannot accept writes requests anymore, but it can still respond to any read request.

There is a strategy for respond to a master crash for keep the system available (*able to receive write requests*):

1. Periodically, each server (*master and slaves*) sends a message to another server in the cluster to check if it is still active (*if active, it will simply respond*).



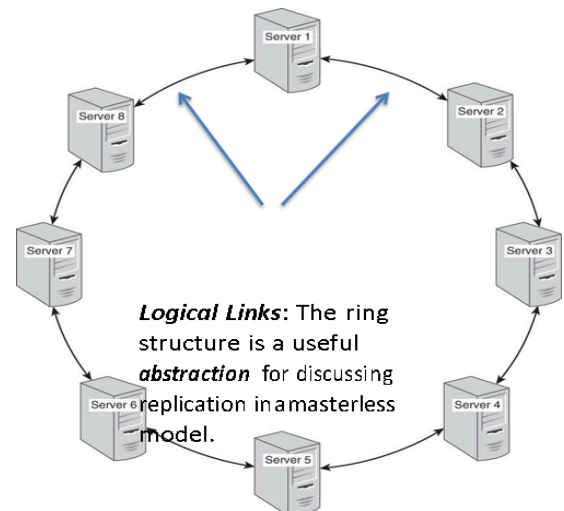
2. If the master crashes, the slaves do not receive updates anymore (*or any other message from the master*).
3. If several slave servers *do not receive any response* from the master within some period, the slaves may determine that the master has failed.
4. The slaves initiate a protocol to *promote* one of the slaves as a *new master*.
5. A new master has been elected so the system continues to work normally.

Masterless replication

The *master-slave replication* architecture will suffer an increasing number of write requests, because only the master accept writes and all the updates must be sent to the slaves. *Solution*: a replication model in a cluster of servers where all nodes may accept write requests.

I don't have a single point of failure because there is *not* a single server that has the *master copy* of updated data (*the most updated one*), and there is not a single server that can copy its data to all other servers. **Each node helps its neighbours to have the updated copy of the database:**

1. A write request is sent to a random server of the cluster.
2. The server that responds to the write request is the master and it will send the update to his neighbours.
3. The neighbours will receive the write request, so they become master, and they do the same thing with their neighbours (*except the one from it received the update*).
4. Eventually, the update will propagate among the cluster.



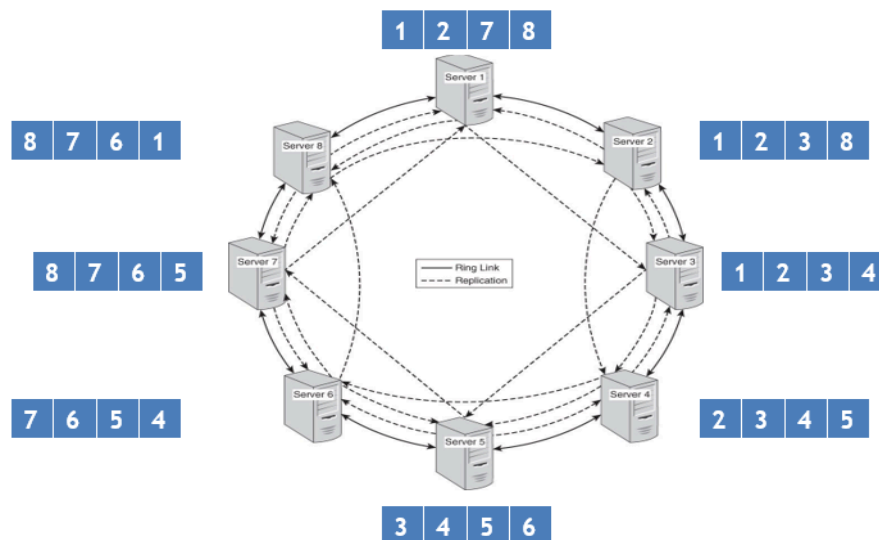
Suitable for applications in which the number of writes may increase and achieve peaks!
Example: the website which sells tickets to concerts.

Handling Replication: An Example

The database is divided in 8 chunks, when a server receives an update of a chunk, then he will forward the update to all the servers that contain the replicas of that chunk.

In this example every chunk is replicated three times; each time there is a write operation to one of the servers, the update is replicated to the three other servers holding its replica.

The rule for this example is: The chunk K have its main copy in the server K , and it is replicated in the server $K-1$, $K+1$ and $K+2$.



The ring structure is obviously a logic abstraction, in reality those servers are interconnected with a switch or a classic network. Usually, in a data centre, all the servers are connected to a single network hub and can communicate with each other.

Retrieve values on a KVDB

In general, keys may be any value/object (*since everything can be converted to a simple string*). But like in every memory, to access a certain value we need the address of the specific memory location in which this value is physically stored.

Hashing for selecting servers

Hash functions are usually used to obtain the memory address (*or the server where it is stored*) starting from the key. Given a key in input, the function returns a hexadecimal number (*that seems to be a random value*).

Note that (*depending on the hash function quality*) the values returned by hash functions (*given two different inputs*) may be not unique (*every Hash function has a certain degree of collision problems*).

Key	Hash Value
customer:1982737: firstName	e135e850b892348a4e516cfcb385eba3bfb6d209
customer:1982737: lastName	f584667c5938571996379f256b8c82d2f5e0f62f

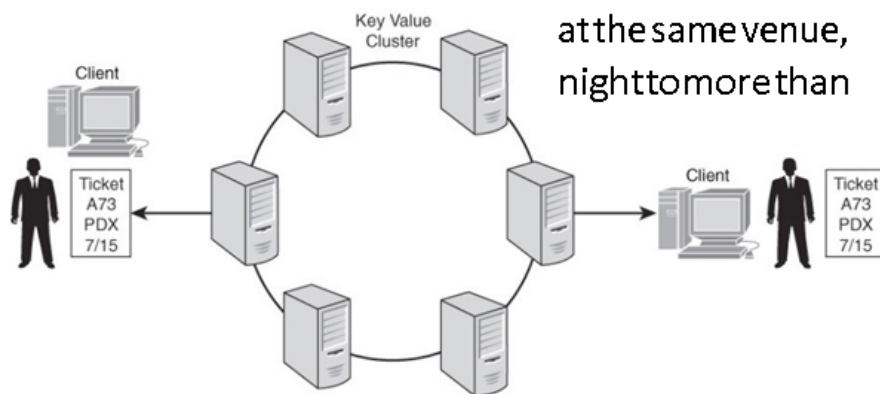
We would like to *balance* the write loads among the servers in a masterless replication architecture.

How can I choose the server of the cluster where I will send the write request for that value?

For this case we can use a simple hash function, like the modulus. *For the sake of simplicity, let consider the ID of a specific server as the address to identify.*

$$ServerNumber = |Hash\ Value|_8$$

Otherwise, the system should be studied for the case that we have to manage:



We want to avoid selling two tickets for the same seat, in the same city.

In the picture, two fans want to purchase the same seat: *A73 at the Civic Center in Portland, Oregon, on July 15*. We can use the following key for the specific ticket:

```
TICK : A73 : CivCen : PDX : 0715
```

Note that the key is generated starting from the entity that I want to memorise; in fact, it contains specific values inside.

Anyone trying to purchase that same seat on the same day would generate the same key and the hash function will return the same value. So, there is no chance for another server to sell that seat because the server who receives the write request will see that there is another record for the same key.

Characteristics of a KVDB

When choosing a certain key-value database, consider the *trade-off of various features*.

A certain key-value database might offer *ACID transactions* but set *limits* on the dimension of keys and values.

Another key-value database might allow for large values but limit keys to numbers or strings.

Our *application requirements* should be considered when weighing the *advantages* and *disadvantages* of different database systems.

KVDB operations and complex queries

All KVDBs offer 3 basic operations:

- Retrieve a value for the key
- Edit/insert a value for the key
- Delete a value for the key

If we want to do more, we will have to do it with an application program written by us that takes advantage of the 3 basic operations listed above.

Search of a value

Search for a specific value is a complex operation that must be made by hand with the basic operations.

```
appData[cust:9877:address] = '1232 NE River Ave, St.  
Louis, MO'
```

```
define findCustomerWithCity(p_startID, p_endID, p_city):  
begin  
    returnList = (); // Empty list.  
    for id in p_startID to p_endID:  
        address = appData[ 'cust': + id + ' :address' ];  
        if inString(p_city , Address):  
            addToList(Address , returnList);  
    return(returnList);  
end
```

Indexes in a KVDB

Some key-value databases incorporate a *support for searching operations* directly into the database, like indexes.

Word	Keys
'IL'	'cust:2149:state', 'cust:4111:state'
'OR'	'cust:9134:state'
'MA'	'cust:7714:state', 'cust:3412:state'
'Boston'	'cust:1839:address'
'St. Louis'	'cust:9877:address', 'cust:1171:address'

A built-in search system would index the string values stored in the database and create an *index* for rapid retrieval. The search system keeps a list of words with the keys of each key-value pair in which that word appears.

Join between namespaces

A namespace is a collection of buckets, which is a collection of key-value pairs.

It is not allowed to have duplicate keys in the same bucket of a namespace; but we can have the same key in different buckets of a namespace.

In NoSQL databases, data are not normalised, they depend on the type of the application, if we have customers and orders, and there's a relationship between them, we can create two different applications that use the same data, but they are represented in different ways (so *we'll still work with only one namespace*)

With different namespaces we can't merge information from different namespaces, only data in the same bucket can be used together (*because they share the namespace part of the key*), but nobody will stop us from doing some simple string manipulation in the code and using the two namespaces.

If we want relationships management and join operation, we should exploit a relational database instead. Creating different namespaces and using them for join operations is the biggest mistake we can make.

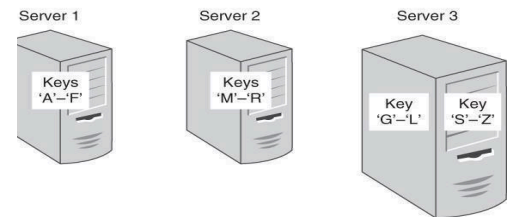
KVDB and clusters

A Cluster is a collection of independent servers.

Data Partitioning

We divide data in partitions we spread partitions among the different to evenly balance the load.

Different partitions may be handled by the different nodes. A node may contain more than one partition.



Partition key

A partition key identifies the specific partition where the value has been stored.

Any key (*or part of it*) in a key-value database can be used as a partition key: Usually, hashing functions are adopted for identifying the specific cluster or partition.

- A - F → Server 1.
- G - L → Server 3.
- M - R → Server 2.
- S - Z → Server 3.

Loosely Coupled Cluster

In Key-Value databases, clusters tend to be loosely coupled:

Minimal coordination between servers

We usually ensure minimal communication in order to deal with some failure situations, or if we have a ring organisation, we can select a server with a query, but if a server is not available, another one will replace it.

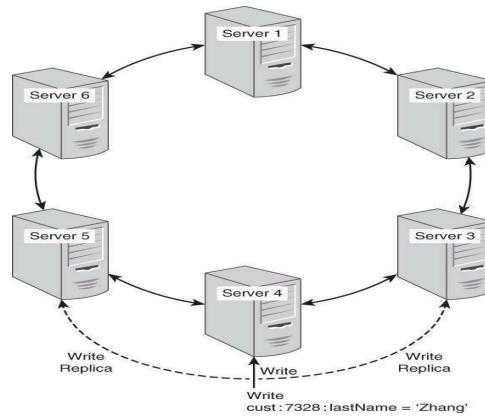
A server is independent and completes many functions on their own

Each server is responsible for the operations on its own partitions and routinely sends messages to each other to indicate they are still functioning. When a node fails, the other nodes in the cluster can respond by taking over the work of that node.

Replication of a KVDB

Let's consider a hashing function that generates a number from a key using the modulo operation, the first server is chosen following this function.

Whenever a piece of data is written to a server, a replica is written to the two servers linked to the first server (*neighbours*)



If Server 4 fails, then both Server 3 and Server 5 could respond to read/write requests for the data on Server 4. When Server 4 is back online, then Servers 3 and 5 can update it with the Write operation made while Server 4 was down.

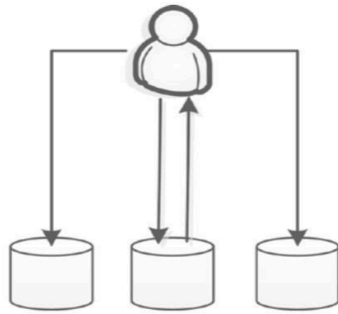
Trade-off for the number of replicas

A system is considered available if whenever we make a query, we receive an answer (*that might be not consistent*) by the database.

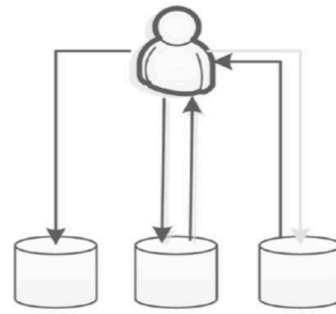
The higher the number of replicas, the less likely we will lose data, but the lower will be the performance of the system (*in terms of response time*).

Mind that during the replication process the system is not available.

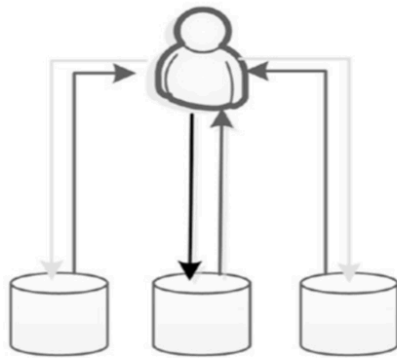
- N = Number of replicas.
- W = Number of copies to be written before the write can complete.
- R = Number of copies to be read for reading a data record.



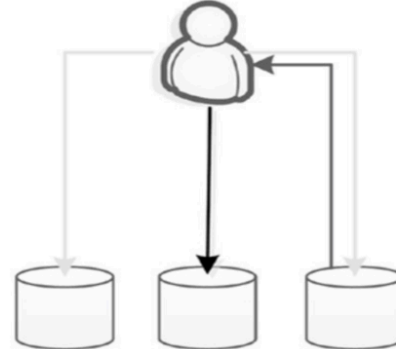
N=3 W=3 R=1
Slow writes, fast reads, consistent
 There will be 3 copies of the data.
 A write request only returns when all 3 have written to disk.
 A read request only needs to read one version.



N=3 W=2 R=2
Faster writes, still consistent (quorum assembly)
 There will be 3 copies of the data.
 A write request returns when 2 copies are written – the other can happen later.
 A read request reads 2 copies make sure it has the latest version.



N=3 W=1 R=N
Fastest write, slow but consistent reads
 There will be 3 copies of the data.
 A write request returns once the first copy is written – the other 2 can happen later.
 A read request reads all copies to make sure it gets the latest version.
 Data might be lost if a node fails before the second write.



N=3 W=1 R=1
Fast, but not consistent
 There will be 3 copies of the data.
 A write request returns once the first copy is written – the other 2 can happen later.
 A read request reads a single version only: it might not get the latest copy.
 Data might be lost if a node fails before the second write.

Hash mapping

In the example below, the hash value is a number in hexadecimal format.

Key	Hash Value
customer:1982737: firstName	e135e850b892348a4e516cfcb385eba3bfb6d209
customer:1982737: lastName	f584667c5938571996379f256b8c82d2f5e0f62f

One of the important characteristics of hash algorithms is that even small changes in the input can lead to large changes in the output. Hash functions are generally designed to distribute inputs evenly over the set of all possible outputs.

Let's assume that we have a cluster of 16 nodes and each node is responsible for one partition.

The key 'cust:8983:firstName' has a hash value of

4b2cf78c7ed41fe19625d5f4e5e3eab20b064c24

would be assigned to partition 4.

The key 'cust:8983:lastName' has a hash value of

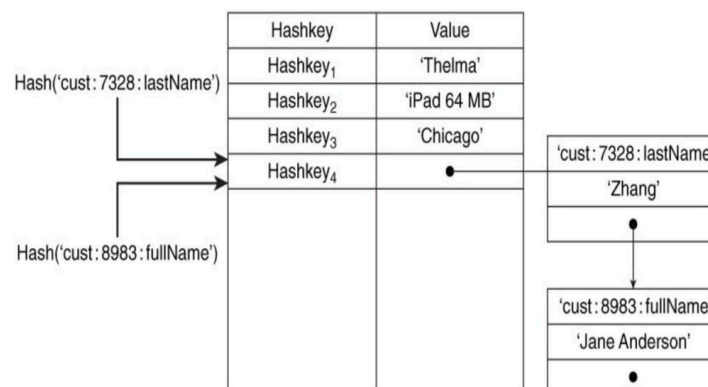
c0017bec2624f736b774efdc61c97f79446fc74f

would be assigned to partition 12 (*c is the hexadecimal digit for the base-10 number 12*).

Even if we have small changes on the value in input of the function, we have a different value as output, there's no problem if we have similar keys distributed in servers, it depends on how we use that key.

Collision Resolution Strategy – Open Hashing

From a logical point of view, the table that projects a hashed key to the corresponding value may include a list of values. In each block of the list, also the original key must be present. Note that collisions are an uncommon event.

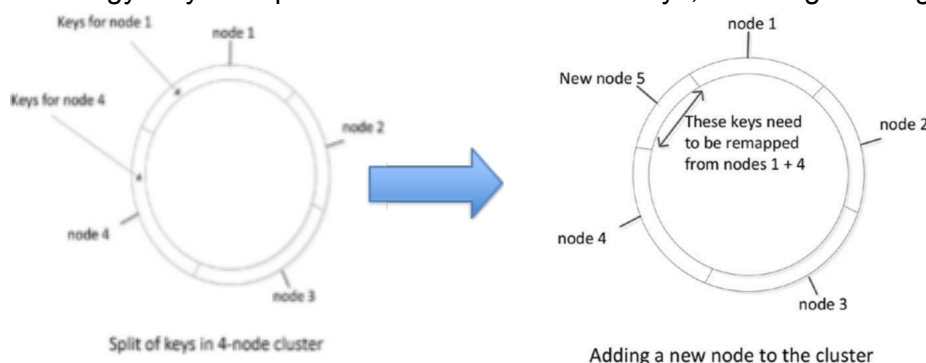


Relocation of Data - Consistent Hashing

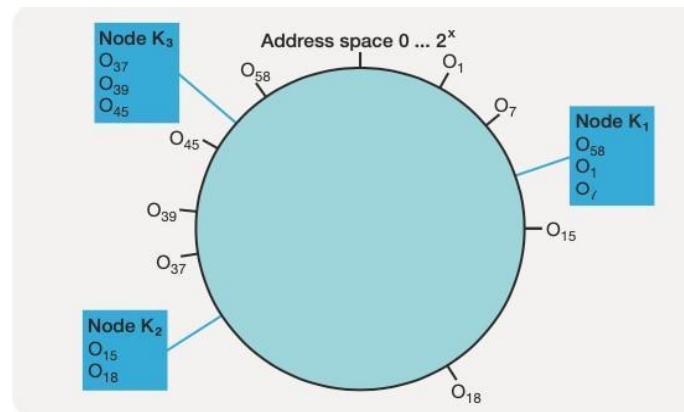
When we remove or add a server, the data inside the partitions of all the other servers must be relocated. During this operation the system is not available.

Consistent hashing is a strategy that permits to remap any the keys of the new nodes and their neighbours without remap the entire dataset.

This strategy only remaps the minimum number of keys, following this ring structure:



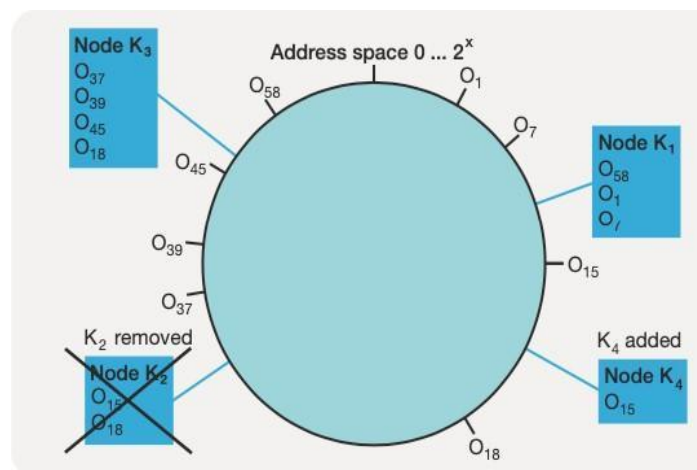
Let's suppose that the value of the hashing function goes from 0 to 2^x (where x is the number of nodes).



The node k_i is associated with some objects (in the case of K_3 , it is associated with O_{37} , O_{39} and O_{45}).

In the graph: All the object that follow the node are stored in him (reverse clockwise).

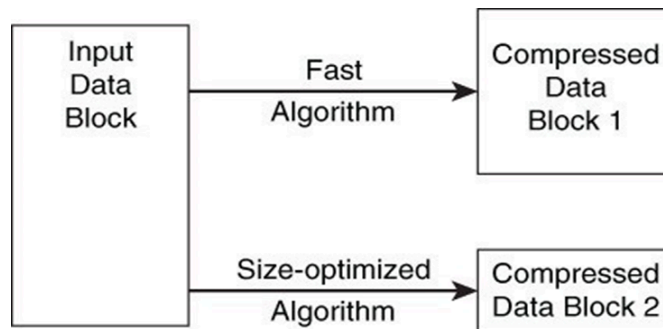
When we need to update the ring, we follow this rule: "Moving the ring in reverse clockwise, the node takes all objects until it finds another node".



When we add a new node K_4 : O_{15} is removed from K_2 , and it is added to the node K_4 .
When we remove node K_2 : O_{18} has been relocated in K_3 .

Data compression for KVDB

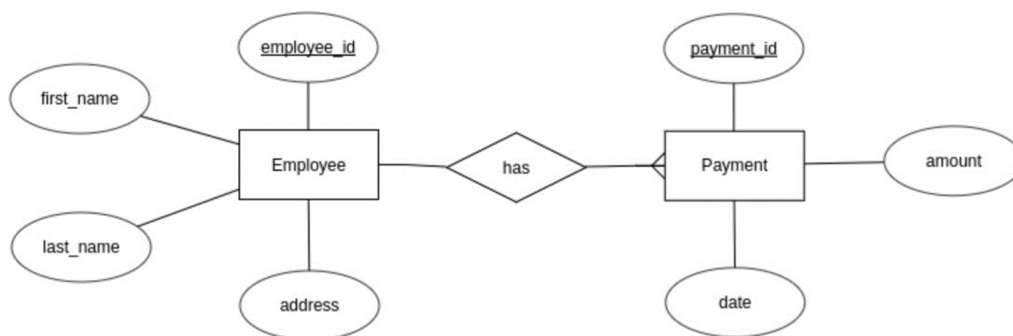
One way to optimise the disk management (in capacity and speed) is to use a data compression technique.



We are looking for a compression algorithm that ensures a trade-off between the speed of compression/decompression and the size of the final compressed data.

Design of a KVDB

Relational to KV



In this case we have an Employee associated with payment with a one-to-many relationship.

In a relational database this one-to-many relationship would be handled by a foreign key, so with the functional dependency constraints.

employee_id	first_name	last_name	address
1	John	Doe	New York
2	Benjamin	Button	Chicago
3	Mycroft	Holmes	London

FOREIGN KEY

payment_id	employee_id	amount	date
1	1	50,000	01/12/2017
2	1	20,000	01/13/2017

The first thing to do is to define the key format: take in mind that keys in a key-value database embed information regarding Entity Name, Entity Identifier and Entity Attributes.

If an entry doesn't have a value for an attribute (*for example the address*), it's an error to put this association with `null` value. The solution: Don't put an information if it isn't associated with a value.

We create only one namespace with one bucket; in order to avoid join operations among buckets or namespaces.

employee_id	first_name	last_name	address
1	John	Doe	New York
2	Benjamin	Button	Chicago
3	Mycroft	Holmes	London



employee:\$employee_id:\$attribute_name = \$value

employee:1:first_name = "John"
employee:1:last_name = "Doe"
employee:1:address = "New York"

employee:2:first_name = "Benjamin"
employee:2:last_name = "Button"
employee:2:address = "Chicago"

employee:3:first_name = "Mycroft"
employee:3:last_name = "Holmes"
employee:3:address = "London"

For building the key we must concatenate the \$employee_id with \$attribute_name, and for every attribute we have the association with the value.

Now we have to translate the Payment table. Also, in this case we have to manage a one-to-many relationship.

< payment:\$payment_id: \$employee_id: \$attribute_name > = \$value

payment_id	employee_id	amount	date
1	1	50,000	01/12/2017
2	1	20,000	01/13/2017
3	2	75,000	01/14/2017
4	3	40,000	01/15/2017
5	3	20,000	01/17/2017
6	3	25,000	01/18/2017



payment:1:1:amount = "50000"
payment:1:1:date = "01/12/2017"

payment:2:1:amount = "20000"
payment:2:1:date = "01/13/2017"

payment:3:2:amount = "75000"
payment:3:2:date = "01/14/2017"

payment:4:3:amount = "40000"
payment:4:3:date = "01/15/2017"

payment:5:3:amount = "20000"
payment:5:3:date = "01/17/2017"

payment:6:3:amount = "25000"
payment:6:3:date = "01/18/2017"

KVDB Design – Tips and Tricks

Key pattern

A well-designed key pattern minimises the amount of code (*and the number of errors*) a developer needs to write to create functions that access and set values.

The keys should be meaningful

In a KVDB is used to retrieve a value and must be *unique* in a specific bucket of a namespace.

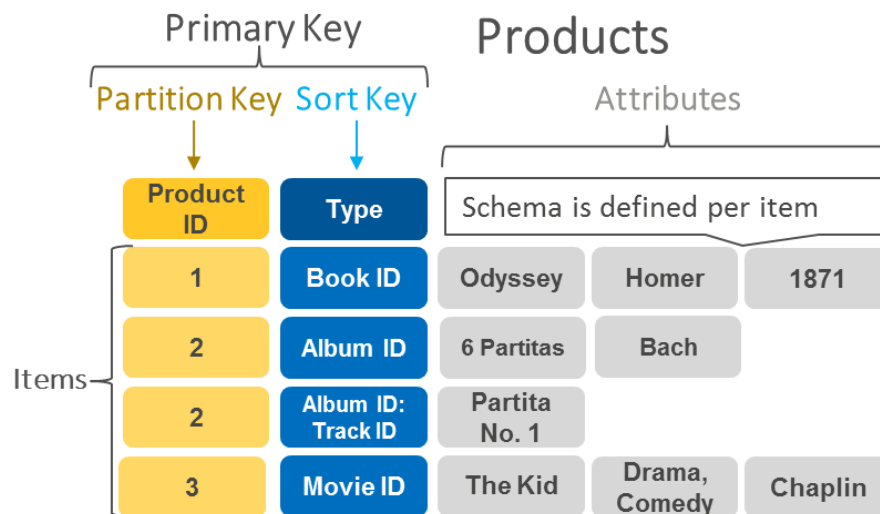
When designing *key-value databases*, adopting meaningful keys is *preferable*. A good practice is to build keys by embedding the entity information like: Name, identifier, or other attributes.

The keys should not be too long or too short

Long keys will use more memory, meanwhile short keys are more likely to lead to conflicts in key names.

The keys should follow a format

For each key, we may store in the database a specific value, so we have a key for identifying every attribute of the entity.



The number of attributes of an entity isn't always predictable, but we can add new attributes by simply changing the format of the key (*the programmer decides it*).

Note that we don't have to define these keys in advance, it oversees the programmer to do this, we have a document with the list of the attributes and the key to an entity. It is normal to make some changes to the key format during the programming of the application.

Before deciding the format of the key, you must consult the *documentation* for limitations on keys and values.

Single entity, no relationships.

A key should be defined on the entity itself not on the relationships with other entities.

The key should contain values that don't change.

Like the name or the date of birth, but not the amount of money in the bank.

Get & Set methods

A class has attributes and methods, among the methods I expect to have the set and the get method for every attribute (*get for read and set for write*).

Using generalised get and set functions help improve the readability of code and reduces the repeated use of low-level operations, such as concatenating strings and looking up values.

Inside the key we can use range-based components to do queries, for example if we use a date as a string in the key, we can do a loop considering the interval of the date.

```
define getCustAttr(p_id, p_attrName)
    v_key = 'cust' + ':' + p_id + ':' + p_attrName;
    return(AppNameSpace[v_key]);
```

```
define setCustAttr(p_id, p_attrName, p_value)
    v_key = 'cust' + ':' + p_id + ':' + p_attrName
    AppNameSpace[v_key] = p_value
```

The namespace is accessed like an associative array!

Dealing with range of values

Retrieve all customers who made a purchase on a particular date. First, we should add some info into the key:

```
cust : p_date : p_id : custId
```

- The date of purchase (`p_date`), to quickly obtain information about the customer that made that specific purchase.
- The product id (`p_id`).
- The customer id (`custId`).

Then I loop the array looking for the keys that contain the value of the date.

```
define getCustPurchByDate(p_date)
    v_custList = makeEmptyList();
    v_rangeCnt = 1;

    v_key = 'cust:' + p_date + ':' + v_rangeCnt +
        ':custId';
    while exists(v_key)
        v_custList.append(myAppNS[v_key]);
        v_rangeCnt = v_rangeCnt + 1;
        v_key = 'cust:' + p_date + ':' + v_rangeCnt +
            ':custId';

    return(v_custList);
```

1. We initially create an empty list of customers.
2. Initialize the range (`v_rangeCnt`) to 1, it is the first element of the list (*it's the counter of purchases*).
 - a. **1** it means: *“that customer was the first to make a purchase in that date”*.
3. We create the key that permits us to retrieve the data we want.
4. Loop:
 - a. Check if the key exists (*if there is an entry*) with `exists(v_key)`
 - b. If the key exists, we retrieve the values `.append(myAppNS[v_key])`

- c. Increment `v_rangeCnt` and update the key.

Sometimes it is requested in the exam to write a code like this.

The only way to look up for values is by the key, so it's not ideal when I must do many range queries.

We may have some DBMS for key-value that offers some functionalities, usually as basic methods offered, we can search by key, delete by key and update by key.

The problem is that there are not standard languages like in SQL, to do those operations.

Retrieve whole entities and complex structures

We want to retrieve the whole customer. So, we will use the `get` method for retrieving all attributes I need and then return them as a list of values or a JSON string.

```
define getCustNameAddr(p_id)
    v_fname = getCustAttr(p_id, 'fname');
    v_lname = getCustAttr(p_id, 'lname');
    v_addr = getCustAttr(p_id, 'addr');
    v_city = getCustAttr(p_id, 'city');
    v_state = getCustAttr(p_id, 'state');
    v_zip = getCustAttr(p_id, 'zip');
    v_fullName = v_fname + ' ' + v_lname;
    v_fullAddr = v_city + ' ' + v_state + ' ' + v_zip;
    return(makeList(v_fullName, v_fullAddr);
```

How many accesses to the database?

In this case we make 6 accesses to the database (*because we called the `getCustAttr()` 6 times*).

Note that a key-value database usually stores the entire list together in data blocks, thus just one block in the disk will be accessed in RAM rather than 6.

But when we do horizontal scaling those six keys could be on different servers, so with the access time I will have latency too.

How can I improve the situation? Unique key

When I am designing the database, I must think about HOW the data will be used, because the database should be organised for the query well 'going to do.

If I realise that I always use this function (*that invokes the `get` method for six times so I will do six access to the database*) then I should define a key that contains all this info together. So, I can retrieve the whole entity with only one look-up, but this redundancy has a cost:

- I must maintain the redundant information always consistent.
- The higher is the dimension of the object, the higher is the probability that this object is distributed in different servers and in different blocks of the disc.

Put a JSON file inside the key-value database

We can do this, but the higher is the dimension of the object, the higher is the probability that this object is distributed in different servers and in different blocks of the disk.

We are not sure that information is stored in the same block of the disc, and even if it is stored in the same server, so with key-value DB we don't have to use complicated data structures.

Case Study – Restaurant

We want to exploit the system for take and keep reservations.

Let's suppose that in one date "yyyy-mm-dd" we can have only one reservation for table.

For each table we have the maximum number of people that can seat in that table, it is a fixed value for each table.

How the system should work

If you make a click on a table booked you have some information about the name and the telephone number of the customer, the DB maintains this information only for the time needed for the reservation and not in a persistent way, on the screen we have also the date.

For the write operations, we also have tables, and we can select a table and insert the seats, the name of the customers and the telephone number, there's also the date.

How can we format the key?

```
Res : $id_table : $date : attribute
```

We can't have two reservations in the same table at the same date, so those two values can identify the reservation (without exploit a reservation_id).

Which attribute can I put?

We need to keep the reservation description, so we can add a new attribute called \$res_des.

```
$res_des is like {Customer name , Cell number , Number of people}
```

At the end we got those entities

```
Res : $id_table : $date : $res_des
```

```
Tab : $id_table : $maximumPeople
```

Mind the requirements! the system is used ONLY for make and see the reservations , for check a reservation I have too many details.

Can I organize the database like a calendar?

So, I add another key, that show a different information:

```
Res : $id_table : $date : $status
```

If status = 0 then the table for that date is available; otherwise, the attribute tells me the number of people of the reservation.

Document Databases

Document databases are non-relational databases that store data as structured documents, usually in XML or JSON formats.

They ensure a high flexibility, and, despite KVDB, they have more support to handle complex data structures, complex operations and sometimes ACID operations (*like MongoDB*).

What do we need?

Complex analysis

KVDB ensure fast response and flexibility, but we would need to make complex analyses maybe using some other attribute than the key.

High Flexibility

We have to ensure high flexibility but, in this case, we want to use complex data structures too, we can define the attributes each time we need, whenever we need because the architecture is schema-less.

XML Documents Databases

XML stands for eXtensible Markup Language.

A markup language (*like HTML*) specifies the structure and content of a document but also the meaning (*a document is a file which contains some text*).

Tags are added to the document to provide the extra information, for example it can contain an attribute of an object.

Properties of XML

- XML tags give a reader some idea what some of the data means.
- XML can represent almost any form of information.
- XML was used a lot in the past for encapsulation and transmission of data, between client and server.
- XML is mostly used in web2.0 because together with CSS gives the possibility to implement new generation websites, XML was also used a lot in the past decades for encapsulating data that are transmit from a client to a server and vice versa, for example with data interchange protocols like SOAP.
- XML is used for formatting documents such as word documents, excel documents and PowerPoint documents.

Pros of XML

XML is text (*Unicode*) based, so it is easy to transmit, and then we can copy text from a file to another, so it has high modularity, we can create separate modulus and merge them together.

One XML document can be displayed differently in different media and software platforms.

XML documents can be modularized, so its parts can be reused for other XML documents.

The code in the image describes three albums.

Each album contains: the title, the name of the artist and all the names of all tracks inside the album.

Inside a track we can put some properties, for example the length of the song.

May exist several APIs that allow us to read this file, the docx file in Microsoft Word is an XML file too.

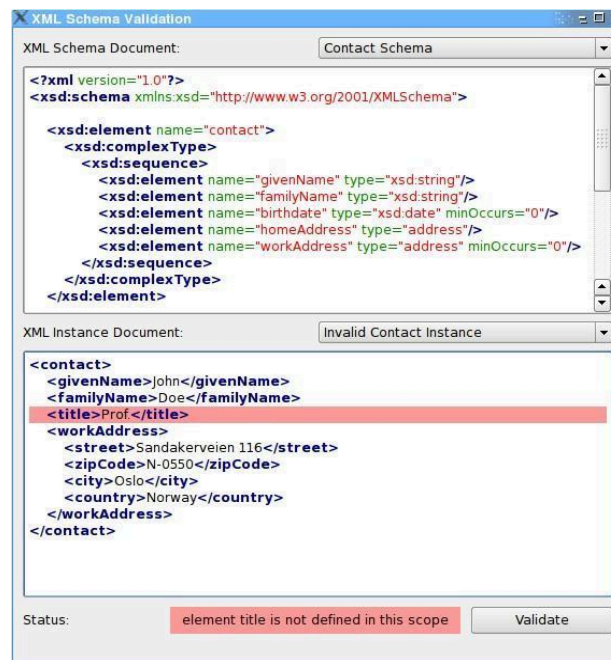
XML Documents: Use cases

- XML (*like* CSS) allows separating data and format of the document.
- XML is the basis for many data interchange protocols and was a foundation for web service specifications such as SOAP (*Simple Object Access Protocol*).
 - The SOAP protocol is the ancestor of the Restful API.
- XML is a standard format for many document types, including word processing documents and spreadsheets (*docx, xlsx and pptx formats are based on XML*).

```
<item>
  <title>Kind of Blue</title>
  <artist>Miles Davis</artist>
  <tracks>
    <track length="9:22">So what</track>
    <track length="9:46">Freddie Freeloader</track>
    <track length="5:37">Blue in Green</track>
    <track length="11:33">All Blues</track>
    <track length="9:26">Flamenco Sketches</track>
  </tracks>
</item>
<item>
  <title>Cookin'</title>
  <artist>Miles Davis</artist>
  <tracks>
    <track length="5:57">My Funny Valentine</track>
    <track length="9:53">Blues by Five</track>
    <track length="4:22">Airegin</track>
    <track length="13:03">Tune-Up</track>
  </tracks>
</item>
<item>
  <title>Blue Train</title>
  <artist>John Coltrane</artist>
  <tracks>
    <track length="10:39">Blue Train</track>
    <track length="9:06">Moment's Notice</track>
    <track length="7:11">Locomotion</track>
    <track length="7:55">I'm Old Fashioned</track>
    <track length="7:03">Lazy Bird</track>
  </tracks>
</item>
```

XML Ecosystem

XML includes or actually included (we aren't sure that it includes all the features in the list).



XPath

Useful to navigate through elements and attributes in an XML document.

XQuery

Is the language for querying XML data and is built on XPath expressions.

XML schema Document

A special type of XML document that describes the elements that may be present in a specified class of XML documents. Often used for validating the structure of an XML file.

In the picture on the right, I can see an example of usage of an XML scheme. Each information has to be embedded in an open and close tag; therefore, each tag could have other properties.

Every XML document represents an object.

In this example, if we have a telephone book, each element is a contact and it must have some attributes, if you create an attribute prof, you'll have an error because prof is not included in the list of the attributes defined before.

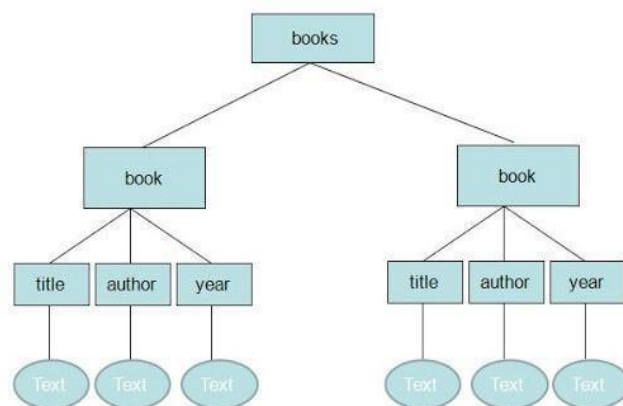
DOM (Document Object Model)

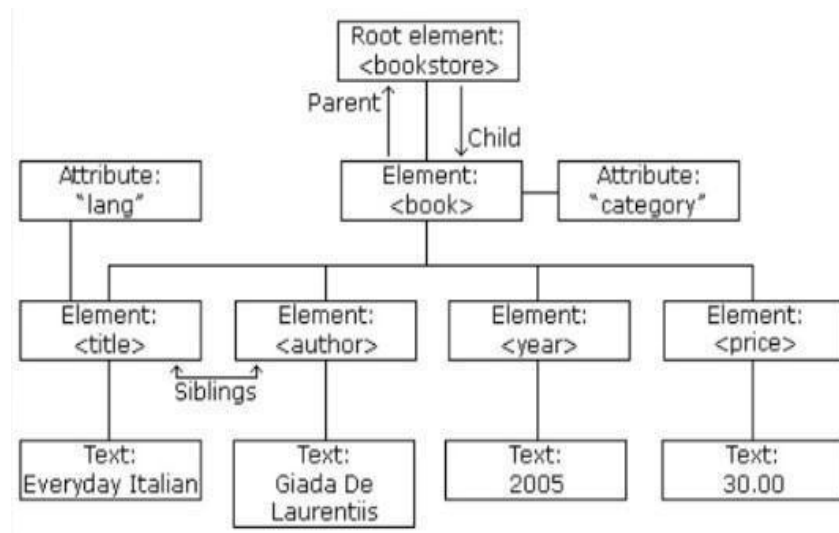
A platform and language-neutral interface for dynamically managing the content, structure, and style of documents such as XML and XHTML.

A document is handled as a tree.

An XML file can be stored as a binary file, a file is a set of bytes, we can represent an XML like a tree in memory and in the disc.

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<bookstore>
  <book category="cooking">
    <title lang="en">Everyday Italian</title>
    <author>Giada De Laurentiis</author>
    <year>2005</year>
    <price>30.00</price>
  </book>
  <book category="web" cover="paperback">
    <title lang="en">Learning XML</title>
    <author>Erik T. Ray</author>
    <year>2003</year>
    <price>39.95</price>
  </book>
</bookstore>
```





Do we have XML available databases?

Some frameworks proposed in the past and a general architecture of XML based databases had this three levels architecture:

XML Databases

XML databases: platforms that implement the various XML standards such as XQuery and XSLT,

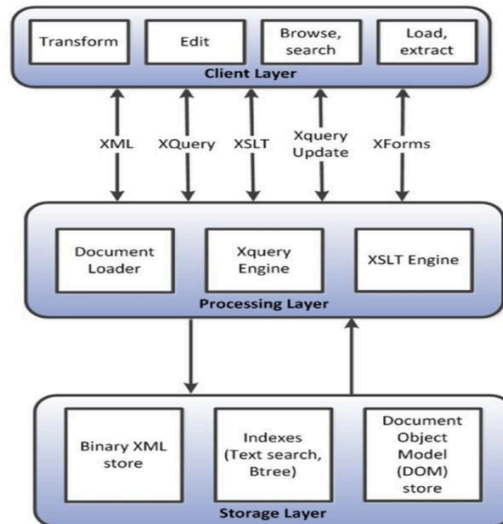
They provide services for the storage, indexing, security, and concurrent access of XML files.

The storage layer is for storing persistently data, in this layer we have indexes too.

The storage layer can be accessed by a set of applications engines which provide services to the clients.

XML databases did not represent an alternative for RDBMSs.

On the other hand, some RDBMSs introduced XML, allowing the storage



Main Drawbacks of XML

- XML tags are verbose and repetitious, thus the amount of storage required increases, due to, for example, the close tag.
- XML documents are wasteful of space and are also computationally expensive to parse.
- When I have too much stuff in an XML file, it becomes very big, so in case of a network transmission it can be a problem.

Is XML still used?

XML format and language was very useful some time ago with the web2.0, however XML is too verbose and with a lot of repetitions: every tag occupies memory, so it is computational expensive to do operations with objects in XML files.

Are XML databases still used?

In these years, XML databases are used as content-management systems: collections of text files (*such as academic papers and business documents*) are organised and maintained in XML format (*like word files*).

JSON (*Javascript Object Notation*) documents

Thanks to its integration with JavaScript, a JSON document has been often preferred to an XML document for data interchanging on the Internet. An example of JSON document:

```
{
  "nome": "Mario",
  "cognome": "Rossi",
  "eta": 30,
  "indirizzo": {
    "via": "Via Roma, 123",
    "citta": "Milano",
    "cap": "20100"
  },
  "telefoni": [
    {
      "tipo": "cellulare",
      "numero": "123-456-7890"
    },
    {
      "tipo": "ufficio",
      "numero": "555-123-4567"
    }
  ]
}
```

A JSON file is a collection of key-value associations, the key is often a name, so there are name-values associations.

A JSON database is similar to a KVDB: indeed “*web*” and “*database*” in the image on the right can be seen as two keys and the values of each key are vectors.

Comparison of JSON and XML

Similarities

- Both are human readable.
- Both have very simple syntax.
- Both are hierarchical.
- Both are language independent.
- Both are supported in APIs of many programming languages.

Differences

- Syntax.
- JSON is less verbose.
- JSON includes arrays.
- Names in JSON must not be JavaScript reserved words.

Features of JSON Databases

Data is stored in JSON documents and High flexibility

A document is the basic unit of storage (*in relational DB the basic unit is a row*).

It includes one or more key-value pairs and may also contain nested documents and arrays (for a more complex hierarchical structure).

Collections and Polymorphic Scheme

A collection is a set of documents sharing some common purpose (*given by the developer*).

The documents in a single collection may be different, but they are used for the same purpose.

Schema less

You don't have to have predefined document elements.

Whenever we want a schemeless database, the designer of the application can write functions for removing, inserting, and updating values without modifying the database.

We may decide to add a new document without modifying the others.

Document databases: data modelling

Schemeless property

A schema is a specification that describes the structure of an object, such as a table. Every object of that group must respect the predefined scheme.

Data modellers must define tables in a relational database before developers can execute code to add, remove, or update rows in the table.

Document databases do not require this formal definition step where we create the schema. Developers can create documents in collections by simply inserting them.

Pros of a schemeless database

High flexibility in handling the structure of the objects to store.

Cons of a schemeless database

Nobody will check if I respect constraints given by the requirements. In fact, a relational database automatically does checks on the type of values and on relationships, because we define the schema before inserting the data.

Some considerations on the third normal form

A document database could theoretically implement a third normal form schema. Tables may be “*simulated*” considering collections with JSON documents with an identical pre-defined structure.

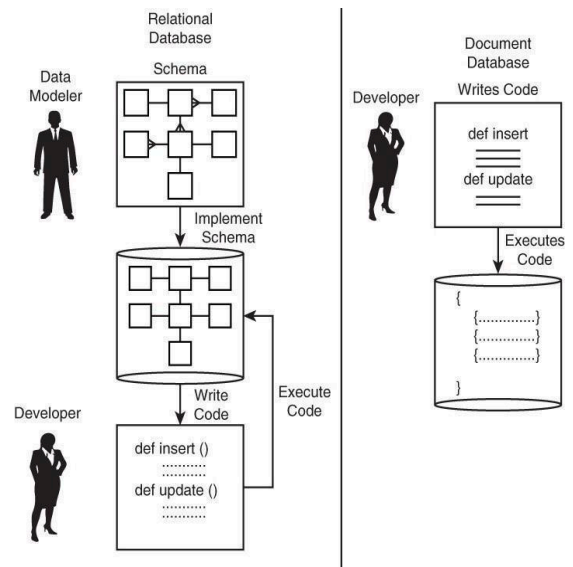
So, with a document database, you can simulate a third normal form schema, but in the project this trick cannot be exploited (*or the project will be rejected*).

How to deal with relationships among entities on a JSON Database

Document Embedding

Document databases usually adopt a reduced number of collections for modelling data. The red part is the embedded document.

```
{  
  "name": "Mario",  
  "phoneNumbers": [  
    {  
      "type": "Mobile",  
      "number": "123-456-7890"  
    }  
  ]  
}
```



```

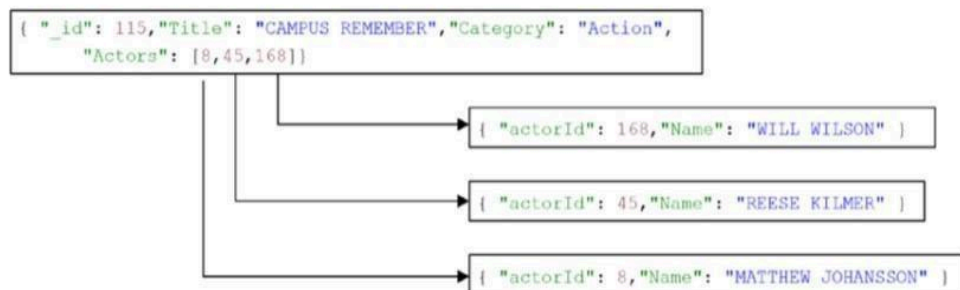
    },
    {
      "type": "Home",
      "number": "987-654-3210"
    }
  ]
}

```

Nested documents are used for representing “one to many” relationships among different entities.

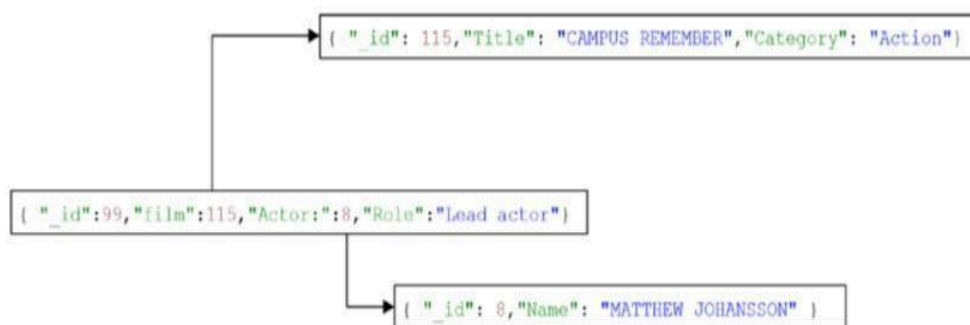
Like in the code above, a person can have multiple phone numbers, and each phone number is represented with a document embedded in the main person document.

Document Linking



An array of actor IDs has been embedded into the main film document.

This kind of strategy can be used in some particular situations, especially when we got “many to many” relations. The retrieved IDs can be used to retrieve the documents of the actors (*in another collection*) who appear in a film.



We are rolling back to the third normal form!

In fact, document linking approaches are usually somewhat an unnatural style for a document database. However, for some workloads it may provide the best balance between performance and maintainability (*embedded document may be updated*).

When modelling data for document databases, there is no equivalent third normal form that defines a “*correct*” model.

In a schemeless database the queries drive the approach to model data, if I have to deal with many and complex relationships, I should choose a relational one instead.

How to model a “one-to-many” relationship

```
Person{  
  "name": "Mario",  
  "phoneNumbers": [  
    {  
      "type": "Mobile",  
      "number": "123-456-7890"  
    },  
    {  
      "type": "Office",  
      "number": "555-123-4567"  
    }  
  ]  
}
```

The basic pattern is that the *one* entity in a “*one-to-many*” relation is the *primary document* (in this case the `person`), and the *many* entities (in this case the `phoneNumbers`) are represented as an array of *embedded documents*.

This is not fully normalised (*often the embedded documents are part of data in another collection and do not depend on the key to the main entity*).

How to model a “many-to-many” relationship

In this example a `student` can be enrolled in many courses, and a `course` can have many `students` enrolled in it.

```

{
  {studentID: 'S1837',
    name: 'Brian Nelson',
    gradYear: 2018,
    courses: ['C1667', 'C2873', 'C3876']},
  {studentID: 'S3737',
    name: 'Yolanda Deltor',
    gradYear: 2017,
    courses: ['C1667', 'C2873']},
  ...
}

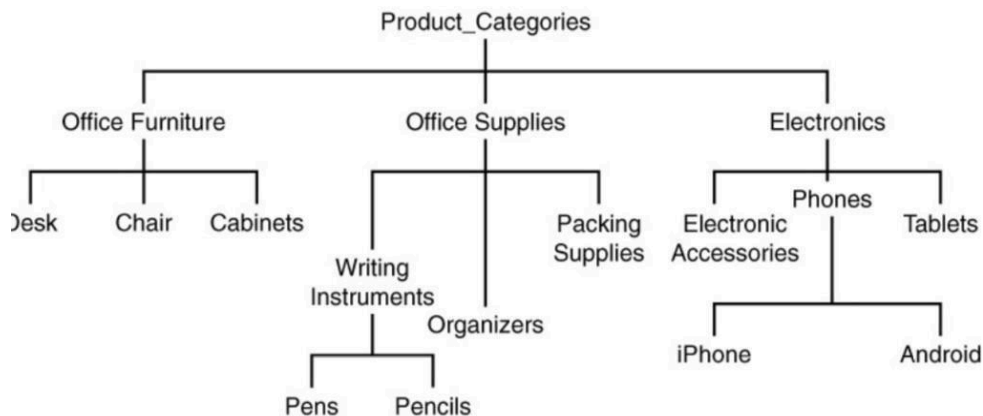
{
  { courseID: 'C1667',
    title: 'Introduction to Anthropology',
    instructor: 'Dr. Margret Austin',
    credits: 3,
    enrolledStudents: ['S1837', 'S3737', 'S9825' ...
      'S1847'] },
  { courseID: 'C2873',
    title: 'Algorithms and Data Structures',
    instructor: 'Dr. Susan Johnson',
    credits: 3,
    enrolledStudents: ['S1837', 'S3737', 'S4321', 'S9825'
      ... 'S1847'] },
  ...
}

```

Indeed, the document DBMS will not control the referential integrity.

In this case is preferable to use Document Linking because when a professor makes the access, he can retrieve all the document related to him; so, using document linking improves the performance of the application. Document embedding it's not preferable because I could put too much stuff in only one object.

How to model hierarchies



We can exploit three types of solutions:

Parent-reference solution

This solution is useful if we have to frequently show a specific instance of an object and then show the more general type of that category.

```
{
  {productCategoryID: 'PC233', name:'Pencils',
    parentID: 'PC72'},
  {productCategoryID: 'PC72', name:'Writing Instruments',
    parentID: 'PC37'},
  {productCategoryID: 'PC37', name:'Office Supplies',
    parentID: 'P01'},
  {productCategoryID: 'P01', name:'Product Categories' }
}
```

Child-reference solution

This solution is useful if we frequently retrieve the children (or sub parts of them) of a specific instance of an object.

```
{
  {productCategoryID: 'P01', name:'Product Categories',

    childrenIDs: ['P37','P39','P41']],
  {productCategoryID: 'PC37', name:'Office Supplies',
    childrenIDs: ['PC72','PC73','PC74']],
  {productCategoryID: 'PC72', name:'Writing
    Instruments', childrenIDs: ['PC233','PC234']},
  {productCategoryID: 'PC233', name:'Pencils'}
}
```

List of Ancestor solutions

This solution allows retrieving the full path of the ancestors with one read operation.

```
{productCategoryID: 'PC233', name: 'Pencils',  
  ancestors: ['PC72', 'PC37', 'P01']}
```

A change to the hierarchy may require many write operations, depending on the level at which the change occurred.

Exercise - Trucks Company

Trucks in a company fleet must periodically transmit this information:

- Current location.
- Average fuel consumption
- Other metrics

The transmission is done every 3 minutes to a fleet management database (*one-to-many relationship between a truck and the transmitted details*). Trucks are moving only between 8:00 AM and 6:00 PM.

Before modelling, let's do some computations

We consider 20 transmissions per hour, and the truck works for 10 hours every day so at the end of the working day database will include 200 new documents for each truck. We can create a document for every transmission: each document contains:

```
Update{  
  
  "truckID": 23,  
  
  "driverID": "MarioRossi00",  
  
  "timestamp": "2023-08-08T17:15:00",  
  
  "AVGFuelConsumption": "4 mpg"  
  
  "Location": <coordinates>  
  
}
```

Every three minutes the client creates this document and transmits it to the company server.

This information has a lot of redundancies: If we look two consecutive update the red fields are always the same.

Alternative solution

Put the variable information is embedded into `operational_data`, in this way it is easier to retrieve information in this situation rather than the previous case in which we had to scan all the information.

This solution may produce enormous documents! mind that the maximum size of the single document has a limit (*in MongoDB the limit is 16Mb per document*).

```
Update{
```

```

"truckID": 23,

"driverID": "MarioRossi00",

"date": "2023-08-08"

Operational_data{

"time" : "17:15:00",

"AVGFuelConsumption": "4 mpg"

"Location": <coordinates>

}

```

Planning for mutable documents

Documents relocation

It's important to have an idea of the maximum dimension of the documents in my database.

When a document is created, the DBMS allocates a certain amount of space (*equal to initial space of document + fixed more memory*) for it into the disk.

If the document grows more than the allocated space:

1. The DBMS search (*and possibly free*) another space in the disk.
2. Relocate (*cut and paste*) the document.

This operation is transparent at high level, but it is very time-consuming.

A solution for avoiding moving oversized documents is to allocate sufficient space now in which the document is created.

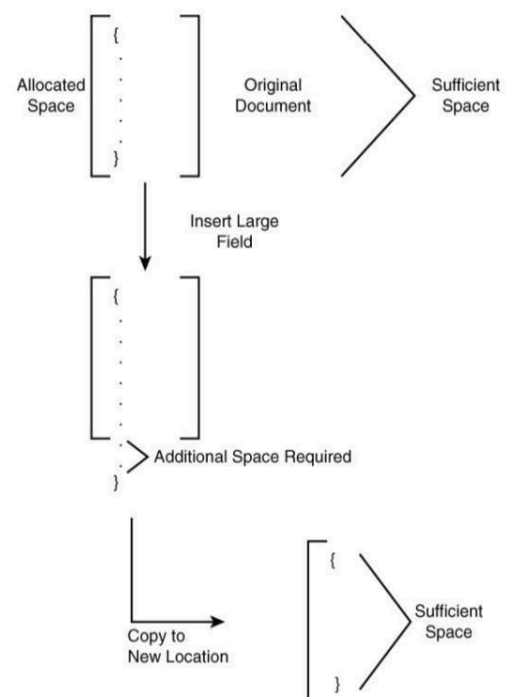
Life cycle of a document

We must plan, if possible, the strategies for handling its growth. Obviously, it strongly depends on requirements. For example (*every year, delete the reservation of the previous year*).

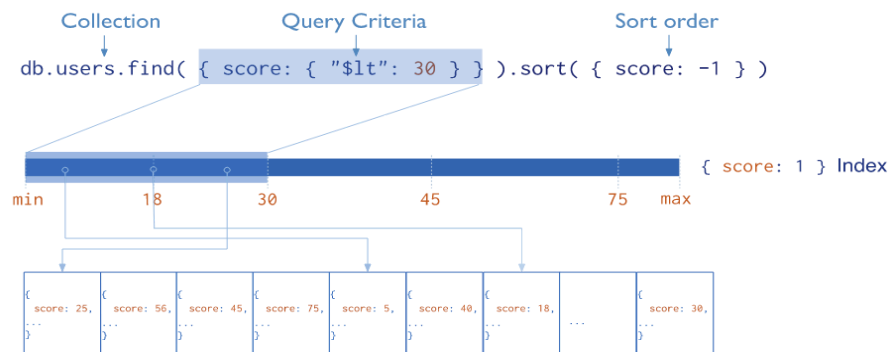
Indexing a document database

To avoid the entire scan of the overall database when I search a certain document, DBMSs for document databases (*for example MongoDB*) allow the definition of *indexes*.

Indexes are **special data structures that store a value of one (or more) field**, ordered by the value of the field (*or fields*) in order to **permit a direct access to it (faster read)**.



Often an index is implemented like a set of pointers to documents.



An index is exploited depending on the match criteria. If the index is created on the field A and the match criteria of the query regard the field A, then the index will be exploited.

Compound indexes

Indexes defined on more than one attribute.

In this case the query has to present the attribute (*in the match field*) in the same exact order defined by the compound index in order to exploit it.

Applications and indexes

Read-Heavy Applications

The use of several indexes allows the user to quickly read something in the database.

More indexes □ Faster read operations □ Slower write operations

Write-Heavy Applications

The higher the number of indexes adopted the higher the amount of time required for perform a write operation, because when I modify the data all the indexes defined on that data must be updated (*and created at the beginning*).

Less indexes □ Faster write operations □ Slower read operations

Trade-off solution

The number, and the type of indexes to adopt must be identified as a trade-off that depends on the estimated number of reads and writes.

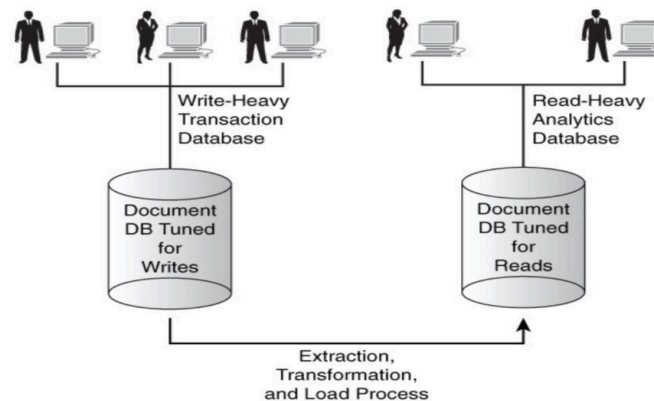
Transactions Processing Systems

In general, we can't optimize reads and write at the same time. Let's introduce two types of systems:

- **OLTP:** OnLine Transaction Processing (*write intensive*).
- **OLAP:** OnLine transaction Analytical Processing (*read intensive*).

For optimize both: We create two databases:

- One tuned for improving reads operations.
- One tuned for improving writes operations.



This solution is very useful in the cases where data doesn't need to be processed in real time (*maybe we don't use analytics every hour*). At night (*low level of load*) I can update the read-tuned one with the new information contained in the write-tuned one.

We may have different granularity of an event entity; we may not need to store in the Document DB each data that we have on the Analytic DB.

We can enforce CA on the server on the left and only AP on the right server (*CAP theorem*).

Collections

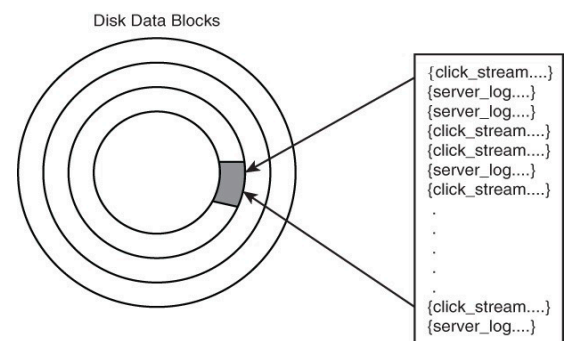
A collection is a set of documents sharing a collective purpose.

"MongoDB stores documents in collections. Collections are analogous to tables in relational databases."

Documents belonging to the same collection will be put inside the same disk block (*or near blocks*), so retrieving all documents of a single collection is a pretty fast operation.

In the figure below all the information is put together in the same disk block, this is nice if we use that information together (*for instance doing some analytic*).

But if you use this information separately then it's inefficient because each time you read a block (*reads in disk are made only on blocks and not on single records*) you must use a filter to retrieve only the data you want.



What if I use an index?

Indexes can solve the problem of having heterogeneous information in the same block, and this avoids making a filter after retrieving the block.

If we have a large collection, it is good to have a lot of indexes.

But the problem remains, so indexes can help but separating information would be better because if there is a lot of data and they are very different, indexes then become very inefficient, it's better to make a separate collection.

Manipulate a collection with code

In general, a code written for this purpose should have:

- A substantial amount of code that applies to all documents.
- Some amount of code that accommodates specialised fields in some documents.

High-Level Branching

Let's suppose that in the collection "log" we got two types of documents (`click_stream` and `server_log`).

We may have two branches in the code: one for the `server_log` and one for `click_stream`.

```
If (doc_type = 'click_stream'):
    process_click_stream (doc)
Else
    process_server_log (doc)
```

In this case we have a high-level difference between the two objects; because we want to separate stuff (*even inside the code of the application*), so we should create separate collections.

Low-Level Branching

We have a document `book` described by the parameters.

If `doc.ebook` is `true`, then only `book.descr` is different.

```
book.title = doc.title
book.author = doc.author
book.year = doc.publication_year
book.publisher = doc.publisher
book.descr = book.title + book.author + book.year + book.publisher
if (doc.ebook = true);
    book.descr = book.descr + doc.ebook_size
```

In this case we don't have to create another collection because there is a minimal difference between the two types of documents.

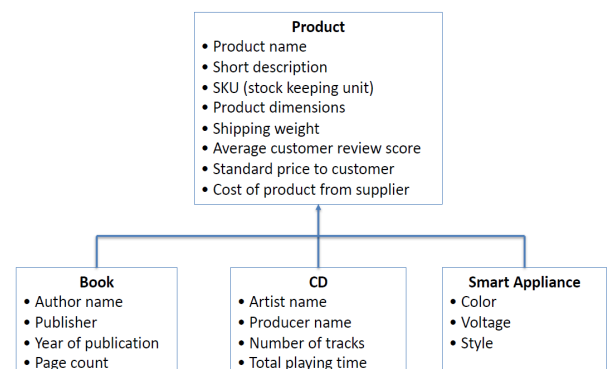
Queries and entities

We must know how many collections are required in our system, and how many fields must be included in each entity, both mandatory fields and optional fields.

When we define entities, we must follow the definition of the queries, so the functional requirements.

Let's suppose that we have these queries of functional requirements:

- What is the average number of products bought by each customer?



- What is the range of number of products purchased by customers
- What are the top 20 most popular products by customer state?
- What is the average value of sales by customer state?
- How many of each type of product were sold in the last 30 days?

We never read in the requirements about the specialisations of products (*there are not queries that discriminate the specialisation*). In this case we must maintain all the products in the same collection.

If we separate entities in different collections, and the number of entity types grows then the number of collections would become unwieldy.

So, we put all in the same collection, then we define one additional field in the product (*which is a document embedded on the first one*) that contains all the information about its specialisation.

Columnar Databases

They are strictly connected to relational databases. A column-oriented (or simply *columnar*) is a DBMS that stores data tables by column rather than by row (*remember that a column contains values of a certain type*).

Pros of columnar databases

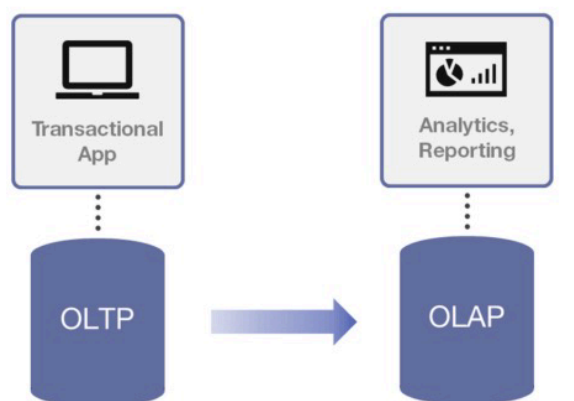
- More efficient access to data when we are only querying a subset of columns (*by eliminating the need to read columns that are not relevant*).
 - Typically, an analytic is computed using only one attribute, so retrieving all the values of a certain column requires one or few more access to the disk.
- More options for data compression.

Cons of columnar databases

Typically, less efficient for inserting new data: indeed, if you have to insert a new row, you have to add 1 value in each attribute column, so you may have to do multiple accesses to the disk.

OLTP vs OLAP

Regards the representation of the relationships using the relational model.



OLTP

- Oriented to handling ACID properties of transactions.
- Normalized data.
- High volume of transactions.
- Simple and fast queries.
- Fast processing of write operation (*few indexes or none*).

Queries like: "Give me the list of the users who bought X".

Usually, an OLTP system is organized in rows: A file is a sequence of records (*list of bytes*) organised in a certain way (*the way to read the file is determined by the format of the file*) in the disk.

For relational databases the rows of the tables are stored on the disk, this is nice for OLTP.

In those systems, CRUD (**Create**, **Read**, **Update** and **Delete**) operations were problematic in terms of time response.

OLAP

- Oriented to fast analysis of data.
- Denormalized data.
- High volume of data.
- Complex and slow queries.
- Fast processing of read operation (*many indexes*).

Queries like: "Tell me how many users bought X?"

When the OLAP system assumed a big relevance, thus the time response of CRUD operations became very important, so the record-based organisation started to give some problems.

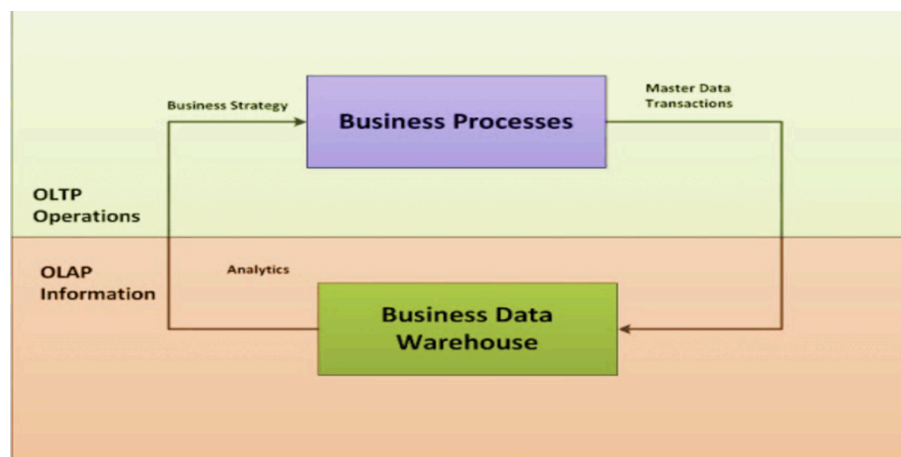
Data Warehouse

A data warehouse is a data management system designed to support business intelligence activity (*big and complex analysis operations*).

In the figure below there's the typical business process, for example:

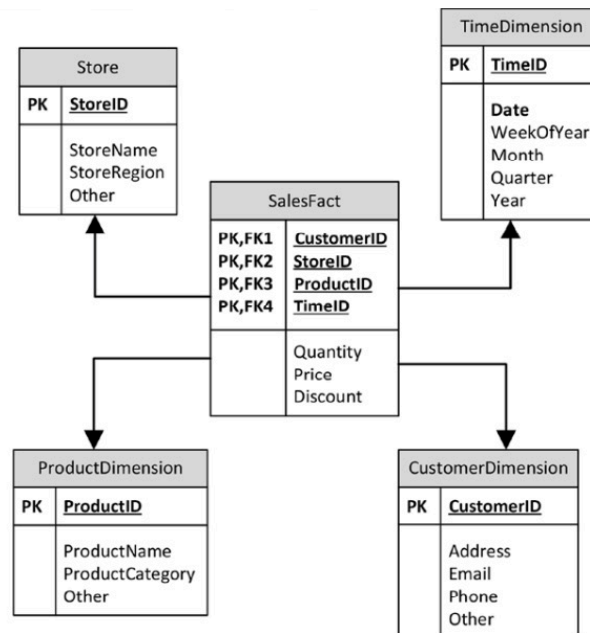
1. A transaction occurs.
2. That transaction generates useful data that we store in the warehouse in some way.
3. Using those data, we can generate analytics for the management.

The most important thing is that we want a fast response from the warehouse.



Optimize analytics - Star Scheme

It is a solution for relational databases: We define a central large table which is associated with numerous smaller tables.



All other tables have a dependency on the central one, so the entire scheme looks like a star (*from this the name of the technique*).

It is a redundancy made for optimising a single complex query; obviously I can have more than one star scheme for a single application, depending on the queries I must perform.

Pros of the Star Scheme

If we want to do some heavy analytics on sales, we can find all the information we need inside only one table, the central one.

Cons of the Star Scheme

- CPU and IO intensive.
 - The central table may become very big, so hard to manage.
- Data volume and the size of the central table grew up along user demands for interactive response times.

Column Storage

When we perform analytics, in most of the cases we only need the value of a single field of many rows. In a row storage this thing is possible but time-consuming, because I could do many accesses to the disk.

If we store data in columns, then, analytics performance will increase because they have to do less access to the disk.



We store the set of the attributes of a relational table in the same section of the disk. In this way I can retrieve values of a certain column with much less disk accesses.

Pros of Column Storage

Queries that work across multiple rows reading only one field are significantly accelerated.

Cons of Column Storage

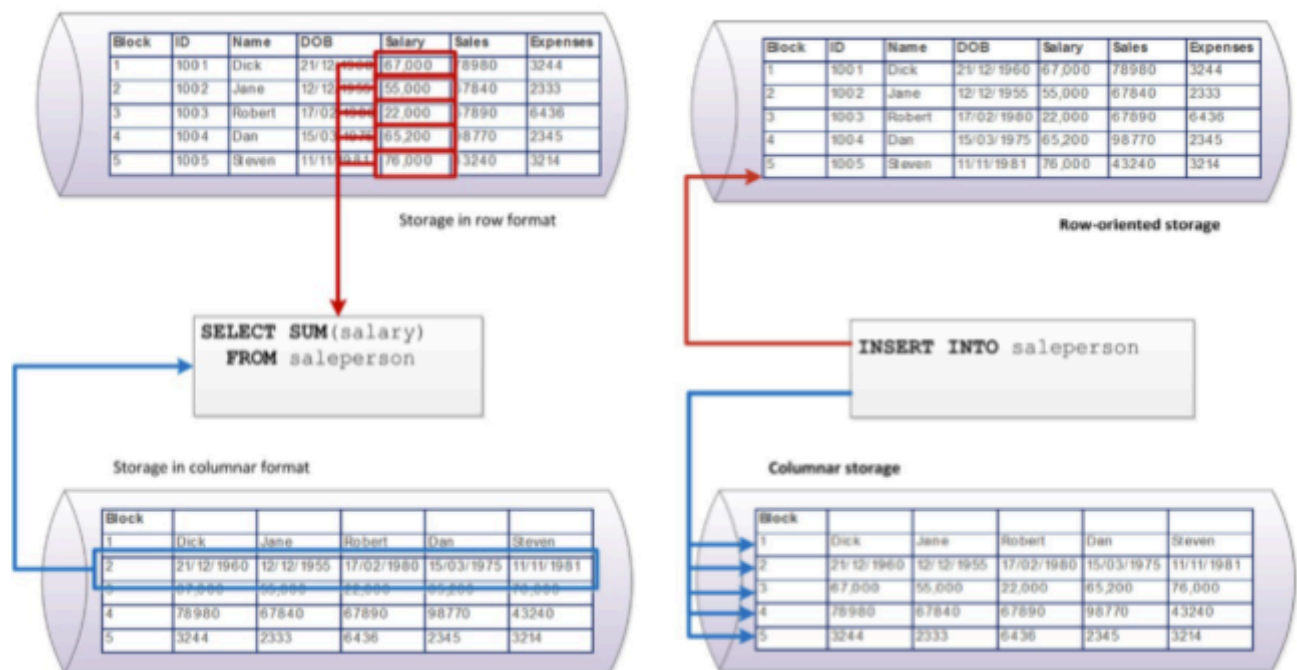
Adding an entire record is more time-consuming, because I must do more accesses to the disk.

Solution for the OLTP and OLAP problem

I use both solutions:

- A row-oriented for the OLTP side (*to have fast write operations*).
- A columnar for the OLAP side (*to have fast analytics*).

The OLAP will be updated with the new information gathered by the OLTP.



Columnar Compression

Data compression algorithms basically remove redundancy within data values. The higher the redundancies number, the higher the compression ratios.

How to find redundancies?

Data compressions algorithms usually try to work on localised subsets of the data. If data is stored in a column database, a very high compression ratio can be achieved with very low computational overheads.

As an example, often in a columnar database data are stored in a sorted order.

In this case, very high compression ratios can be achieved simply by representing each column value as a "delta" from the preceding column value.

Delta Store

The basic column storage architecture is unable to cope with a constant stream of row-level modifications.

Note that these modifications are to be done instantly only if the data warehouse must provide real-time “*up-to-the-minute*” information.

Can we group these modifications somewhere and write them in another moment all together?

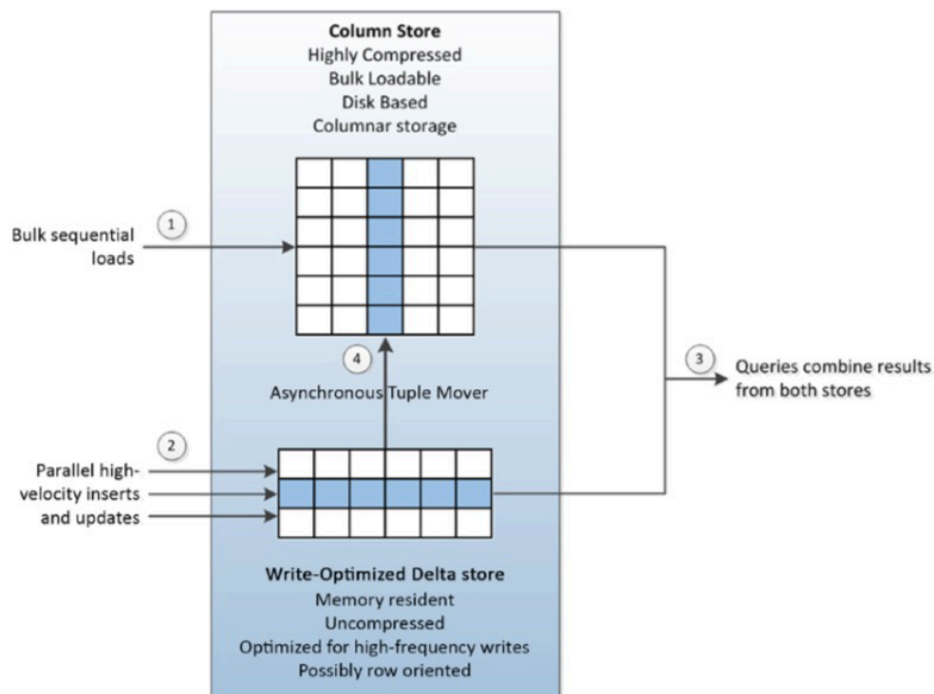
Delta store is an additional part of the database (*often row-oriented, so it can optimise insert operations and row-level read operations*), devoted to merging the information of the last write operations to the database.

In the delta store we don't have compression and it is **designed to optimise high-frequency data modification operations**.

In case of analytics that require real-time results, we may need to access both the delta store and the column store to return complete and accurate results.

How does the Delta store work?

1. An insert operation occurs.
2. New data stay temporarily in the delta store, which is optimized of write operations.
3. During a low load period, data will be shifted to the columnar one in a batch way.



Projection

Sometimes, queries need to read more than one column (we have to use a subset of attributes together). A projection is a group of columns that are frequently accessed together.

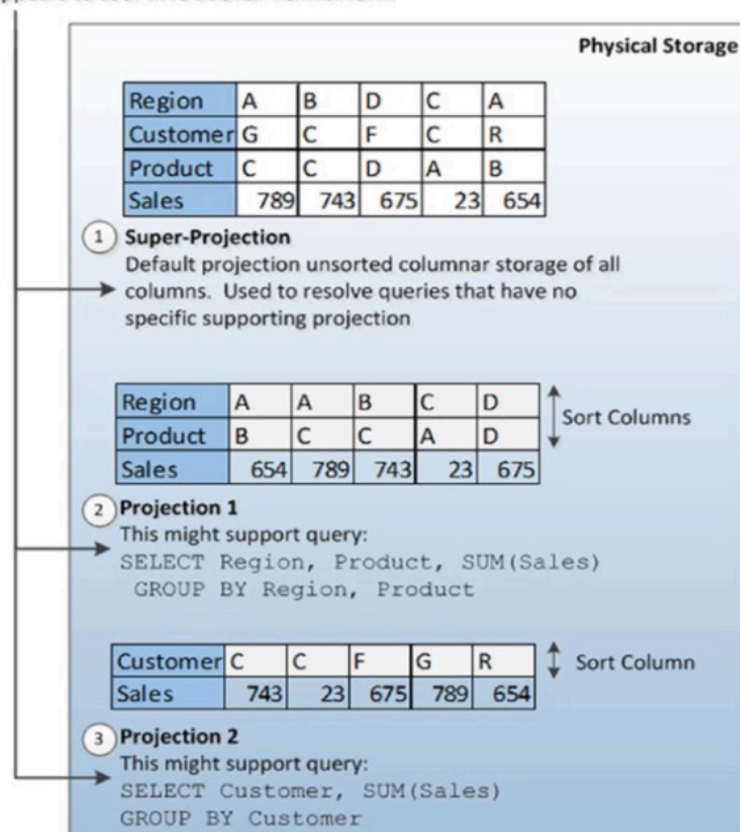
Columns of a specific projection are stored together on the disk (so retrieving all of them is much less time-consuming).

An important projection is the “Super-projection”, which is the projection of all the columns of the database.

Region	Customer	Product	Sales
A	G	C	789
B	C	C	743
D	F	D	675
C	C	A	23
A	R	B	654

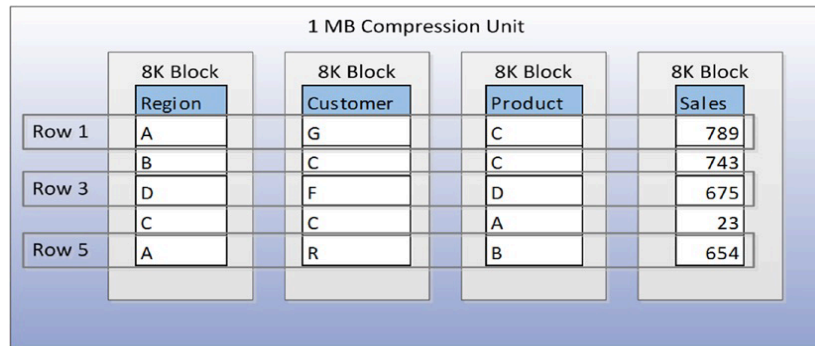
Logical Table

Table appears to user in relational normal form



Hybrid Columnar Compression (Oracle)

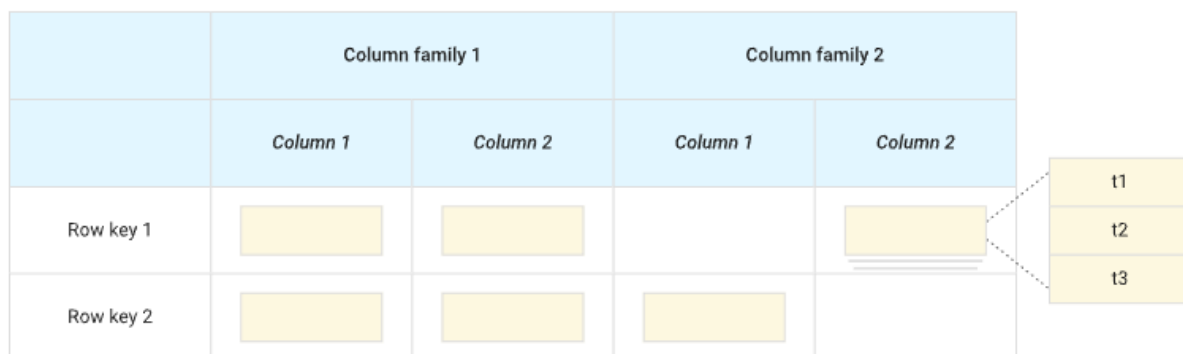
- Data is divided into main units of 1MB.
- Each unit is divided into blocks of 8KB.
- Each block contains the values of a certain column.



BigTable

Distributed storage system designed to handle store data in massively scalable tables, each of which is a sorted key-value map.

It is particularly well-suited for managing large-scale data across many servers and is used in various Google services like Google Search, Google Maps, and Google Analytics.



Every read or write operation on a row is atomic, so I can't access two rows simultaneously.

Row and row key

A table is composed of *rows*, each of which typically describes an entity.

A row is composed by a certain number of columns grouped in column families.

Each row is indexed by a `row key`, which it is the equivalent of the primary key for a relational database.

The `row key` defines how data are distributed among the cluster and permit to retrieve a row.

Keyspace

A top-level logical container: is the set of all possible keys that can be used to access data in the database. It is the equivalent of the database in a relational system; typically, there is one keyspace per application.

A keyspace contains some multiple column families; for example, here it is a Cassandra script for create a keyspace where we also define the replication strategy:

```
CREATE KEYSPACE key_space  
  
WITH replication = {  
    'class': 'SimpleStrategy',  
    'replication_factor': 3  
};
```

Columns

A column is a set of values, each associated with a timestamp t_i (*version of the value*).

Each column is part of a column family, and it has a column qualifier, which is unique within the column family.

Row Key →

Column Name	
Value ₁	Timestamp ₁
Value ₂	Timestamp ₂
Value _n	Timestamp _n

Column family

It is the equivalent of a table in a relational database, but it has more flexibility: each row in a column family can have a different number of columns (*in a relational database those would have been null values*).

In BigTable columns that are frequently used together are typically grouped into a *column family*; for this reason, they are kept near to each other in the disk.

Row key: 123

Column Family: user

Column: name = "Alice"

Column: age = 30

Column Family: address

Column: city = "Roma"

Column Family: other_info

Column: driving_license = "2018-01-06"

Row key: 456

Column Family: user

Column: name = "Mario"

Column: age = 26

Column Family: address

Column: city = "Milano"

Column Family: other_info

Column: email = "mario@gmail.com"

As we can see in the code above:

- Each row can have a different set of columns.
- Some columnarDB permits you to have columns that belongs to different column families in a row.
 - In BigTable this thing is possible.
 - In Cassandra this thing is **not** possible.
- If a column is not used in a particular row, it does not take up any space.

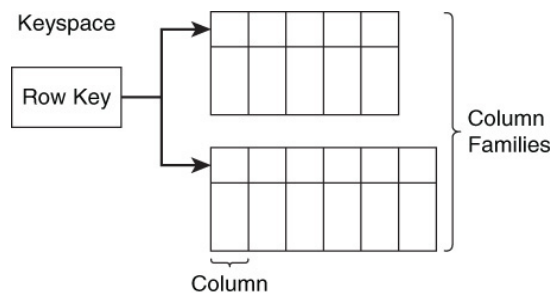
Column families must be pre-defined (*like tables in RDBMS*).

If we want to define a new column family, we must add a new column with a new list of attributes. We don't have a scheme; we must know in advance which attributes must be used together.

Columns in a family can be modified dynamically, so applications can add or remove columns inside families (*like in a KVDB*).

Columnar databases – Design tips

In this type of databases, we can store information in a strong unnormalized form, so, we must define the so-called key space, which is composed of a set of rows keys.



For each table we have a row key with a specific meaning. For each row key we have the column families which contain a set of attributes that have some common characteristics.

Advantages of a columnar database

Read Performance: Queries that require only a few columns can be executed faster because only the necessary columns are read.

Data Compression: Columnar databases often can compress data more efficiently because values in columns tend to be more similar to each other compared to values in rows.

Typical queries in columnar databases

Queries provide information to effectively design column family databases.

- How many orders have been placed in the X region?
- When did a particular customer last place an order?
- What orders are in route to customers in London?
- What products in Ohio warehouses have fewer than the stock keeping minimum number of items?

The selection criteria used in the queries should be used to determine optimal ways to organize data with tables and column families and how to build partition; in order to minimize the accesses to the disk.

Example: Relational to columnar

Customers (*one-to-many relationship with Orders*)

- CustomerID
- Name
- Address

Orders (*one-to-many relationship with Customers*)

- OrderID
- CustomerID
- Date
- Amount

Products (*many-to-many relationship with Orders via OrderProducts*)

- ProductID
- Name
- Price

OrderProducts (*linking table for the many-to-many relationship*)

- OrderID
- ProductID
- Quantity

In a columnar database, data is stored in columns rather than rows.

The query: *Counting the number of products purchased by a specific user.*

We need to optimize our schema in Google BigTable to support these queries efficiently.

OrderProducts Table

- **Row key:** CustomerID#OrderID#ProductID.
- **Column family:** called “details”
 - Quantity.

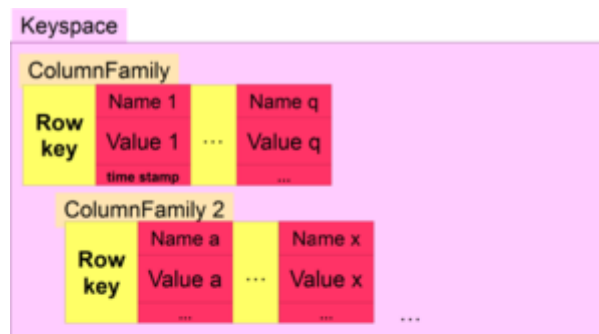
Query the `OrderProducts` table using a prefix scan with `CustomerID#` to fetch all orders for a specific customer and aggregate the product quantities.

Differences between Relational and Columnar databases

Inside the keyspace, a columnar database stores the column families.

For simplicity, you can think of the keyspace like a RDBMS schema and column families like tables, but their similarities end here.

A column family is like a table which is identified by a row key.



Column families “float around”

A column family “floats around”, it is not linked to any other column family (*in the relational one, tables can be linked with relationships*).

Column families do not represent an entity

A column family consists of a set of data about a specific topic, a column family does not represent an entity (*in the relational one, tables represent entities*), in fact data presented in a column family could have been taken from multiple rows.

Column families do not have a schema

Columns can vary between rows (*see the image below*): in a column family, rows can contain a different number of columns to each other, as well as data from different columns.

Moreover, in a column family, columns can be added dynamically at different times (*instead, in a RDBMS you can’t easily change the structure of a table*).

Street	City	State	Province	Zip	Postal Code	Country
178 Main St.	Boise	ID		83701		U.S.
89 Woodridge	Baltimore	MD		21218		U.S.
293 Archer St.	Ottawa		ON		K1A 2C5	Canada
8713 Alberta DR	Vancouver		BC		VSK 0A1	Canada

Join operations cannot be made easily

Join operations can’t be used because data is denormalized.

Empty values

In a RDBMS a table is always a complete matrix: if we don't have a value for an attribute, we set it to null and at a low level will occupy some memory.

Let's consider 3 columns families {1, 2 and 3} and the row with `row_key = "Pietro"`.

- 1° column → 2, 3, 9.
- 2° column → 9, 1, 1.
- 3° column → empty.

In a columnar database Pietro is composed in this way:

Pietro	2	9
	3	1
	9	1

The third column (*which is totally empty*) doesn't occupy any space, so in the columnar database an empty information does not occupy memory.

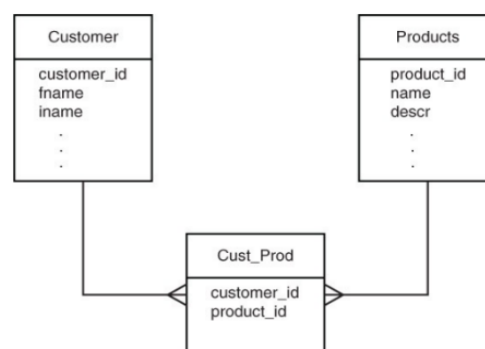
In a relational database the same information would be stored into a vector like this:

```
[2, 3, 9, -1, -1, 9, 1, 1, null, null, null]
```

It occupies more memory than the columnar one, because the length of the structure doesn't change even if some elements are empty.

Avoid join operation and the name of the column

I add a data structure with derived values that I normally could obtain only with a join operation.



For every customer I create a redundant structure which contains the list of the object IDs.

The redundant structure is a column family, where every column is a value itself (as can be seen in the image below, the name of the column can store information too).

Cust_Prod						
Column Name	→	38383	48282	59595	←	Product ID
Column Value	→	Dell Laptop	Apple iPhone	Galaxy Tab S	←	Product Name

This column family is linked to the same row key of the customer column family, so using the key `customer1` we can also access the column family of all the products ordered by that customer.

Row key: `customer_id`

Column Family: `customer`

Column: `fname = "Alice"`

Column: `lname = "Rossi"`

...

Column Family: `Cust_Prod`

Column: `38383 = "Dell Laptop"`

Column: `48282 = "Apple iPhone"`

...

Row key: `product_id`

Column Family: `product`

Column: `name = "Dell Laptop"`

...

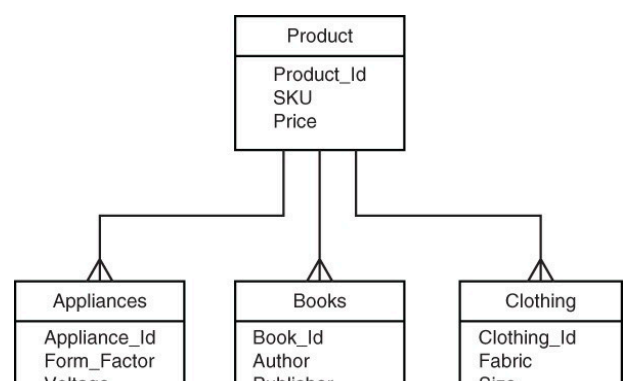
Column Family: `Prod_Cust`

Column: `customer_id = "Alice"`

...

Model an entity with specialisations

Suppose that we have this situation, we have an ecommerce application, we have a product, and this is specialised in three different products.



In a relational database we may create 4 tables, where one is the father and the other 3 are specialisations of the first.

In a column database we need one table with four columns.



If we can easily predict the number of products, we can follow this approach. If the number of products change too often, then this approach is not the right choice.

How to manage the previous versions of values

BigTable permits us to keep old versions of the values and they are useful in case of roll back changes.

Mind that those values will occupy memory and keep them even if they are useless is a waste of memory.

The number of versions that you have to keep depends on the requirements of the application.

Avoid complex structures

If the structure to store is too complex (*because every attribute needs to be separate in different columns*) it may be a good idea to simply store the JSON version of it like a big string.

No Indexes

Using separate columns for each attribute makes it easier to use indexes. If we store the JSON string, we can't use indexes anymore.

We don't have any column families-related strategy

Separating attributes into individual columns allows you to use different column families if needed.

Indexes

In a column database an index is used to quickly find rows that references that column value.

We have two types of indexes:

- Primary indexes are indexes on the row keys.
- Secondary indexes are indexes created on one or more column values.

If we want to speed-up a query like this:

```
SELECT * FROM Tab WHERE state = "OR" AND Lname = "TR"
```

We should put a secondary index on state and Lname.

Secondary indexes are created for specific columns and store the association between the column value and the row's position.

Secondary indexes can improve query efficiency but can also introduce additional overhead in terms of space and performance during data updates.

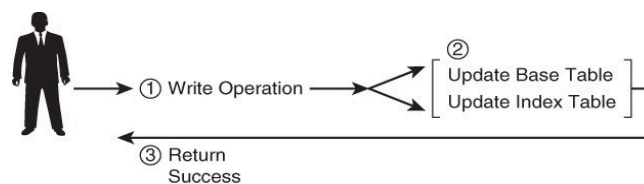
When an index is useless?

- When the distribution of the values in a column is uniform (*roughly equal numbers of each value*).
- When the distribution of the values in a column is composed of only two values (*like a boolean*).
- When the number of different values is too high.
- When in a column we got too many empty values.

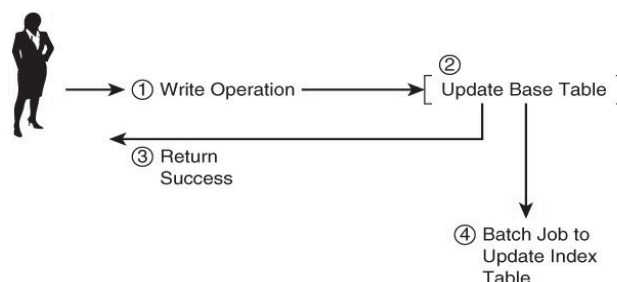
How to maintain indexes

I have two approaches to do this:

Updating by our application an index table during the write operation and then returning the confirmation to the user only after the index has been updated. The index is always updated but the write operations are slower.



Another way is to batch more than one updates for the index and perform them in a second moment all in one time. The application is faster, but we may leave indexes temporarily not updated.



Translate relationships

Architecture of a Distributed Database

Let's see how to distribute a database.

Partitioning

Partitioning refers to the process of dividing a single database into distinct, non-overlapping parts called partitions. This is typically done to improve performance and manageability within a single database.

Usually, partitions are all stored in a single physical server.

Partition policy

Range

rowID or column name.

Hash value based

Hash value of the key, rowID or column name.

List of values

Name of states or provinces.

Pros of partitioning

- **Improved performance:** Queries can be executed faster by accessing only the relevant partitions.
- **Improved manageability:** Manage a partition is easier than manage the entire database.
- **Scalability:** The entire database can be enormous, so a single physical server could not handle it.

Sharding

Sharding is a form of database partitioning but applied at the level of an entire database instance.

Each shard is a separate database that contains a subset of the data, and it is stored on a separate physical server.

Sharding is typically used to scale out databases horizontally across multiple servers or locations.

Sharding policy

Horizontal sharding

Rows are distributed across multiple shards. Each shard contains a subset of the rows.

Policy like: *"A social network database, where users with ID < 1000 are in shard 1, the other in shard 2"*

Vertical sharding

Different tables or columns are stored in different shards.

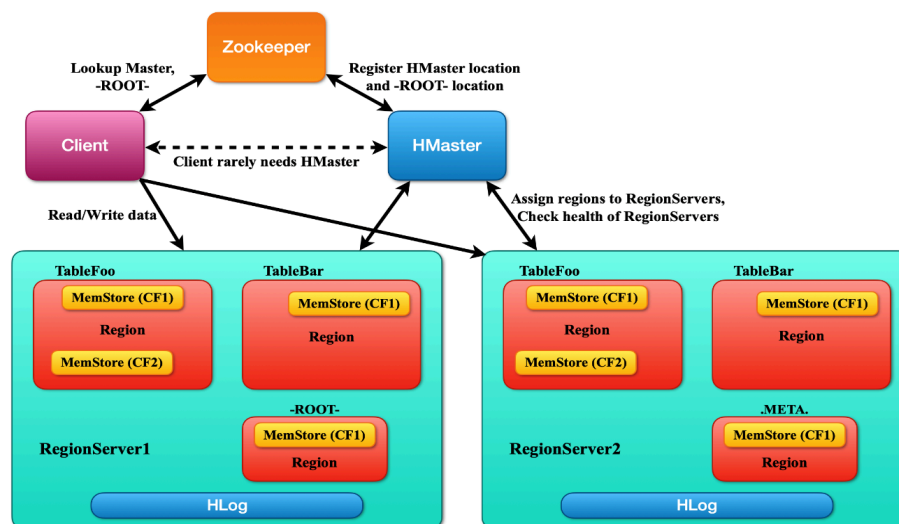
Policy like: “A social network database, where user details are in shard 2 and posts are in shard 1”

Pros of sharding

- **Load balance:** Thanks to the horizontal scaling, I can distribute the read and write load across multiple physical servers.
- **Fault isolation:** Issues in one shard do not affect other shards.
- **Geographical distribution:** Data can be placed closer to users in different regions, reducing latency.

HBase

The Hadoop solution for create a distributed file system. This software make appear a distributed database like a simple disk.



The file system permits us to see data split on various servers like one unique node. The file system of HBase is called HDFS.

The master has the duty of associating to the region server a specific portion of the data. The region servers are the end nodes that manage the user request.

The zookeeper is a node of Hadoop that coordinates the communication between the master and all other nodes.

Cons of HBase

- We must deploy the software for the master, region servers and for the Zookeeper.
- The zookeeper and the master are both single points of failure.

HBase is based on the distributed file system of Google, the data is distributed in different servers, we have data partitioning, data redundancy and so on.

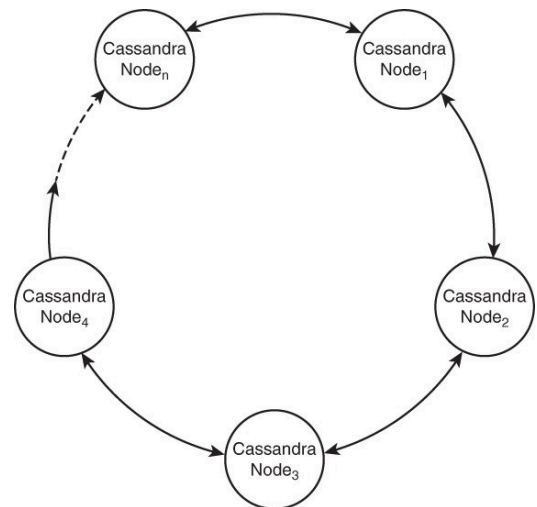
Name node contains the collections of nodes, the data node contains the collections of data.

Masterless (Cassandra)

All servers have the same software installed. Nodes can still have specific functionality, but in general they are all the same, so they have the same capabilities.

Servers adopt a strategy to communicate with each other.

In this approach there are no single point of failures: when a node fails, the entire system can still respond to the user, another server will respond to the operation.



Commit Log

An append-only file (*no delete, only append to the end*) for speed-up write operations and for define some recovery procedures.

What we want to do is delay the write operation on the disk but without the risk of losing the updates if a server fails.

A commit is a conclusion of a transaction; a commit grants us that the atomicity has been respected.

How the commit log works:

1. The user sends a write operation to a random server.
2. The server will respond and write it in the commit log.
3. Someone in charge of reading the info on the commit log, will update the information also on the disk (*so will effectively send the write operation*).

Speed up writes operations

The server can avoid waiting until the write operation has been completed.

The write operation will be stored in the commit log, then, will be effectively sent to the target server to be performed.

Update propagation

The commit log file can be send to the other servers.

Recovery

In case of disaster, we can recover the portion of the database by using the commit log. All the write operation will be executed a second time (*keeping the original order*), and at the end the database partition will be recovered.

Bloom Filter

In case of a distributed database (*with different partition in different server that can also be in different places*), searching for a specific value can be very time-consuming; so, we introduce a probabilistic data structure used to speed up read operations.

The best way to explain this is with an example:

1. We want to retrieve information about Pisa in the database.
2. I don't know where it is (*neither if it exists or not*), so I must check if that value exists in the database.
 - I have to scan the entire partition.
3. If I do not find it, I may have to repeat the operation for another partition.

At the end, we performed more than one disk access, without consider the latency between communication.

Member Function			Bloom Filter		
Input Set	Test Element	In Set	Input Set	Test Element	In Set
{a,b,c}	a	Yes	{a,b,c}	a	Yes
{a,b,c}	c	Yes	{a,b,c}	c	Yes
{a,b,c}	e	No	{a,b,c}	e	Yes
			{a,b,c}	f	No
			{a,b,c}	g	No

Low
Probability
but
Possible

The bloom filter evaluates whether an element is a member of a set or not. The possible answers given by a bloom filter are:

- "Yes"(actually "*Maybe*").
- "No".

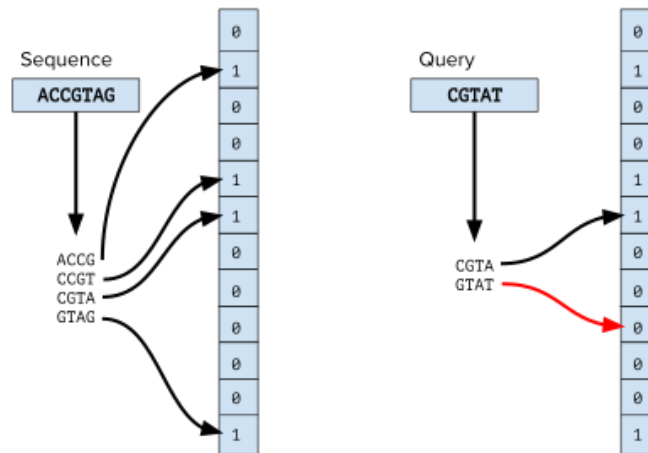
I cannot have a false-negative, but I can obtain a false-positive. If the answer is no, I am sure that in that set the info does not exist in that set.

How to create and how to use a bloom filter?

The bloom filter is an array of n positions unique for each partition

Initially, the array is initialised by 0. Then we select $k \ll n$ hash functions. Each hash function can give as output a number between 0 and n (*we use modulo n*).

I do the operation (*explained below*) for every element inside the database (*so creating a bloom filter is time-consuming, but you do it rarely*).



When a new data x is added (*write*) to the partition:

1. We calculate the k hash functions.
 - The hash is between 0 and $n-1$, so it can be used like an index of the array.
 - Some hash functions can give the same output (*is not a problem generally*).
2. We set to 1 the k position of the array calculated by the k hash functions.

If an element of the array is 1, then it means that during the insert of an element the location has been set to 1.

But if the location is 0, then I am sure that any of the elements inserted into the partition set the location to 1; so, if an element has that location set to 1, I am sure that is not in that partition.

If is 1, then I am not sure if x is in the partition or not, because another element could have set it to 1.

When a read operation on x occurs:

1. We calculate the k hash functions.
2. The access to the partition is performed if and only if all the corresponding k elements of the array selected by the hash functions are set to 1.

If even one of the k positions is not equal to the obtained result, then I am sure that x is not in this partition.



Value of k and n

The maximum false positive rate depends on n and k .

The probability of a false-positive decrease if n increase; because the probability that the k hash functions set the same bit of two values, x and y for example, is lower.

Suggested value of n and k :

- $n: O(h)$
- $k: \log_2(n/h)$

A bloom filter should be used only if I cannot remove elements from the database (*or if it a very rare operation*); if I delete an element from the partition I have to rebuild the filter of that partition from scratch.

If we remove some information, and we put 0 in the position selected by the hash function of the array, we will generate also false negatives (*because some other elements set that location to 1*).

Replication

Consistency level

The consistency level refers to the consistency between copies of data on different replicas.

The chosen consistency level depends on the specific application requirements. We can consider different levels of consistency.

Strict Consistency

I always want the query to return the most recent value; so updated data must be written in all the replicas.

Low Consistency

Data must be written in at least one replica. Some replicas could store old values and queries can return not updated values.

Moderate Consistency

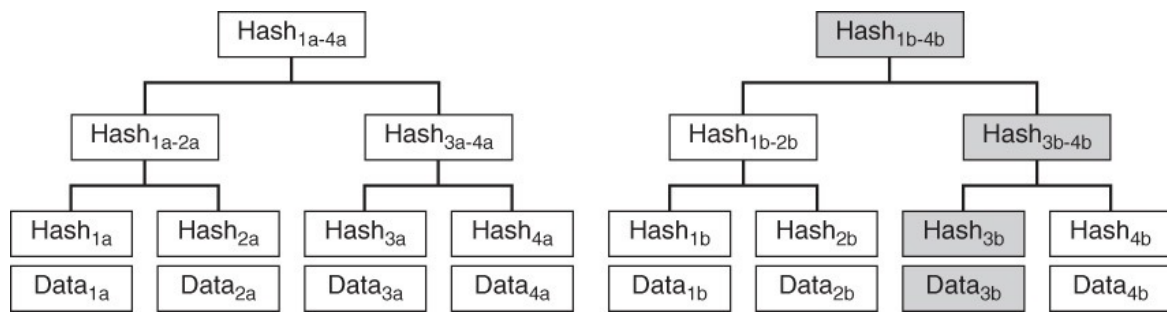
All the intermediate solutions.

Anti-Entropy

Usually there is a main replica which oversees identifying the position of other replicas among the available servers. The primary server is often identified by a hash function.

But how can the primary server know if a replica is outdated or not? It is important to spot differences using the minimum amount of exchanged data.

Anti-entropy is a strategy (*that can be used in all kinds of databases, but heavily used in columnar databases*) for detecting differences (*inconsistencies*) in replicas.



Confronting a hash value is much faster than scanning the entire partition.

Every server builds its own hash tree, and then exchange their hash trees among them to spot differences.

Each leaf contains the hash value of a data set in the partition.

Each parent node contains the hash value of the child nodes content.

1. A server periodically receives the tree of the other nodes.
2. The server confronts it with its tree.
3. If the two roots are the same: the copies are identical, and the algorithm stops.
4. Otherwise, it will navigate the tree and confront the child of the root.
5. Following the different node, the different partition will be spotted.
6. The server with the older partition gets updated by the other.

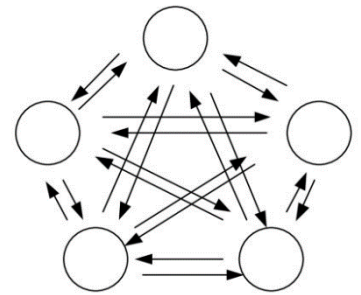
Communication Protocols

We need protocols to check the status of the other servers.

Fully Connected Approach

Every server communicates with all the other servers.

If there are a lot of servers this approach gives problems: the number of messages increases exponentially with the number of nodes.

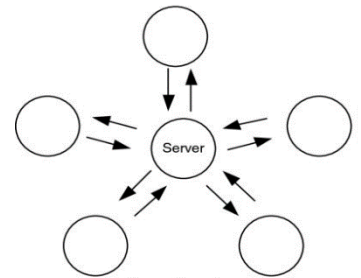


Centralised Approach

There is a central server which collects info from other servers about the write operations.

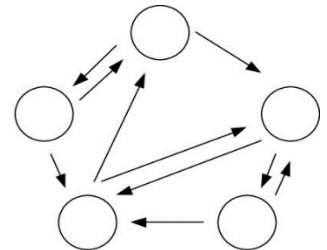
Periodically, the central server sends the updates to the others.

The problem is that the central server is a single point of failure.



Gossip Approach

Each node sends info about himself to **a certain number of random servers**. Can be demonstrate can the system converge in a very fast way.



Cassandra Approach

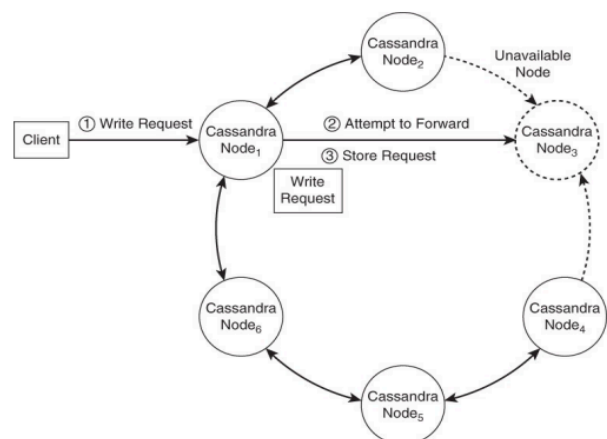
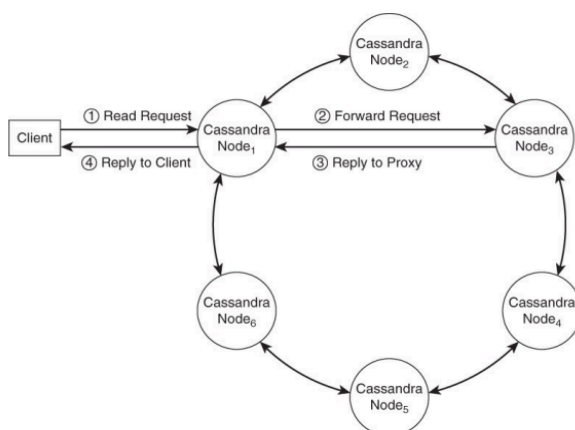
Any node can answer to the client: The request can be executed by that server, or if it is having an overload, the request will be forwarded to another server.

Read request

The first server will function as a proxy; the second server can respond to the read operation or (*like the first server did with him*) forward the request to a third server and act like a proxy too.

Write request

The proxy forwards the write operation to the target server where the write should be performed (*decided by a partition policy*).



Hinted-Handoff process

What if the server interested in a write operation is down? The write operation can be lost: thus, the target node will be in an inconsistent state after its recovery, because it has not executed the update.

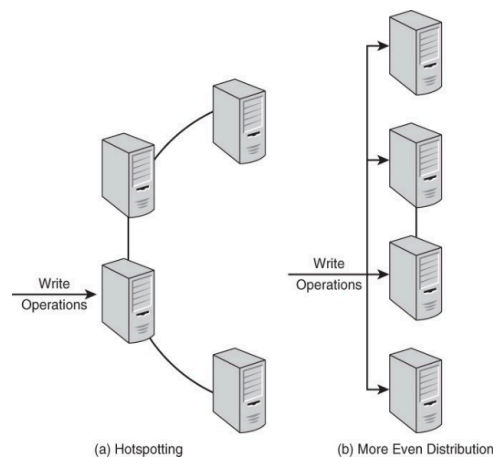
We can use a “*designated node*” to receive write operations and forward them to the target node when it becomes available.

The designated node keeps a data structure (*like a commit log*) to store information about the write operation and in which server should be performed; obviously it preserves the order.

1. The first server checks if the target node is alive.
2. If not, the first server sends the write info to the designated node.
3. The designated node group the write operations in this log and periodically checks the status of the target server.
4. The target server returns on.
5. The designated node sends the write operation stored to the target server.

Hotspotting and load balancing

Hotspotting is when only a part of the cluster works. This may happen when the work is not distributed properly.



We can use a sequential Hash function (*at the first operation give 1, at the seconds give 2...*) that takes as input the row key value.

In this way the write load will be uniformly distributed among the servers.

Graph Databases

Recap of graph's theory

Vertices

In graph theory we use a vertex for representing an instance of an entity.

Vertices (*obviously*) have some interesting properties that we'll exploit later.

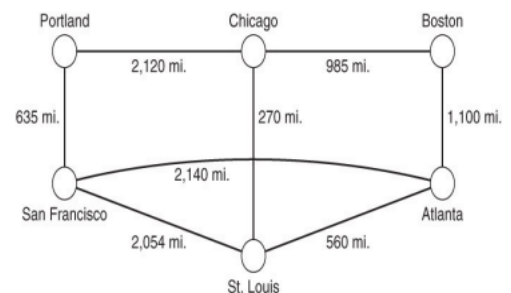
Edges

Edges (*also called links*) represent relationships between two vertices (*entities*). An edge can have properties such as the "weight" (*the weight commonly represents the "cost" to traverse that edge*).

Example of a Graph

In the example below, vertices represent cities of the United States, and the edges represent the roads that connect two cities.

Properties that a vertex may have: population, geographic coordinates, name of the city (*like in the image*), geographic region, and so on. Instead, the properties that an edge can have may be: The distance of the road that links two cities (*like in the image*), the speed limit of the road or the number of lines of the road.



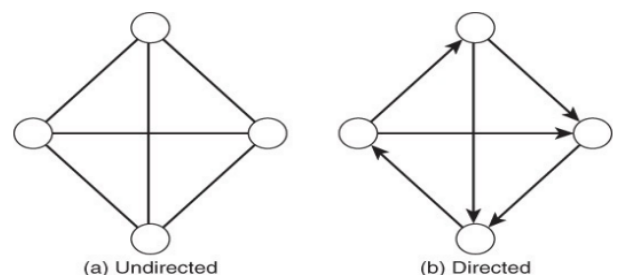
Types of Graphs

Undirected graph

The edges do not have a direction. An Example of this type of graph may be highways network graphs.

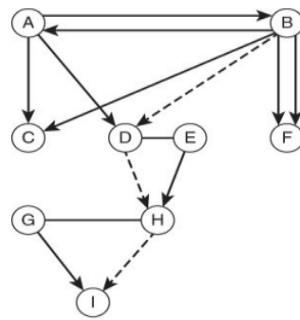
Direct graph

The edges do have a direction. Some edges may be unidirectional, or the edge that goes from X to Y may have different properties of the edge that goes from Y to X. An example of this type of graph may be a family relation graph.



Path in a graph

A path is a sequence of nodes and edges, that from a node X take us to another node Y. Paths allow us to capture information about the relationships of vertices of a graph.



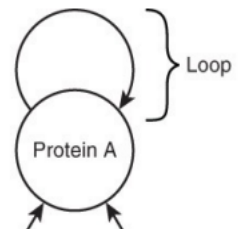
In the image above, we can see the dotted path B → D → H → I.

Dijkstra Algorithm

Find (*among all possible paths*) the path starting from a node X and ending to a node Y with the minimum cost.

Loops

Is a type of relationship between an instance of an entity and itself. An example of this type of relationship may be the proteins, a protein of a certain type may interact in a certain way with proteins of the same type.



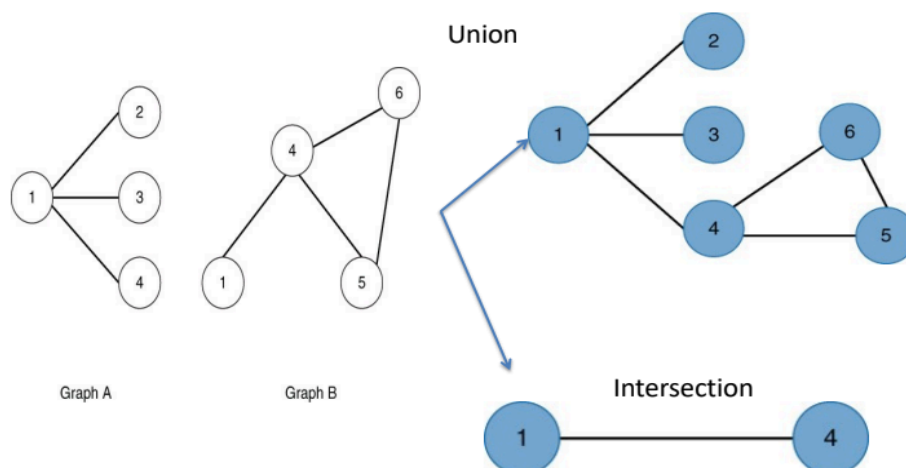
Union & Intersection of Graphs

Union

The union of graphs is a graph composed by all nodes and edges of the united graphs. Nodes and edges in common are not repeated.

Intersection

The intersection of graphs is a graph composed by all nodes and edges that are in common in all the intersected graphs.



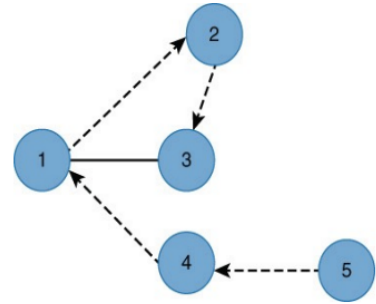
Graph traversal

A graph traversal is the process of visiting all vertices in a graph, often to set or read some property value in a graph.

It's common that, in a graph traversal, a vertex is visited more than one time, so a graph traversal could have a non-linear complexity.

Graph traversal can be a type of operation that you may tend to do very often, so you shouldn't choose a graphDB in an application where you will end up doing many queries that need to explore all nodes of the graph.

Instead, a graphDB should be exploited for applications that require exploring relationships.

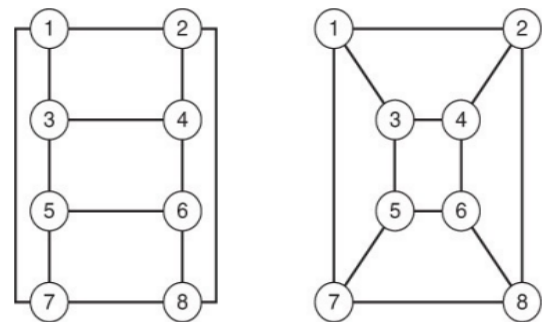


Isomorphism

Two graphs (or subgraphs) are isomorphic if:

- For each vertex in the first graph there is a corresponding vertex in the second graph.
- For each edge between a pair of vertices in the first graph there is a corresponding edge between the corresponding vertices of the second graph.

Detecting this characteristic is useful for exploiting useful patterns among the graphs.



Properties of a graph

These two properties are related to the complexity of graph traversals and of the memory occupancy.

Order

The order is the number of total vertices in the graph.

Size

The size is the number of total edges in the graph.

Properties of a vertex

Degree

The number of edges linked to the vertex: the higher the degree is and more "important" the vertex is in the graph.

Closeness

How far the vertex is from all the other vertices in the graph. Given a vertex X, the closeness can be calculated as:

$$C_B(x) = \frac{1}{\sum_y d(y, x)}$$

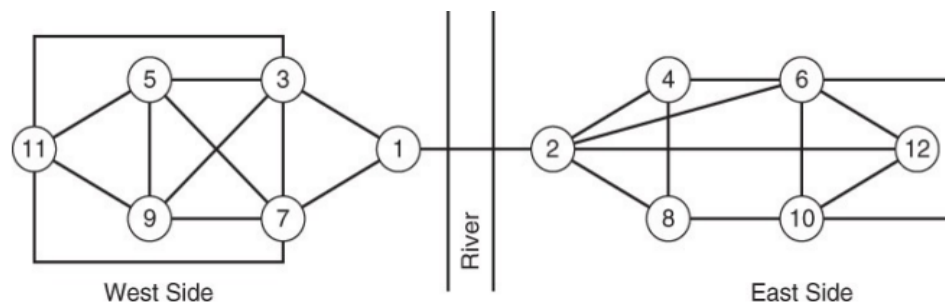
Where the denominator is the sum of the length of the shortest paths between the node X and all other nodes Y of the graph.

The closeness is useful to evaluate the speed of information spreading in a network, starting from a specific node X.

Betweenness

Counts the number of times a vertex appears on the shortest path between any two other vertices, so as to represent the significance of this vertex to the network connectivity.

It can be seen as the level of how much a vertex **can be a bottleneck**. (so if the vertex has bad performances, then is more probably that the vertex can become a bottleneck).



In the example above, the vertices 1 and 2 will surely have a high betweenness, because they are the only one to provide a link to the other side of the river.

Other types of graphs

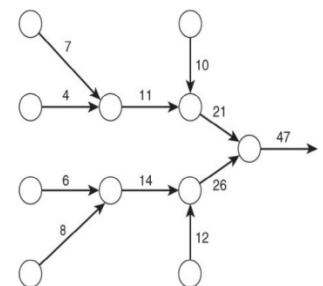
Flow Networks

Is a **direct graph with capacitated edges** used in logistics.

Each vertex has a set of incoming and outgoing edges.

The sum of the capacities of incoming edges cannot be than the sum of the capacities of outgoing edges (*I can't send more than I am receiving*).

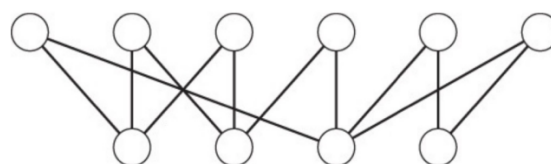
The unique exceptions to this rule are the sources and the sink nodes.



Bipartite graph

It can be used for modelling relationships between students and teachers.

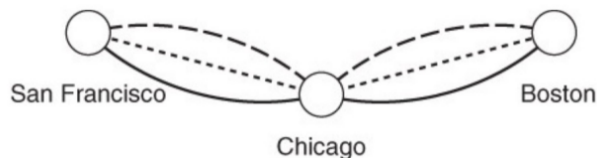
We have two sets of vertices, and each vertex can be connected only to the vertex of the other set.



Multigraph

We can have multiple edges between two vertices. It is **useful when we have multiple ways to reach another node** (*between two cities there can be more than one road*).

Note that given some edge that connect the same two nodes, each edge can have different properties than the other edges.



Graph databases introduction

A vertex represents an entity, and usually, entities in a graph belong to the same category.

An edge specifies a relation between two vertices, and relations may be “*long terms*” or “*short terms*”.

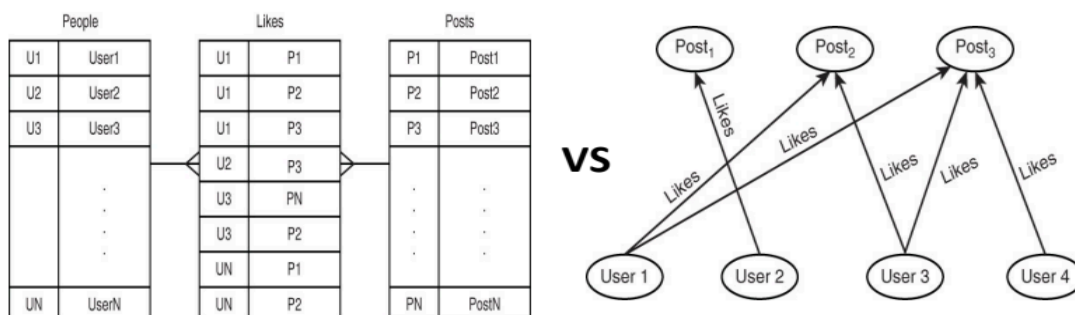


Graph databases vs Relational databases

Query: *List all courses where a particular student is enrolled in.*

In a RDBMS we would need to do a join operation between at least two tables.

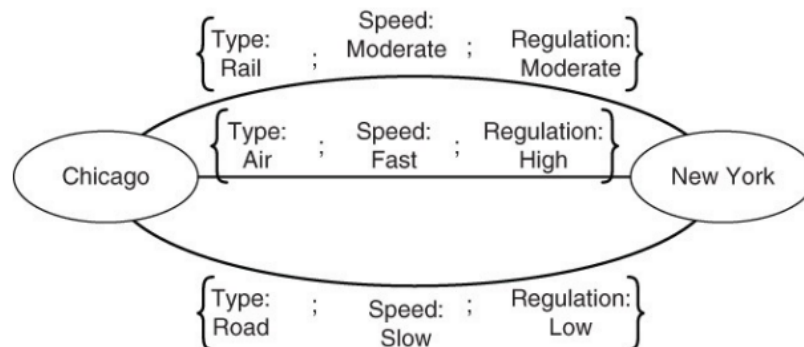
In a graphDB this query is faster because a relationship (*edges*) between two entities (*vertices*) are explicitly stored, so (*like in the example below*) in a bipartite graph we only need to check if between a student and a course there is an edge.



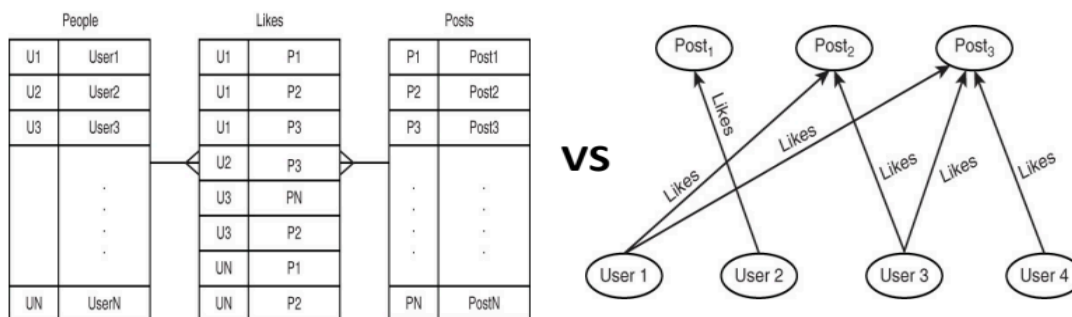
Relationships in graph databases

A graphDB explicitly store relationships between entities.

Relationships may have properties too, and in a graph database this thing is easy to implement, because I can have multiple types of edges (*specifying different values of edge properties*) and each type of edge can have different properties.



Many-to-Many



Typical queries on a graph database

- How many edges between two nodes with certain properties?
- How many edges are linked to a node.
- Closeness and betweenness of a node?
- Is a node a bottleneck?
- If a node is removed which parts of the graph become disconnected?

Example of graph database: HollywoodDB

Let's design a DB for handling movies, actors and directors.

Let supposed to offer the following services for users:

- Find all the actors that have ever appeared as a co-star in a movie starring a specific actor.
- Find all movies and actors directed by a specific director.
- Find all movies in which specific actor has starred in, together with all co-stars and directors

SQL lacks the syntax to easily perform a graph traversal, especially traversals where the depth is unknown or unbounded.

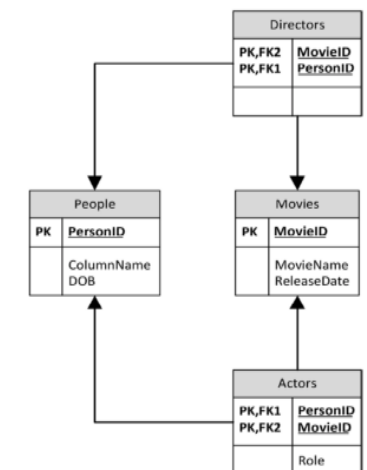
Performance degrades quickly as we traverse the graph.

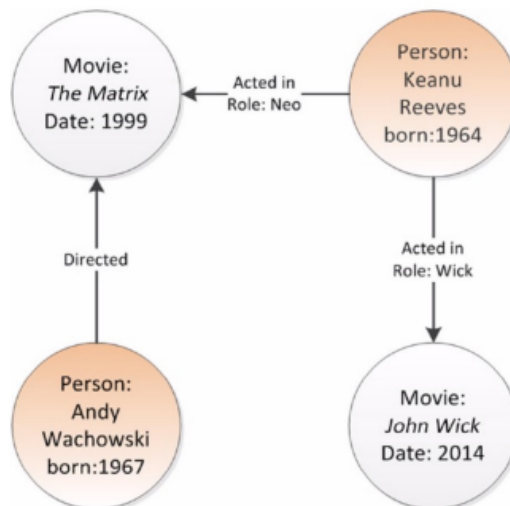
Each level of traversal adds significant response time to a specific query. In the right image we can see the equivalent graph model.

In fact, as you can see in the query below: *“for finding all actors who have ever worked with Keanu Reeve.”*

```
SELECT p2.personname, m1.movieName
FROM people p1
  JOIN actors a1 ON (p1.personid = a1.personid)
  JOIN movies m1 ON (a1.movieid = m1.movieid)
  JOIN actors a2 ON (a2.movieid = m1.movieid)
  JOIN people p2 ON (p2.personid = a2.personid)
WHERE p1.personname = 'Keanu Reeves';
```

The query is very heavy, instead in a graphDB:





```

neo4j-sh (?)$ MATCH (kenau:Person {name:"Keanu Reeves"})
               -[:ACTED_IN]->(movie)<-[:ACTED_IN]-(coStar)
               RETURN coStar.name;

```


Example of graph database: Social Network

Define the main use cases

1. Join and leave the site.
2. Follow a developer.
3. Post questions for experts in a specific area.
4. Suggest new connections with other developers based on shared interests.
5. Rank members according to their number of connections, posts and answers.

Define entities and their properties

Developer	Post
• Name • Location • NoSQL databases used • Years of experience • Area of interests	• Date of creation • Topic Keywords • Post type ○ Example ○ Question ○ Tip ○ News ○ ... • Title of the post • Body of the post

Identify relationships between entities

1. Developer $\square$ Developer.
2. Developer $\square$ Post.
3. Post $\square$ Developer.
4. Post $\square$ Post.

Let's explore the 1° relationship (*many-to-many*)

A developer can follow another developer, so the name of the first relationship could be "*follows*".

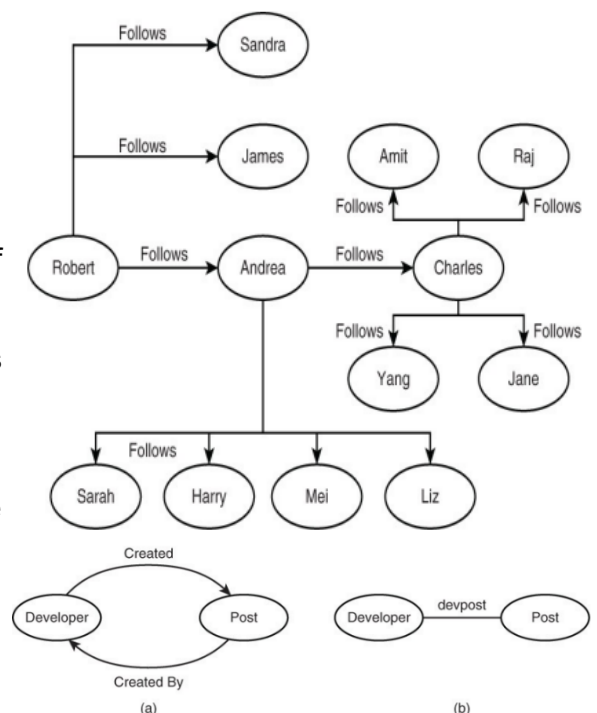
If a developer X follows another developer Y, then X is interested in Y's posts.

Let's explore the 2° and 3° relationship (*1-to-many*)

A developer can create and publish a post, so the name of the second relationship could be "*create*".

At the same time, another relationship is created (*one of the third type*), because at the same time the post is created and published by the developer.

So, the name of the third relationship could be "*is created*".

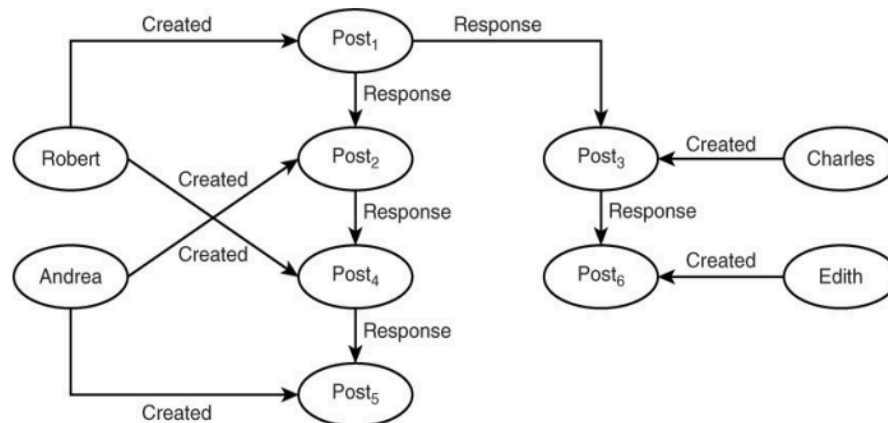


Let's explore the fourth relationship (1-to-many)

A post can be a reply to a question (*which is a post itself*), so the post that contains the reply is linked to the post which contains the question.

The name of the fourth relationship could be “*refers to*”.

The two posts can be created by two different developers in different moments (*even with a difference of years between the creation dates*).



Mapping queries

Domain-specific queries must be translated into graph-centric queries, that often results in a graph traversal.

Graph-Centric Query	Domain-Specific Query
How many hops (that is, edges) does it take to get from vertex A to vertex B?	How many follows relations are between Developer A and Developer B?
How many incoming edges are incident to vertex A?	How many developers follow Andrea Wilson?
What is the centrality measure of vertex B?	How well connected is Edith Woolfe in the social network?
Is vertex C a bottleneck; that is, if vertex C is removed, which parts of the graph become disconnected?	If a developer left the social network, would there be disconnected groups of developers?

Implement graph queries

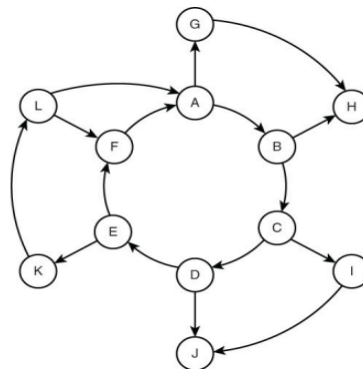
To implement them, you must use good graph algorithms.

- Always control the dimension of the graph (*even if it is not easy*); bigger the graph, bigger the computations.
- Use indexes for improving retrieval times.
- Use appropriate types of edges.
 - If the relation is symmetrical, use an undirected edge.
- Design properly the properties of an edge.

- Prefer to use simple edges because properties require memory.
- Use the right types for properties.
 - If a property will have only the values 0 or 1, use a boolean instead of an integer.
- To avoid endless cycles, put a time limit to every graph traversal.
 - Use proper data structures to keep track of visited nodes (*avoid reading the same node more than one time*).

Cycles in graphs

If there is no indication that we have already visited A, we could find us in an endless cycle of revisiting the same six nodes over and over again.



Scalability of graph databases

Most of the famous implementations of graph databases **do not consider the deployment in a cluster of servers** (*no partition nor replicas management*).

Indeed, they are **designed for running on a single server** (*vertical scaling only*).

For this reason, the developer must take special attention to the dimension of the database; in particular of: the number of nodes, the number of users and the number and the size (*type*) of properties.

MongoDB

MongoDB stands for “Humongous DB”.

MongoDB key features

MongoDB is open-source and implemented in C++, in fact we can use C++ for implement some analytics or procedures.

We deal with BSON (*a JSON document with some additional properties*) format documents.

The maximum dimension of a document is 16 MB.

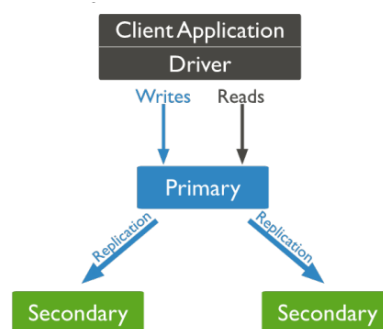
Every document is stored in a collection. A collection is like a table of a RDBMS but documents inside a collection do not need to have uniform structures; they might share indexes (*which are implemented for a specific collection*).

MongoDB gives support for embedded data models (*document embedding*), which reduces I/O activity on the database.

MongoDB offers a query language very easy to learn. This language supports CRUD (*read and write*) operations as well as: data aggregation, text search and support geospatial queries.

Architecture of MongoDB

Replication in MongoDB

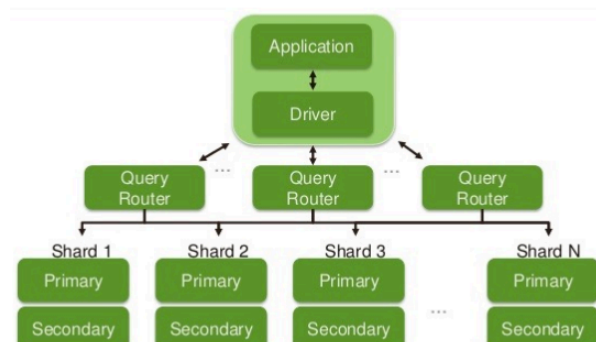


The MongoDB replica management system is Master-Slave (*primary-secondary*) based. A MongoDB instance is composed of all replicas and all shards.

1. The client application (*written in any programming language*) sends a request (*write or read operation*) to the primary (*master*) replica.
2. The master server oversees communicating (*forward*) to all (*or some of them*) slaves to update replicas.

Sharding in MongoDB

Let's suppose that we want N shards; for each shard we need N primary physical servers.



Let's suppose that each shard should be replicated at least 3 times; we need 2N more physical servers for replicas.

For each shard we have a master-slave system.

We also need a server called “*Config Server*” (which should be replicated too); we need at least 1 more server.

In total we need $3*N$ servers to implement sharding with N shard and 2 replicas for each shard.

Core processes of MongoDB

mongod

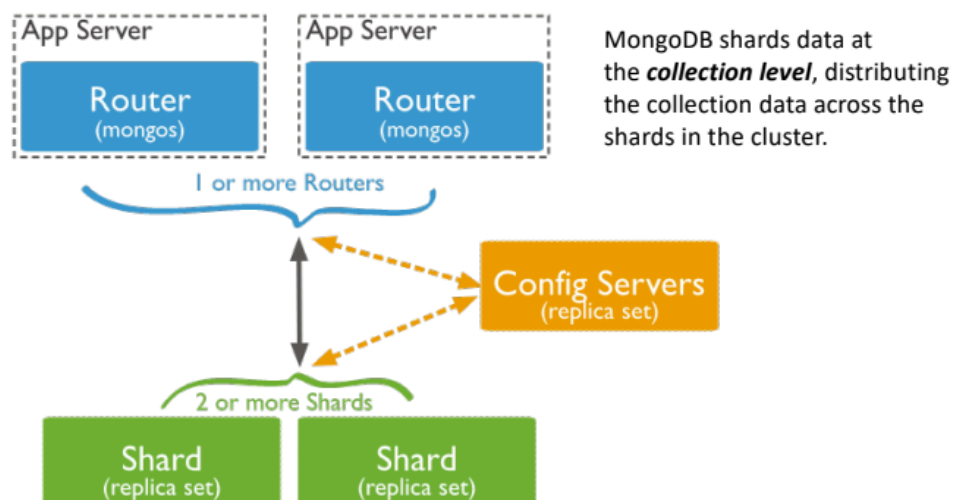
The mongod is the process which maintains the activity of the MongoDB server.

It handles data requests, manages data access and database processes.

mongos

They are strictly related to sharding activities.

The mongos are the controller and query router interfaced between the client application and the sharded cluster.



Documents in MongoDB

The BSON file

BSON stands for “*Binary JSON*” and it is the binary serialization of JSON-like documents. Inside a BSON document we got additional data types and ordered fields.

```
{  
  Name: "sue", Age: 26,  
  Status: "A", Groups: [ "news" , "sports" ]  
}
```

The `_id` field

By default, each collection contains an `_id` field. This field is like a primary key: it is immutable, cannot be an array and must be unique for each collection.

This field is always automatically indexed by mongoDB, and this index is special, for this reason document linking must be done using this field.

MongoDB Operations

Insert Operation

If the collection does not exist, the insert operation will create the collection too.

Write operations are atomic on the level of a single document: a document is completely added or not added at all.

`insertOne()` and `insertMany()`

It takes a document-like parameter and inserts it into the target collection (`users`).

```
db.users.insertOne({ Name : "Sue", Age : 26 })
```

With `insertMany()` I can insert multiple documents in a target collection (*in the example the collection* `persons`).

```
db.users.insertMany(  
(  
  { Name : "Sue", Age : 26 },  
  { Name : "Alex", Age : 25 }  
)
```

Drop Operations

`db.users.drop()` it drops the target collection (users).

Query operations

Example of Queries

find()

By default, the result set is sorted in ascending order by `_id`.

```
db.users.find( {} )
```

“Get all the documents in the collection users”

```
db.users.find( { age : 26 } )
```

“Get all the documents in the collection users with age = 26”

```
db.users.find( { age : { $in: [ 25 , 26 ] } } )
```

“Get all the documents in the collection users with age = 25 **or** age = 26”

```
db.users.find( { name : 'Alex' , age : { $lt : 25 } } )
```

“Get all the documents in the collection users with name = 'Alex' **and** age < 25”

The “And” condition is implemented with the ‘, ’.

Instead, the inequality is implemented with `$lt` □ “Less Than”.

```
db.users.find( { $or[ {name : 'Alex' }, {age : { $lt : 25 } } ] } )
```

“Get all the documents with name = 'Alex' **or** age < 25”

```
db.users.find({
  name : 'Alex',
  $or[
    {age : { $lt : 25 }},
    {age : { $gt : 26 }}
  ],
  job: /^p/
})
```

“Get all the documents with name = ‘Alex’ **and** (age < 25 **or** age > 26) **and** the job starts with a ‘p’”

Instead, the inequality is implemented with `$gt` ▯ “*Greater Than*”.

Instead, the “*starts with a p*” condition is evaluated through a regular expression.

Update operations

How to change something inside an existing document?

```
updateOne()
```

Update only one document, specified through filters.

```
db.collection.updateOne( <filters> , <update> , <options>)
```

```
db.items.updateOne
```

```
(
{ name : “Computer”}, // the item must be Computer
{
  $set: {“size.uom” : “cm” , status : “P”}
  // field to change
  $currentDate:{lastModified : true}
})
```

If in the document that we are updating the field to update does not exist, it will be created.

If I am using `updateOne` and many documents are positive to the filtering, only the first that mongoDB finds will be updated (*generally the oldest one*).

```
updateMany()
```

Update many, instead, applies the update to all the documents that are positive to the filter.

Cursors

Cursors Keyword

Sort

Change the sorting method of the result set.

```
db.users.find({}).sort({ age : k })
```

ASC ▯ **k = 1** DESC ▯ **k = -1**

Example of Sort

```
db.inventory.find({}).sort({ year : 1 , qty : -1})
```

Ordered by year and for “*parimerito*” they are ordered for qty.

Limit

```
db.users.find({}).limit(n)
```

Limit(**n**) only returns the first n rows.

Skip

```
db.users.find({}).skip(n)
```

Skip(**n**) ignore the first n rows.

Counting documents

```
db.inventory.countDocuments(query)
```

It does not return a cursor; it returns an integer, which represents the number of documents returned by the specified query.

Note that `countDocuments` is deprecated, but mongoDB will still accept this command. Please use instead:

```
db.inventory.find(query).count()
```

Queries on embedded and nested documents

How to perform queries with embedded or nested documents. We perform a filter like before, but now the “value” to test is a document. Let’s suppose that the field size is a document:

```
Size { h : 4 , w : 21 , uom : “cm” }
```

So, we write something like (*for strict equality*):

```
db.inventory.find { size : { h : 2 , w : 3 , uom : “cm” } }
```

Or something like this (*for inequality*):

```
db.inventory.find {  
  “size.uom” : “cm”,  
  “size.h” : { $lt : 15 },  
  “size.w” : { $gt : 15 }  
}
```

We must quote the field too, in this way:

```
< “field_name.attribute_name” : “condition” >
```

Queries on arrays

With square brackets

```
db.inventory.find { dim_cm : [ 14 , 15 ] }
```

The array must contain only those values and in that precise order. So, for the previous example:

- [14 , 15] will match the filter.
- [14 , 16 , 15] will **not** match the filter.
- [15 , 14] will **not** match the filter.

Without square brackets

```
db.inventory.find { dim_cm : 14 }
```

The array must contain that value. So, for the previous example:

- [14 , 15] will match the filter.
- [14 , 16 , 15] will match the filter.
- [15 , 14] will match the filter.
- [15 , 16] will not match the filter.

\$all

```
db.inventory.find { dim_cm : { $all [ 14 , 15 ] } }
```

The array must contain (among all its values) those values without considering the order. So, for the previous example:

- [14 , 15] will match the filter.
- [15 , 14] will match the filter, the order does not count.
- [14 , 16] will **not** match the filter, because it does not contain 15.
- [14 , 16 , 15] will match the filter.

Apply conditions

```
db.inventory.find { dim_cm : { $gt [25] } }
```

The array must contain at least one value greater than 25. So, for the previous example:

- [14 , 15] will not match the filter, because neither 14 nor 15 is greater than 25.
- [15 , 30] will match the filter, because 30 is greater than 25.
- [35 , 30] will match the filter, because 30 and 35 are both greater than 25.

Multiple conditions

```
db.inventory.find { dim_cm : { $gt : 15 , $lt : 20 } }
```

This one is quite complicated... The array must contain: At least one value greater than 15 **and** at least one value lower than 20 **or** at least one value greater than 15 **and** lower than 20. So, for the previous example:

- [14 , 16] will match, because 14 is lower than 20 and 16 is greater than 15.
- [17 , 30] will match.
- [35 , 30] will **not** match, because there aren't numbers lower than 20.

- [17 , 30] will match.
- [12 , 13] will **not** match, because there aren't numbers greater than **15**.

Projection of fields

```
db.inventory.find(
    {status : 'A'} , // Filter.
    { item : 1 , status : 1 , _id : 0} // Projection.
)
```

This query will show the `item` and the `status`, but not the `_id`. The `item : 1` specifies that I want to see that attribute in the result set of the query. The `_id : 0` specifies to suppress the `_id` field.

Pros of projection

Projection permits us to optimise the query on the client-side because more fields mean more bytes to send, so more latency and more bytes to manage.

Projection and embedded documents

```
db.inventory.find( {status : 'A'} , { item : 1 , "instock.qty" : 1})
```

The field `qty` is in the document embedded in the `instock` array.

```
> db.inventory.find( { status: "A" }, { item: 1, status: 1, "size.uom": 1 } )
{ "_id" : ObjectId("5db958068215ec9bc20c6c75"), "item" : "journal", "status" : "A", "size" : { "uom" : "cm" } }
{ "_id" : ObjectId("5db958068215ec9bc20c6c76"), "item" : "notebook", "status" : "A", "size" : { "uom" : "in" } }
{ "_id" : ObjectId("5db958068215ec9bc20c6c79"), "item" : "postcard", "status" : "A", "size" : { "uom" : "cm" } }
```

Projection and arrays

\$slice

How to select specific elements of an array?

```
db.inventory.find( {status : 'A'} , { instock: { $slice : k } } )
```

With `k = -1` the query returns the **last** element of the array `instock`.

With `k = 1` the query returns the **first** element of the array `instock`.

\$elemMatch

It returns the first value of a field of the array that matches a given condition. So in this case, the order inside the array may change the result of the query.

```
db.inventory.find(
    {status : 'A'} ,
    { instock: { $elemMatch : { qty : { $gt : 20 } } } }
```

)

Queries on the type of the field value

Let's suppose that we got this collection:

Inventory

```
{
  { _id : 1 , item : null }
  { _id : 2 }
  { _id : 3 , item : 'book' }
}
```

\$type

I can do queries based on the type of the field.

```
db.inventory.find( { item { $type : k } })
```

The number **k** is an integer which value represents a specific type of data, for example we got:

Type	Number	Alias
Double	1	"double"
String	2	"string"
Object	3	"object"
Array	4	"array"
Binary data	5	"binData"
Undefined	6	"undefined"
ObjectId	7	"objectId"
Boolean	8	"bool"

Date	9	"date"
Null	10	"null"
Regular Expression	11	"regex"
DBPointer	12	"dbPointer"
JavaScript	13	"javascript"
Symbol	14	"symbol"
JavaScript code with scope	15	"javascriptWithScope"
32-bit integer	16	"int"
Timestamp	17	"timestamp"
64-bit integer	18	"long"
Decimal128	19	"decimal"
Min key	-1	"minKey"
Max key	127	"maxKey"

Returning to the collection above if `k = 2` (string) then the query:

- The `_id = 1` will not be shown.
- The `_id = 2` will not be shown.
- The `_id = 3` will be shown.

About the `null` type

A `null` field occupies memory. Yes, it does.

The filter on the `null` type can be also explicitly specified like a query based on the value, because a field of type `null` can have only one possible value which is `null`.

```
db.inventory.find( { item : null } )
```

```
db.inventory.find( { item { $type : 10 } } )
```

Both queries return only the documents with `item = null`. Returning to the collection above:

- The `_id = 1` will be shown.
- The `_id = 2` will not be shown.
- The `_id = 3` will not be shown.

\$exists

```
db.inventory.find( { item : { $exists : true } } )
```

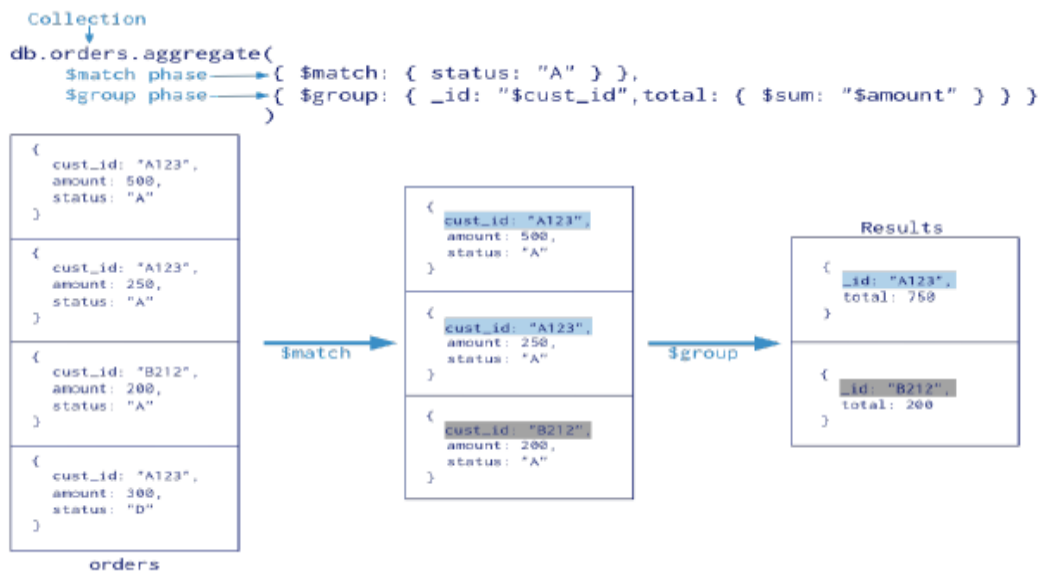
It returns the document only if the `item` field is defined (*it must exist*). Returning to the collection above:

- The `_id = 1` will be shown.
 - Even if it's `null`, the field exists, and it occupies memory.
- The `_id = 2` will not be shown.
- The `_id = 3` will be shown.

Aggregation

Process results set and return computed results. I can write very complex query by simply creating multiple steps to obtain the final result.

Pipeline



1. `$match`: documents of the collection `orders` are filtered following the condition `status = "A"`.
2. `$group` groups the documents following a field, in this case `_id`.
 - a. `$cust_id` will be the new `_id` field name in the result set.

3. Add a new field called `total`, which contains the sum of the field `amount` of each document that remained after the `$match`.

Obvious note about aggregation

This operation is just a query, so the initial collection does not change, it is like SQL, a query does not change the database.

`$lookup`

Forbidden. It performs a **left outer join** to a collection in the same database to filter in documents from the "joined" collection for processing. The `$lookup` stage adds a new array field to each input document. The new array field contains the matching documents from the "joined" collection. The `$lookup` stage passes these reshaped documents to the next stage.

`$unwind`

You can use it, but it must be very well justified. The `$unwind` deconstructs an array field from the input documents to output a document for each element. Each output document is the input document with the value of the array field replaced by the element.

```
$unwind:
{
  path: <field path>,
  includeArrayIndex: <string>,
  preserveNullAndEmptyArrays: <boolean>
}
```

The collection is:

```
db.clothing.insertMany([
  { "_id" : 1, "item" : "Shirt", "sizes": [ "S", "M", "L" ] },
  { "_id" : 2, "item" : "Shorts", "sizes" : [ ] },
  { "_id" : 3, "item" : "Hat", "sizes": "M" },
  { "_id" : 4, "item" : "Gloves" },
  { "_id" : 5, "item" : "Scarf", "sizes" : null }
])
```

The operation is:

```
db.clothing.aggregate( [ { $unwind: { path: "$sizes" } } ] )
```

The result of the unwind will be:

```
{ _id: 1, item: 'Shirt', sizes: 'S' },
{ _id: 1, item: 'Shirt', sizes: 'M' },
{ _id: 1, item: 'Shirt', sizes: 'L' },
{ _id: 3, item: 'Hat', sizes: 'M' }
```

Aggregation example

Each document represents a neighbourhood.

```
{
  "_id": "10280",
  "city": "NEW YORK",
  "state": "NY",
  "pop": 5574,
  "loc": [
    -74.016323,
    40.710537
  ]
}
```

We want to return the **states** with a population greater than 10 million.

For each of them, show the total population.

\$match + \$group

```
> db.zipcodes.aggregate( [
...   { $group: { _id: "$state", totalPop: { $sum: "$pop" } } },
...   { $match: { totalPop: { $gte: 10*1000*1000 } } }
... ] )
{ "_id" : "TX", "totalPop" : 16984601 }
{ "_id" : "FL", "totalPop" : 12686644 }
{ "_id" : "IL", "totalPop" : 11427576 }
```

Other examples

\$group + \$group


```
> db.zipcodes.aggregate( [
...   { $group: { _id: { state: "$state", city: "$city" }, pop: { $sum: "$pop" } } },
...   { $group: { _id: "$_id.state", avgCityPop: { $avg: "$pop" } } }
[... ] )
{ "_id" : "SD", "avgCityPop" : 1839.6746031746031 }
{ "_id" : "NV", "avgCityPop" : 18209.590909090908 }
{ "_id" : "TN", "avgCityPop" : 9656.350495049504 }
{ "_id" : "AR", "avgCityPop" : 4175.355239786856 }
{ "_id" : "WI", "avgCityPop" : 7323.00748502994 }
```

In the query above:

1. We group the neighbourhoods based on the state and the city; then we compute the total population of the city in that state.
2. Then, the result is grouped by the state, and we compute the average population of the cities in each state.

\$group + \$sort + \$group

Here we create 4 new attributes:

- The value of the biggest city.
- The value of the smallest city.
- The value of the biggest population.
- The value of the smallest population.

```
> db.zipcodes.aggregate( [
...   { $group:
...     {
...       _id: { state: "$state", city: "$city" },
...       pop: { $sum: "$pop" }
...     }
...   },
...   { $sort: { pop: 1 } },
...   { $group:
...     {
...       _id: "$_id.state",
...       biggestCity: { $last: "$_id.city" },
...       biggestPop: { $last: "$pop" },
...       smallestCity: { $first: "$_id.city" },
...       smallestPop: { $first: "$pop" }
...     }
...   })
{ "_id" : "RI", "biggestCity" : "CRANSTON", "biggestPop" : 176404, "smallestCity" : "CLAYVILLE", "smallestPop" : 45 }
{ "_id" : "CT", "biggestCity" : "BRIDGEPORT", "biggestPop" : 141638, "smallestCity" : "EAST KILLINGLY", "smallestPop" : 25 }
{ "_id" : "KS", "biggestCity" : "WICHITA", "biggestPop" : 295115, "smallestCity" : "ARNOLD", "smallestPop" : 0 }
{ "_id" : "NC", "biggestCity" : "CHARLOTTE", "biggestPop" : 465833, "smallestCity" : "GLOUCESTER", "smallestPop" : 0 }
{ "_id" : "PA", "biggestCity" : "PHILADELPHIA", "biggestPop" : 1610956, "smallestCity" : "HAMILTON", "smallestPop" : 0 }
```

In the query above:

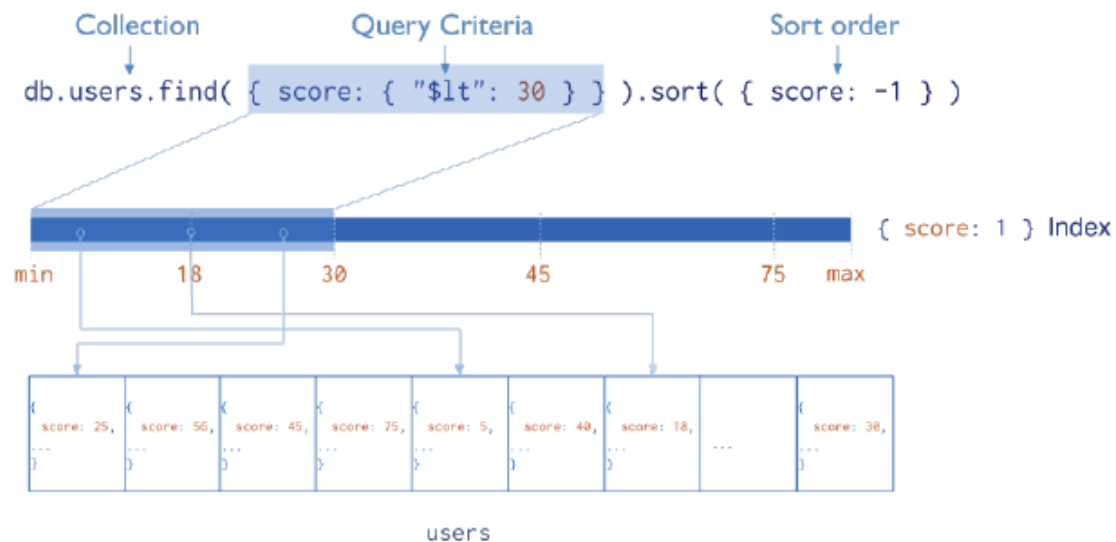
1. We group the neighbourhoods based on the state and the city; then we compute the total population of the city in that state.
2. We sort the result by the total population in ascending order.
 - So, the first will be the smallest and the last will be the largest.
3. Then, the result is grouped by the state, and we compute the 4 attributes specified above using the keyword of arrays (\$last and \$first).

MongoDB Indexes

Indexes are a special data structure that stores a small portion of the collection's data set in an easy traversal form and are built for speed-up a specific query. Without indexes, MongoDB would have to scan every document in a collection to select the documents that match the condition.

The index stores the value of a specific field or set of fields ordered by the value of the field.

The index permits also to return an ordered result sets trough the keyword `sort`.



The index has an entry associated with every value that the field could have. Every entry has a pointer to a document of the collection (*an index works only on the collection level*).

Pros and cons of indexes

Whenever we get a write operation, we must update the index.

An index speed-up a read operation but it can slow-down a write operation, especially if the write operation is frequently performed in the field for which the index has been defined.

The default index

MongoDB has a default index on the `_id` field, which is mandatory for every document in a collection; moreover, the value of this field can be generated automatically by MongoDB during the insertion operation.

This index cannot be dropped, and it has the `unique` property: prevents clients from inserting two documents with the same `_id`.

Index commands

`getIndexes()`

Shows all defined indexes in the database.

`createIndex()`

```
db.users.createIndex( { keys } , { options } )
```

The method creates the index only if an index with the same specification (*same keys and options*) does not exist.

```
db.cities.createIndex(  
  { population : 1 } , // 1 => Ascending order  
  { name : "query for population" } // Indexes can have names  
)
```

dropIndex

```
db.cities.dropIndex( "query for population" )
```

Can we rename an index?

No, we cannot rename an index once created, we must drop and create it again.

Compound Indexes

Indexes defined on more than one field { userID : 1 , score : -1 }

- 1 □ Ascending order
- -1 □ Descending order.



This index can support a query in which we consider as match parameters those and only those two fields in that exact order.

```
.find({ userID : 23 , score : 45 } }) Exploited.
```

```
.find({ score : 45 , userID : 23 }) Not exploited.
```

```
.find({ userID : 23 , score : 45 , name : "Mario" }) Not exploited.
```

In the first two cases the result will be the same (*because both queries request the same thing and they are both correct*), but in the second case, the index will not be used.

Index for embedded documents and arrays

```
{
  _id : 1 ,
  Comments = [
    { test = "aaa" , date : "1022" } ,
    { test = "bbb" , date : "1023" }
  ]
  Ratings = [ 2 , 4 , 9 ] ,
  Gros = [
    { test = "A1" , ty = [ 2 , 4 ] } ,
    { test = "A2" , ty = [ 5 , 7 ] }
  ]
}
```

```
db.posts.createIndex({ "Gros.test" : 1})
```

This index will support queries for the attribute `city` of the embedded document `Gros`; the index will have one key for every value of `ty`, so one key could point to many documents.

```
db.posts.createIndex({ Gros : 1})
```

For each document I'll have an entry of the index. This index will support queries on the entire document, but not queries on specific fields of the document.

```
db.posts.createIndex( { Comments : 1 } )
```

`Comments` is an array of documents: The index will contain two keys, one for each value of the document.

```
db.posts.createIndex( { Ratings : 1 } )
```

`Ratings` is an array of numbers: The index contains three keys, one for each value of the array.

Advanced Indexing

These indexes have not been fully explored during the lectures; they have only been nominated by the teacher. If you want to use them in the project you can do it, but you have to study them on your own.

Geospatial Index

It supports queries based on coordinates.

Text Index

It supports text search (specific word inside a document).

Hash index

Build on the hash value of a field.

Types of Indexes

Unique

MongoDB will reject duplicate values for the indexed field. This works also for compound attributes.

Partial

In the index will be inserted only those documents which respect a certain condition. So only a part of the collection will be indexed.

Sparse

The index contains entries only for documents that have the indexed field. So, the index skips documents that do not have the indexed field. Normally, we got a separate entry for null values. A sparse index simply ignores the document.

TTL

These indexes are used to automatically remove documents after a certain period from the collection. This is useful for deleting old logs and session information that only need to persist for a certain amount of time.

In Memory Databases

In some situations, I do not need to manage a big amount of data (*no big data*), so I can create an application where the entire database can be loaded in the RAM.

An InMemory database grant a very fast access to the data, because we don't read from the disk but from the primary memory (which is million times faster).

Suitable for applications that do not need to write continuously to a persistent medium.

Persistency management

This kind of architecture is designed to avoid data loss even in the event of power failure.

For grant the persistence of the data, generally two approaches are used (*usually together*).

Snapshot

A snapshot is a complete copy of the database currently stored in the primary memory.

A snapshot is memorized in a certain area of the disk and generally is used to load the database in the RAM during the start-up.

Updates of the snapshot is done periodically.

Snapshots can be replicated and stored in other machine of the cluster, the updates of those replica follows the same technique that we saw before.

Transaction log

The log of all the transactions computed (*like the commit log*) are stored in a light file in the disk.

The update of the transaction log is generally done after each commit (*we don't group writes together, because is very important to maintain this file updated*); write operations in the log are done in an ACID manner for grant the maximum consistency level.

In case of disaster, we can recover the entire data partition by executing all the transactions, obviously following the same order.

TimesTen

Produced by Oracles and implements a similar SQL-based relational model. All data is memory resident.

Periodic snapshots

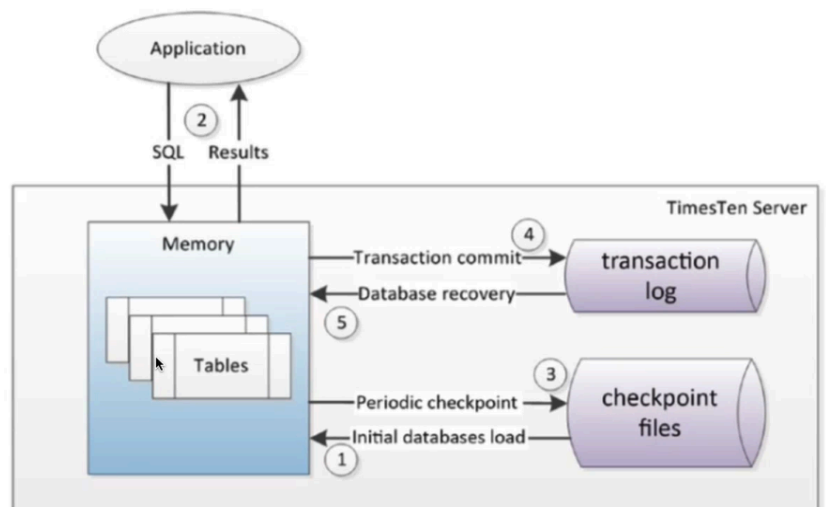
Also called “*Checkpoint files*”. Periodically the application does a checkpoint (a snapshot of the primary memory) and loads it in the disk. It is also used to load the database when the machine starts.

Transaction log

We write the commits of all the write transactions in a log file (like the commit log). The records of the log are like: “At this time t, has occurred a transaction with this value X”.

By default, the disk writes are asynchronous (they are performed in a batch way in a second moment).

To ensure ACID transactions we need to configure the server for make synchronous writes to a transaction log after each write, at the cost of a performance loss (every access to the log is an access to the disk).



Redis

Redis is a InMemory KVDB. The values that Redis can store consist mainly of strings and various types of collections of strings (lists, hash maps, ...).

Redis is mostly used for creating queues of requests that can be directed to our real server.

Redis indexes

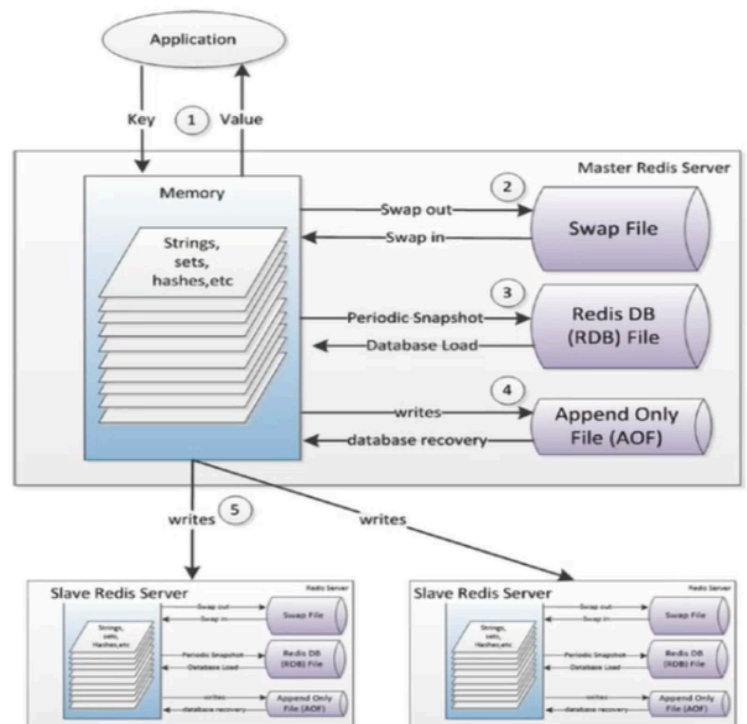
Only primary key lookups are supported, no secondary indexes are available.

Redis virtual memory

Mechanism for storing an amount of data larger than the server's primary memory.

Old (less frequently used) keys are swapped-out to the disk (in the swap file) and recovered if needed.

Redis Replication System



Redis has a Master-Slave replication system; but designed to reduce the level of persistence on the master, so for reduce the probability on un-availability in case of failure of the master.

The master receives the write operations and with an eventually consistent procedure, also the replicas will be updated.

Persistence in Redis

Like TimesTen, Redis do periodic snapshots and keeps a transaction log in an append-only file.

This thing is done by the master and the replicas.

HANA DB

Is a relational database which combines an in-memory technology with a columnar storage solution. In the case of HANA, we need an optimised hardware configuration.

As we said in the OLTP & OLAP chapter:

- The columnar part is devoted to analytics but is often used also for some OLTP operations (*like simple queries that read only one attribute*).
- The classic relational part is devoted for OLTP operations (*typical transaction, so write operations*) and it is stored in-memory.

By default, the dataset is loaded in memory by demand. However, can be configured to load in memory a set of columns or entire tables from the beginning.

Load the snapshots (1)

The first thing to do is loading the last snapshot from the disk to the memory.

Read/Write operations in the row part (2)

Some write and read operations in the row part occurs.

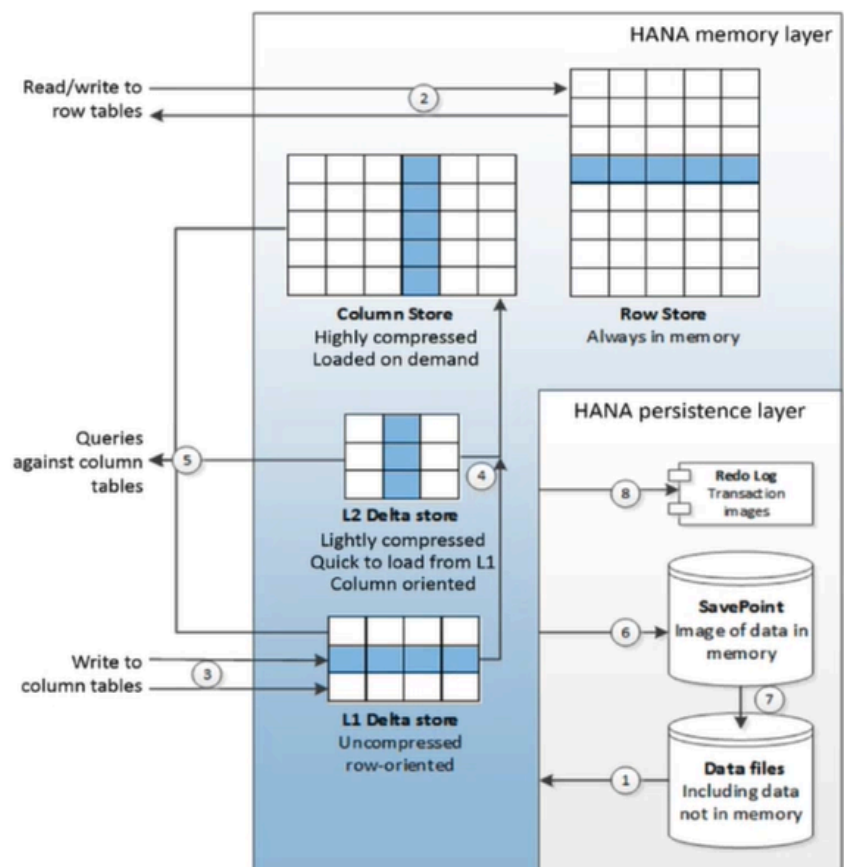
Those operations are performed in the in-memory part which is organised in rows.

Read/Write operations in the columnar part (3)

Some write and read operations in the columnar part occurs.

These operations are recorded in-memory in a cache organised in rows (*no compression*) called L1 Delta Store.

Delta stores (4)



Data to be used for analytics, they have to be loaded in a temporary cache called L2 Delta Store, which is organised in columns instead (*lightly compressed*).

After that, they are slowly loaded in the real column store (*highly compressed and better organised*).

Analytics (5)

Analytics are performed in the Column Store, L2 Delta Store, and the L1 Delta Store.

Why do we need all three? The column store may not be always updated, some information could be in the delta stores.

Persistence (6) (7) (8)

For assure the persistence we got:

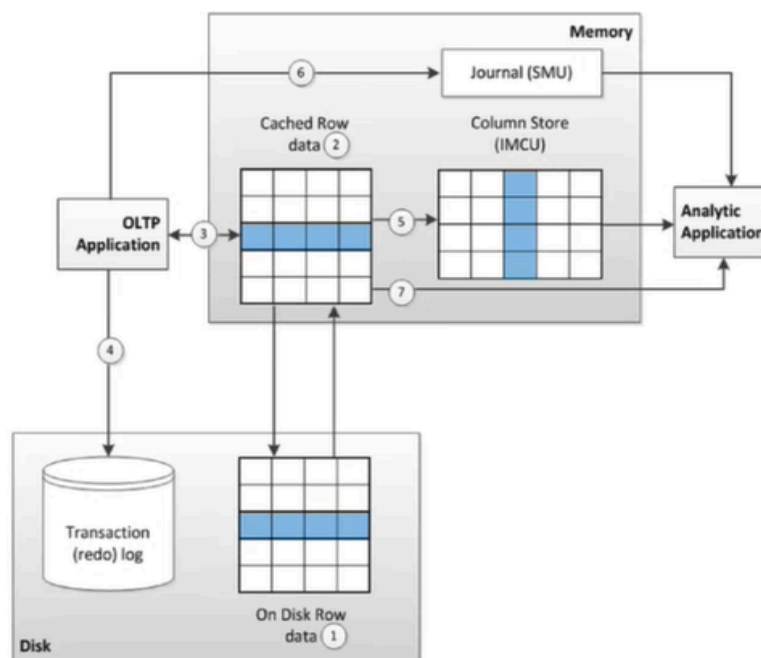
- **Transaction log** (*append-only files*): which store all the transaction images (*the value and the timestamp*) (8).
- **Snapshots**: A subsystem that periodically creates snapshots (6) of the memory (SavePoint) and loads them in the disk (7).

Oracle 12c

InMemory solution for relational databases; designed to meet the modern requirements:

- Scalability
- Performance
- Cloud management

Notice that the first 2 are the main problems of the relational databases in modern solutions.



Where are the files? (1,2)

The persistent side is contained in a classic relational database with row storage (*snapshot*); but it is cached in the primary memory (*keeping the row organization*) when the system starts.

OLTP Operations (3)

An OLTP (*simple queries, ACID transactions*) application primarily reads and writes from the memory (*in the row part*).

Persistence (4)

Any committed transaction is always written immediately to the **transaction log** in the disk.

The updates to the disk row storage are performed in a second moment when the system has a low load.

In case of failure we don't care, just by using the transaction log we can redo all the transactions.

Analytics (5)

Periodically, the InMemory row part is loaded into a columnar representation (*IMCU – In Memory Column Unit*); mainly used for analytics.

Journal (SMU) (6) (7)

As transactions occur, changes are written to redo log buffers in memory and then periodically flushed to the redo log files on disk.

When analytics occurs, a new record in the journal is written. This file helps a lot in terms of performance because it tells the system (*when analytics occurs*) if the data has been updated. If so, the analytic must be computed from scratch.

But if the journal tells that data has not been updated, then the analytic can return the same result that gave the last time that has been executed.

Neo4J

Neo4j is a native graph database, built from the ground up to leverage not only data but also data relationships.

Unlike traditional databases, which arrange data in rows, columns and tables, Neo4j has a flexible structure defined by stored relationships between data records.

Each data record, or node, stores direct pointers to all the nodes it's connected to.

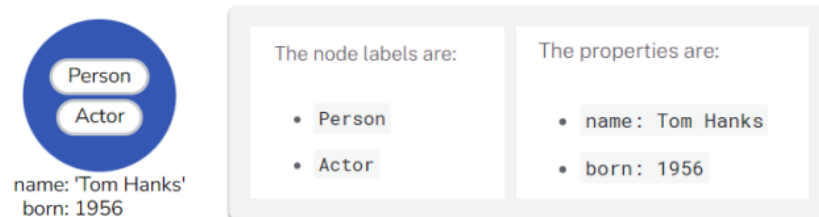
Neo4j's design allows to perform queries with complex connections orders of magnitude faster, and with more depth, than other databases.

Node

Describe an entity of a domain. It can have some labels to define what kind of node it is.

Labels

They shape the domain by grouping nodes into sets where all nodes that have a certain label belong to the same set.



Relationships

It connects two nodes. Relationships organize nodes into structures, allowing a graph to resemble a list, a tree, a map or a compound entity.

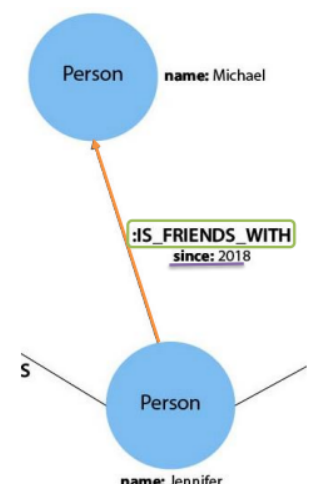
A relationship must have exactly one **relationship_type**, typically expressed in UPPER_CASE. A relation can be recursive.

Properties

A relationship can have associated multiple **properties**.

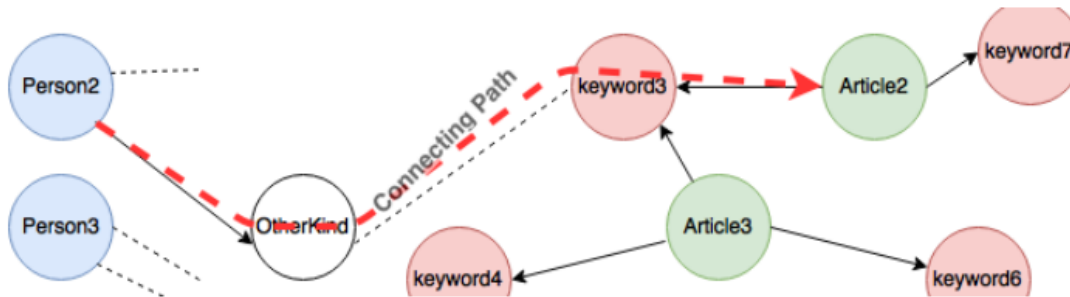
Properties can be removed or added dynamically.

Properties can be of many types: numbers, string, boolean, timestamp, etc...



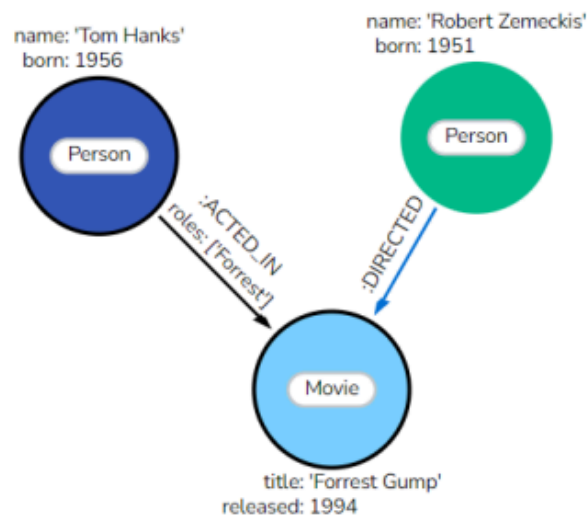
Traversal and path

Traversing a graph means visiting nodes by following relationships according to some rules. The traversal result could be returned as a path.

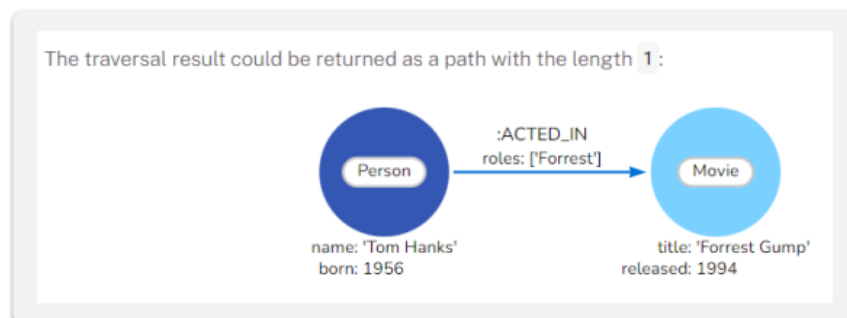


En example of traversal:

To find out which movies Tom Hanks acted in according to the tiny example database, the traversal would start from the `Tom Hanks` node, follow any `ACTED_IN` relationships connected to the node, and end up with the `Movie` node `Forrest Gump` as the result (see the black lines):



So, the resulting path will be:



Indexes

The main reason for using indexes in a graph database is to find the starting point of a graph traversal.

Once that starting point is found, the traversal relies on in-graph structures to achieve high performance.

An index can be:

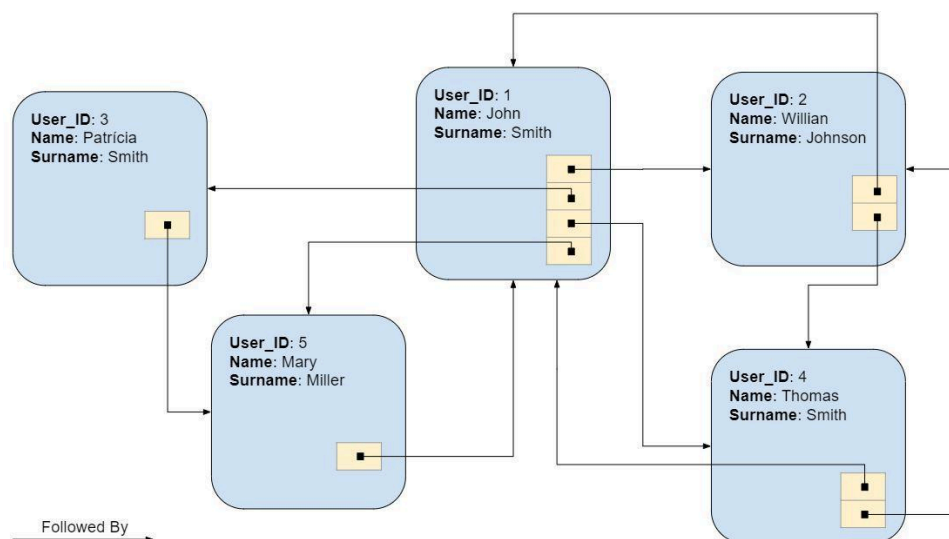
- **Single property:**
 - The index refers to a single property for a given label.
 - It can match ranges.
- **Composite:**
 - The index refers to multiple properties for a given label.
 - It can match only by equality.

Index free adjacency

Graph databases take advantage of index-free adjacency for traversing data more efficiently and it is a key concept in graph databases like Neo4j.

The relationships between nodes are stored directly within the node data itself, without the need for an additional global index to track connections between them. This allows for very fast access to relationships without having to traverse an entire data structure to find them.

You can think that each node actually holds a mini-index to all nearby nodes, making traversal operations extremely cheap.



So instead of do a time-consuming graph traversal, we just read the mini-index inside a node to get all the connections.

Guidelines for selecting a database

Everything driven by the requirements, common sense, and experience.

What guide us?

Non-functional requirements

- Volume of reads and writes
- Tolerance for inconsistent data in replicas
- Availability and disaster recovery requirements
- Latency requirements.

Key-Value databases

Redis, Riak, DynamoDB (*mix of key-value and document*).

Preferrable when:

- The application has to handle frequent but small read and writes operations.
- Simple data models with none or few relations.
- Simple queries.

They do not allow too much flexibility (*on the things we can do, not the flexibility of the scheme*) and the possibility to make queries on different attributes.

Some examples of use:

- Caching data from relational databases to improve performance.
- Tracking transient attributes in a web application, such as shopping cart.
- Storing configuration and user data information for mobile applications.
- Storing large objects, such as images and audio files.

Document databases

MongoDB, CouchDB.

Preferrable when:

- We got vary attributes, in terms of number and type.
- Elaborate huge amount of data.
- Complex queries (*on different type of attributes*) with the use of indexes and advanced filtering.

Pros

- We got high flexibility on the things we can do.
- High performance and easy to use.
- Embedded documents allow us to store together attributes frequently accessed together.

Some examples of use

- Backend support for sites with high volumes.
- Manage data types with variable attributes, such as products.

- Tracking variable types of metadata.
- Application that uses JSON data structures.
- Application benefiting from denormalization.

Software Engineering Recap

“Everyone here takes 30 cum laude at the exam, so I don’t need to do this part”

cit. Prof. Pietro Ducange

When you design big applications, you should follow a scheme that ensures a correct production of the software.

The scheme is divided in steps called “*workflows*”.

Every workflow requires some time, but some workflows could overlap.

1. Requirements extraction.
 - Often the customer doesn't know what he wants, so be patient.
 - We must understand what the system must do and who will use it.
2. Requirements definition.
 - Define functional requirements and non-functional ones.
3. Use Case definition.
 - Expressed with diagrams for a better understanding.
4. Identification of data structures and main procedures.
5. Refinement of data structures and DB design.
6. Implementation & Test.
 - We create/improve a prototype and then we test it.
7. Deploy the System.

Functional Requirements

Functional Requirements express what the application should do the job, not how the app does it.

Very high-level states, that express some input-output behaviours, so they describe the output that I desire from the system.

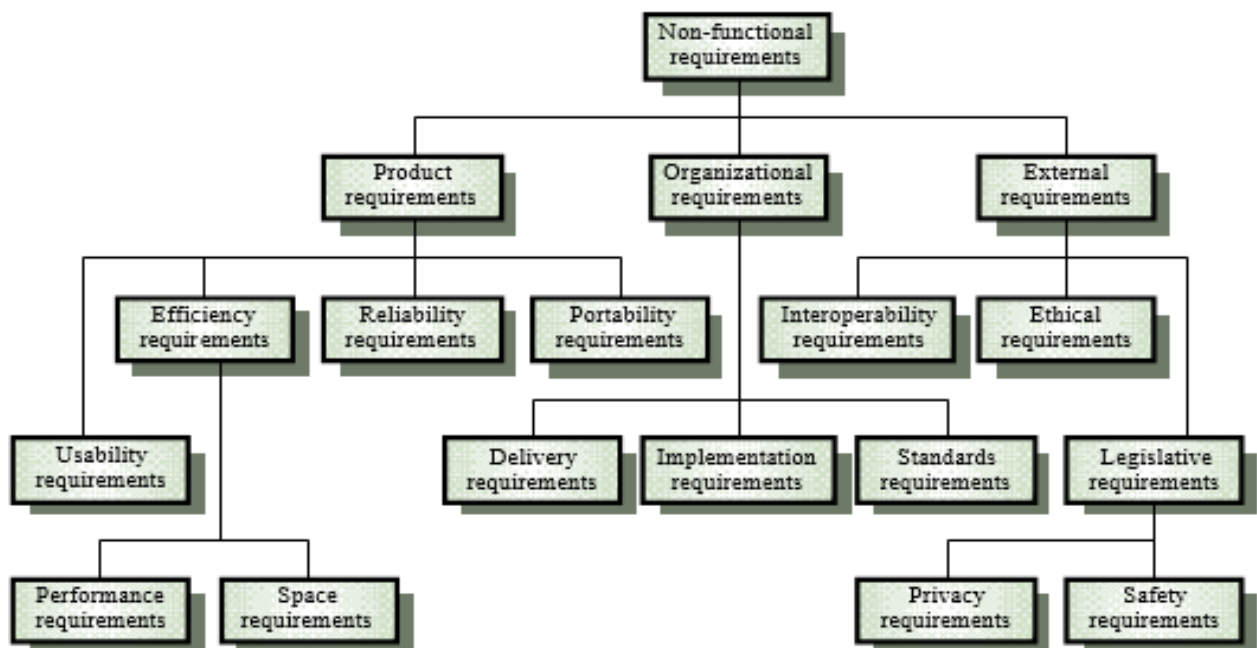
“The system must send a confirmation email whenever an order is placed”. (a sort of cause-effect relationship).

Not Functional Requirements

Not Functional Requirements express properties and constraints, if these are not met then the system is useless.

Can be divided in three main categories:

- Product Requirements
 - Requirements which specify that the delivered product must behave way.
 - “The response time must be below 1 second”.
- Organisational Requirements
 - Requirements which are a consequence of organisational policies and procedures e.g., *process standards used*, implementation requirements, etc.
 - “Must be realized in C++”.
- External Requirements
 - Requirements which arise from factors which are external to the system and its development process :
 - “Legislative, privacy and safety requirements”.



Use Case and Actors

We define how the system interacts with the external environment.

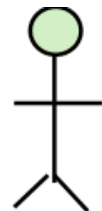
A use case describes the steps that should be performed to complete a task, in other words how the task should be completed.

We need to identify the actors, prerequisites, post requisites and what is happening in the middle.

An actor is someone or something that somehow interacts with our system for obtain something.

An actor can really be everything:

- Human or Animal.
- Peripheral device (hardware).
- External system or subsystem.
- Time or time-based event.



Use case is a sequence of actions that can be expressed with use-case diagram.

Use case diagram

Use case diagrams describe what a system does from the standpoint of an external observer. The emphasis is on what a system does rather than how.

Use case diagrams are closely connected to scenarios.

A scenario is an example of what happens when someone interacts with the system.

Include → A base use case may be dependent on the included use case.

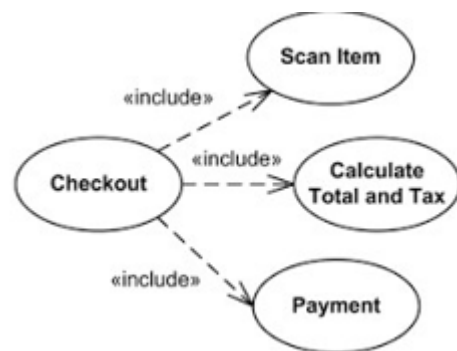
Without the included one the base case is incomplete, because the included use cases represent a subsequence of the base use case.

Base → Included.

Extend → The extended use case is dependent on the base use case.

Without the extended one the base case remains complete, because the extended one represents additional functionality.

Extended → Base



Data Modelling

Provide a description of the data and the relationship.

Entity sets and relationships sets do not depend on the type of Database and DBMS.

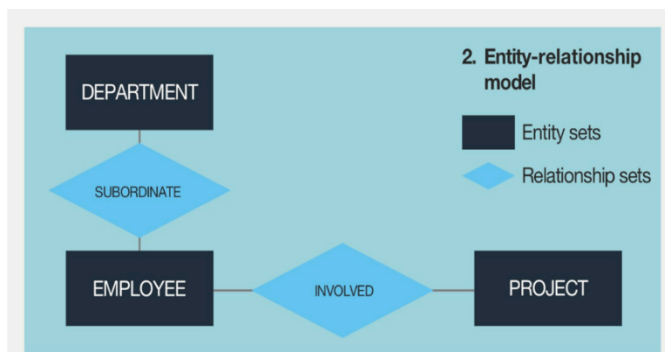
The overall Data Modelling process includes **three stages**:

Data analysis

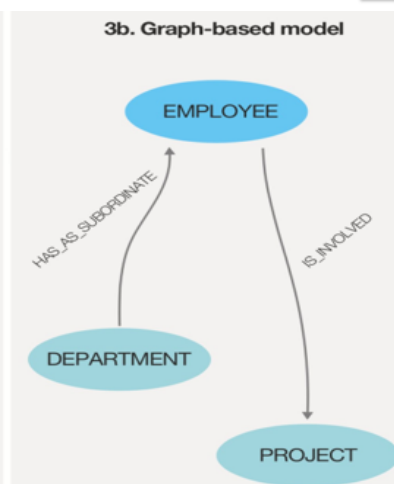
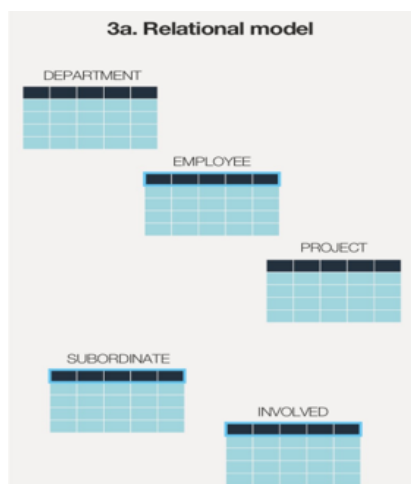
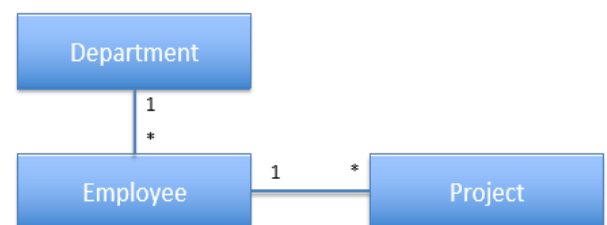
1. Data analysis	Order no. 112 Date: 07/14/2015
Goal	For project monitoring purposes, employees, work, and project times should periodically be logged per department.
1.	Employees report to departments, with each employee being assigned to exactly one department.
2.	Each project is centrally assigned a unique project number.
3.	Employees can work on multiple projects simultaneously; the respective percentages of their time are logged.
4.	...

Designing a conceptual data model

Entity-Relationship Model □ UML Analysis Classes

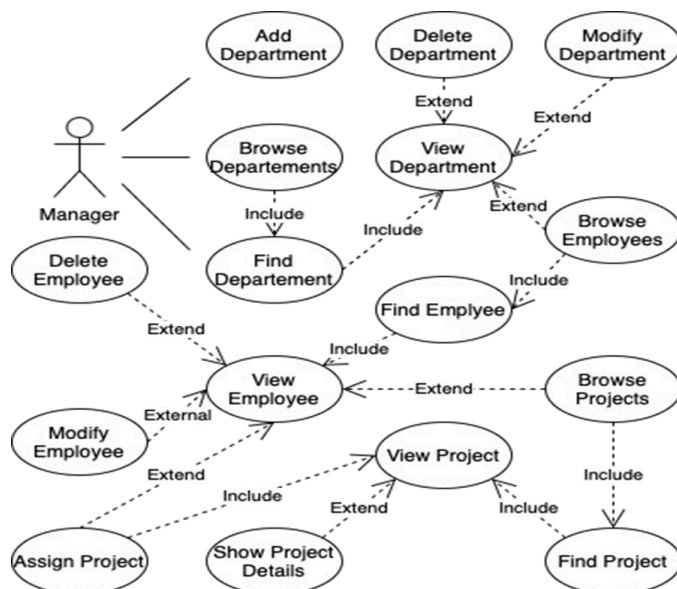


“*” equals to “many”.



Use Case Diagram Example

1. A Manager can browse the list of Departments
2. A Manager can find a Department (By Name)
3. A Manager can add a new Department
4. A Manager can select and delete a Department
5. A Manager can select and modify the description of a Departments
6. Once a Department has been selected, a Manager can browse the list of its Employees
7. Once an Employee has been selected, a Manager can browse the list of his/her Projects
8. Once an Employee has been selected, a Manager can assign a Project to him/her
9. Once a Project has been selected, a Manager can view the its details



Java Recap

Java is an interpreted language, it is interpreted because it works with a **Java virtual machine** which executes the code, you don't need a specific compiler, it works with different CPUs.

It is strongly object oriented, a robust language, architecture neutral and it is cross-platform.

Some keywords:

- Java is Simple: very high level.
- Java is Secure and Portable: They needed a language capable
- Java is not compiled, but it's Interpreted:
 - By the java virtual machine, which executes the code.
- Java is an Object-Oriented programming language.
 - Java is object oriented, so it's structured in classes.
- Java widely use the concept of class extension.
 - Encapsulation
 - Polymorphism
 - Inheritance
- Java is Architecture neutral and Platform independent.
 - Java is cross-platform because the program runs everywhere there is the Java virtual machine.
- Java has many libraries for distributed programming.

Java can be embedded in web pages; browsers can support Java (it's not used anymore in this way).

A well-organized java program should consist of a set of classes, each class should represent a real entity.

A unique huge class is not good, it is better to have well organized and simple classes.

Every program has a class with a main method, it's the first method that the JVM executes.

In Java there are the concepts of packages and methods:

The **packages** are a way to categorize classes, it is the next level hierarchy after classes.

The **methods** are the functions that enable the interaction of data between classes.

How is a Java file ‘compiled’ and executed?

Every file contains only one class definition, and the class must have the same name as the ‘.java’ file. In Java you should have a file for every class.

The java source code compiler (*javac*) translates the source code to bytes (*java byte code*).

Then the JVM loads and executes the files (the JVM may compile them to native code internally).

The main method tells the JVM where to start and execute the program.

This is an example of a class to write Hello world:

```
public static void main(String[] args){  
    System.out.println("Hello world!");  
}
```

If you save this file with the .java extension, if you run in the terminal inside the folder where the .java file is saved, and you write `java file_name.java` the code is executed.

How to Write a Java program

Every class must be defined.

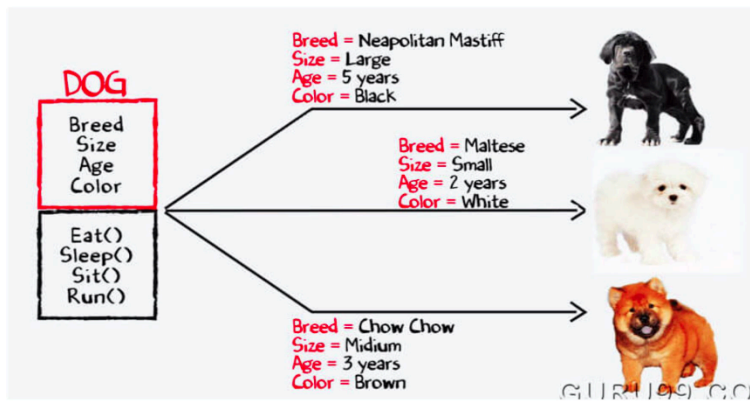
A class has attributes and methods.

```
public class Dog{  
    String breed;  
    int age;  
  
    public String getBreed(){  
        return(breed);  
    }  
    public static void main(String[] args){  
        Dog maltese = new Dog();  
        maltese.breed = "Maltese";  
        maltese.age = 3;  
        System.out.println(maltese.getBreed());  
    }  
}
```

Encapsulation

Every class has attributes and methods, you create a sort of box which represents data structures you need and methods.

Methods perform operations on the attributes of the instance of the class that called the method.



In red there are data fields and in black the methods, for example you can declare the class below for this example:

```
// Class Declaration
public class Dog {
    // Instance Variables
    String breed;
    String size;
    int age;
    String color;

    // method 1
    public String getInfo() {
        return ("Breed is: "+breed+" Size is:"+size+" Age is:"+age+" color is: "+color);
    }

    public static void main(String[] args) {
        Dog maltese = new Dog();
        maltese.breed="Maltese";
        maltese.size="Small";
        maltese.age=2;
        maltese.color="white";
        System.out.println(maltese.getInfo());
    }
}
```

In the main class a new object is instantiated with the **new** operator, and then the attributes of the object are initialized, but this is strange because in every Java program you should use the constructor, you should not allow to directly access the parameters of a class, you must create the constructor, it isn't safe to allow direct access of attributes of a class.

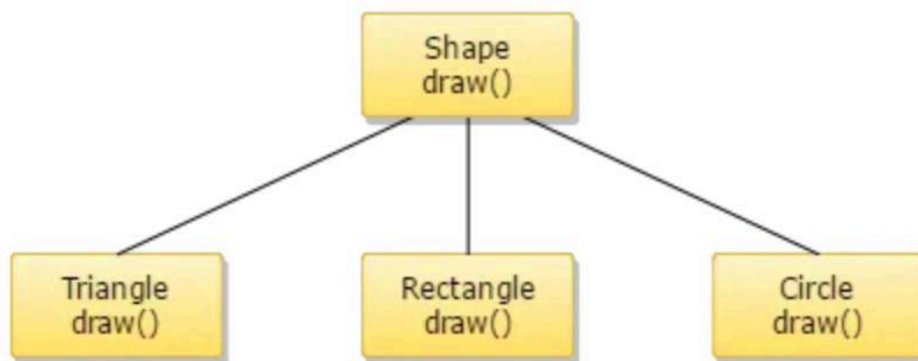
Polymorphism

I can have a variable, function or object that has more than one form.

I can have a common class that specialize in many subclasses, for example:

- Common class: Shape
- Subclasses: Triangle and Rectangle.

I can use the code of Shape in Triangles and Rectangles.



I have the overwriting too; I can redefine methods (so write them differently but using the same name) in the subclasses.


```

public class Animal {
    public void makeNoise()
    {
        System.out.println("Some sound");
    }
}

class Dog extends Animal{
    public void makeNoise()
    {
        System.out.println("Bark");
    }
}

class Cat extends Animal{
    public void makeNoise()
    {
        System.out.println("Meawoo");
    }
}

```

```

public class Demo
{
    public static void main(String[] args) {
        Animal a1 = new Cat();
        a1.makeNoise(); //Prints Meowoo

        Animal a2 = new Dog();
        a2.makeNoise(); //Prints Bark
    }
}

```

The superclass defines the characteristics in general, then the dog class is a specific class which is a subclass of the superclass.

We can't assign to a cat an object from an animal class, but we can do the opposite.

The keyword `extends`, extends the properties of a class with all properties of the superclass.

Packages

Java permits you to use a big collection of packages.

I can import classes from another package and use them like they are written by me.

A package consists of a set of classes.

I can use those classes with the command `import`, which tells the compiler to make available classes and methods of another package, it is the same thing as they include in c++.

A package should contain classes that are related themselves.

The `main` method indicates where to begin executing a class.

References and Primitive data types and Functions

Java has two types of entities.

The primitive type is stored in primitive variables, instead the reference variable stores only the address of the entity.

When I pass a primitive type to a function, the passing is made by value, instead if I pass an object then the passing is made by reference.

Now let's create a new class:

```
public static void modifyNumber(int i){
```

```
    int j=10;
```

```
    i =i*j;
```

```
}
```

```
public class Test01{
```

```
    public static void main(String [] args){
```

```
        List<integer> numbers = new ArrayList<integer>();
```

```
        numbers.add(10);
```

```
        numbers.add(20);
```

```
        System.out.println("Before numbers contains: " + numbers);
```

```
        for(Integer i:numbers){
```

```
        modifyNumber(n);  
    }  
    System.out.println("After numbers contains: " + numbers);  
}  
}
```

For this method we use the call by value method to pass the variables, only the copy will change, but the array will not be affected by that, so the two println will show the same values.

```
class Person {  
    String name;  
    String surname;  
}  
Person(String name){  
    this.name=name;  
}  
public static void modifyName(String newName){  
    this.name=newName;  
}  
public class Test02{  
    public static void main(String [] args){  
        List<Person> people = new ArrayList<people>();  
        people.add(new Person(name: "Alessio"));  
        people.add(new Person(name: "Pietro"));  
        System.out.println("Before : " + people);  
  
        for(Person p: people){  
            p.modifyName(newName: "NewName");  
        }  
        System.out.println("After: " + people);  
    }  
}
```

```
}
```

This time we are passing an object which corresponds to a reference (a pointer in c++), so the output is different between the two println in this case.

The primitive data types are numbers, characters and boolean values.

Integers can be represented by byte, short, int and long;

Real numbers can be represented as float or double;

Characters can be represented as char.

An object is a pointer to the class.

Input in Java

Before the main method we must add the keyword **static** because the main method is the first thing that the Java virtual machine starts executing, without it, the Java virtual machine it would have nothing to start executing from, all methods are called by value.

A method that is not static is an instance method.

```
import java.util.Scanner;
```

```
Scanner scanner = new Scanner(System.in);
```

```
int numero = scanner.nextInt();
```

Java Code Notes

Try & Catch & Finally block

When executing Java code, different errors can occur coding errors made by the programmer, errors due to wrong input, or other unforeseeable things.

When an error occurs, Java will normally stop and generate an error message.

The technical term for this is: Java will throw an **exception** (*throw an error*).

The `try` statement allows you to define a block of code to be tested for errors while it is being executed.

The `catch` statement allows you to define a block of code to be executed if an error occurs in the `try` block.

The `try` and `catch` keywords always come in pairs.

The `finally` statement lets you execute code, after `try...catch`, regardless of the result.

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            // Code to try.  
            int[] myNumbers = {1, 2, 3};  
            System.out.println(myNumbers[10]);  
        } catch (Exception e) {  
            // Code to execute if an exception occurs.  
            System.out.println("Something went wrong.");  
        } finally {  
            // Code to run in any case.  
            System.out.println("The 'try catch' is finished.");  
        }  
    }  
}
```

}

}

Throw keyword

The `throw` statement allows you to create a custom error.

The `throw` statement is used together with an **exception type**.

There are many exception types available in Java:

- `ArithmeticException`
- `FileNotFoundException`
- `ArrayIndexOutOfBoundsException`
- `SecurityException`
- ...

```
if (age < 18) {  
    throw new ArithmeticException("Access denied");  
}
```


Connectors

When you need your application to interact with a database, one of the problems that happens is that the types of the database and Java are not one to one, there are some differences, this phenomenon is called Object-Relational Impedance Mismatch.

DBMS provide connectors (*drivers*) for certain programming languages.

Each DBMS has its own API (*low portability*), so the application became DBMS-dependent.

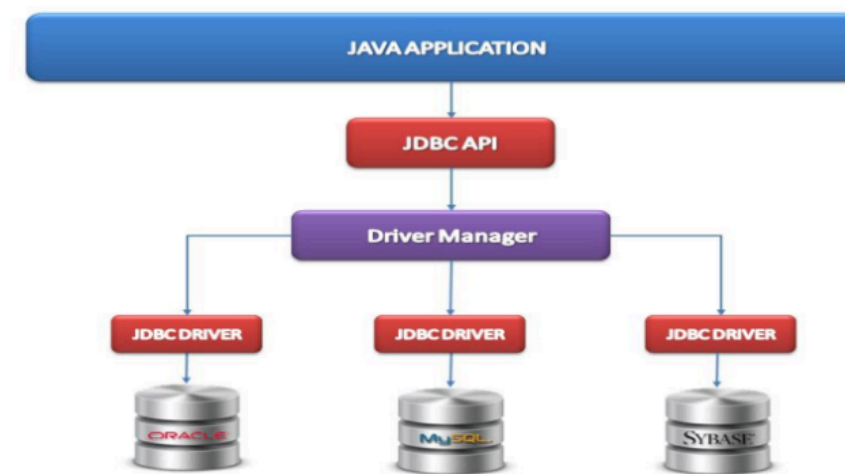
Microsoft created ODBC (*Open DataBase Connectivity*) which defines a unified API for accessing databases in C.

A bit later Java introduced JDBC (*Java DataBase Connectivity*), that is equivalent to ODBC, but the APIs are more “java-specific”.

The class that contains all of this in the package `java.sql`.

So JDBC is a new layer between my java application and the Driver Manager.

For each DBMS there is a JDBC driver, and each driver communicate with the Driver Manager who it's the only one who can communicate with the application.



Driver Manager

Responsible of managing all the JDBC Drivers and is the responsible of the connection to the database, it is the main class

The application can set some parameters like *(some of them are obviously mandatory)*:

- Type of Driver.
- Type of DBMS.
- IP address of the host where the DBMS is located.
- Credential of a user to connect to the DBMS.
- Name of the Database scheme.

Connection to MySQL

```
package it.unipi.lmsd.example;

import java.sql.*;

public class ExampleConnect {

    public static void main(String[] args) {
        Connection connection = null;
        try{
            connection = DriverManager.getConnection( url: "jdbc:mysql://localhost:3306/unipi", user: "jose", password: "jose");
            // Do something...
        } catch (SQLException e) {
            throw new RuntimeException(e);
        } finally{
            if (connection != null) {
                try { connection.close(); } catch (SQLException e) {}
            }
        }
    }
}
```

Returns from the DriverManager a connection to the DB specified by the connection string

At the end of all the operations we have to close the connection

The `getConnection` method creates a connection with the DB, the first parameter is the url of the database, other parameters are the username and the password that must be the credentials of an existing user of the database.

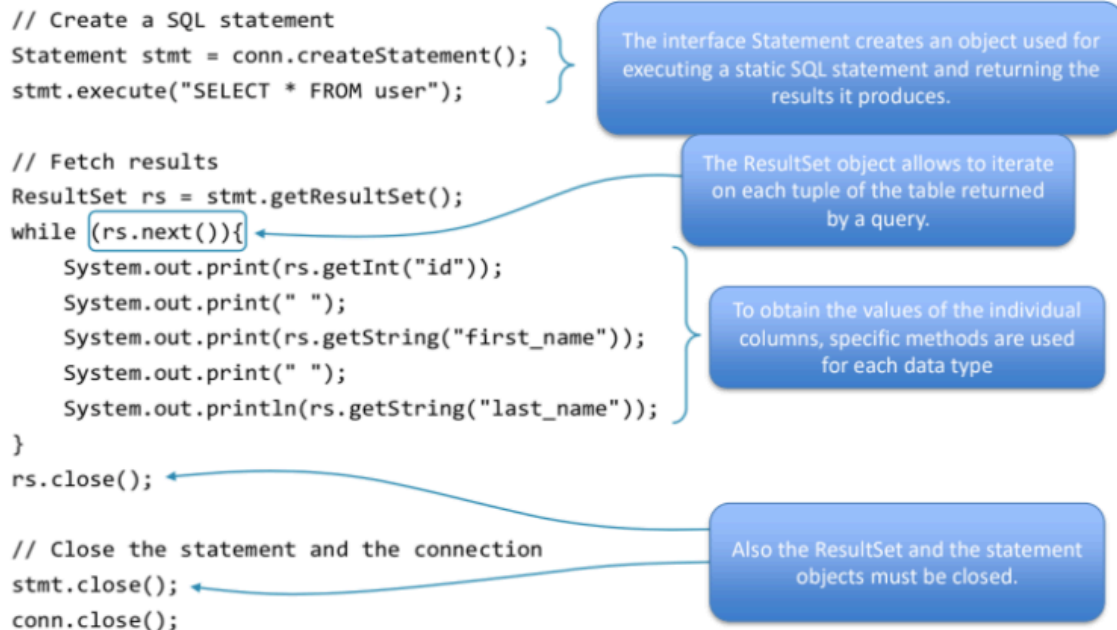
The connection object always needs to be closed at the end of communications.

We put the connection in a `try` for not block the app in case of fail.

Perform Queries and Prepare Statements

After connecting to the DataBase, I have the connection object and for perform queries or operation to the database I must call its method.

At the end, the object must be closed!



I can pre-fill a statement for execute a query multiple time but changing some parameters.

Once you get the connection with the DB you can create interactions with your database, the `createStatement()` function creates a statement object which is needed to perform queries on DB, the resources must be closed in reverse order of the order when they were instantiated.

Update Tables

INSERT, **UPDATE** and **DELETE** statements do not return a ResultSet object.

The **`executeUpdate()`** method executes the statement and returns the number of rows actually inserted/modified/deleted.

```
String sql = "INSERT INTO user SET username = ?, first_name = ?, last_name = ?, "
            + "email = ?, password = SHA1(?)";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, "alice.ferrari");
pstmt.setString(2, "Alice");
pstmt.setString(3, "Ferrari");
pstmt.setString(4, "alice.ferrari@example.com");
pstmt.setString(5, "1234");

int insertedRows = pstmt.executeUpdate();
System.out.println("Rows inserted: " + insertedRows);
```

Executes the statement and returns the number of inserted rows

? is the placeholder where the parameters are placed, so ? will be replaced.

The `executeUpdate()` method executes the statement and returns the number of rows inserted, modified, or deleted.

If we don't know what kind of operation we make (*insert/update/delete*), we can use the `execute()` method which returns true if it is an update and false if it is a deletion.

Maven

Maven is a software project management tools, which helps you to manage the software production cycle.

Is also a way to standardize these thighs, program, documentation etc.

The main feature of Maven is that you don't need to include libraries manually in your application.

We will use Maven for managing the dependencies in our Java programs (*avoid to manually add/update libraries*).

The POM File

Every Maven project has a POM file.

Maven can be configured through the configuration file, an XML file called "POM file".

This file contains all the configuration details used by Maven for build the project.

When we have a new dependency, we manually add new fields to the dependencies list.

groupid □ Uniquely identifies a project across all projects.

artifactid □ The name of the jar without the version.

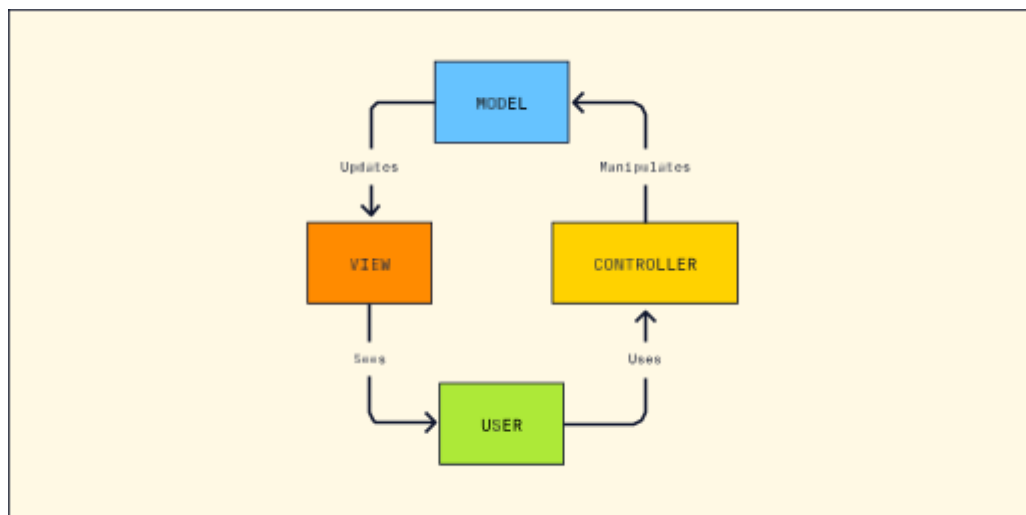
By default, Maven will download form the central repository.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>exercises</groupId>
  <artifactId>exercises</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <dependencies>
    <dependency>
      <groupId>mysql</groupId>
      <artifactId>mysql-connector-java</artifactId>
      <version>8.0.17</version>
    </dependency>
  </dependencies>
  <build>
    <sourceDirectory>src</sourceDirectory>
    <plugins>
      <plugin>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.8.0</version>
        <configuration>
          <source>1.8</source>
          <target>1.8</target>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Java MVC

MVC stands for “*Model-View-Controller*” which are the three main components of design analysis of your software, it is commonly used in modern applications.

The idea behind MVC consists in divide the code in sections and each section has a specific purpose.



Model

Encapsulates the data structures of the application.

It is the part of the code that should be responsible to implement the entities.

You should have a java class for each entity, and this should map how you store data in the DB.

The classes in the directory `model` should have:

- Attributes
- Constructor & Destructor
- Get & Set

View

It is the front-end part of the application, so it offers the user interface for the user.

So, it is responsible of the visual output of the application.

In a web application the view part is composed by `html` and `css` files.

Its main role is to receive the input from the user and to display it with the output in the best way.

It is generally independent (*unaware*) of the application's business logic and data manipulation (*so of the model and controller part*).

Controller

A controller is just a class like others.

A controller is responsible for controlling the way that a user interacts with the application.

The controller contains the flow control logic for an application:

- Interacts with the model layer to manipulate data.
 - Returns the information to the user by modifying a value in a class of the model part.
- Interact with the view layer to take the input from the user.
- Defines what is a task and how the task is represented.

Its purpose is to handle and process the input provided by the user (*preprocess input*) and act as intermediary between the user and the models of the applications.

Example of MVC: To-do list application

Requirements

1. This app will let users create tasks and organize them into lists.
2. The tasks could have large font or be a certain colour.

MVC part

- The model in a todo app might define what a “*task*” is and that a “*list*” is a collection of tasks.
 - So, we got a class for the `task` and a class for the `list of tasks`.
- The view code will define what the todos and lists looks like.
 - The GUI.
- The controller could define how a user adds a task or marks another as complete.
 - The Controller connects the view’s `add button` to the model part (*so to the data*).
 - Example: When you click “`add task`” the model adds a new task in the task list.

DAO & DTO

Let's use the case of a Bookshop application.

Model

In this class are only specified:

- Attributes
- Definitions of Constructors & Destructors
- Definitions of Getters & Setters.

Why only the definition and not the implementation too?

I don't know, the model defines just the possible operations.

DAO - Data Access Object

Every class of the model (`book.java`) has the relative DAO class (`bookDAO.java`).

The baseDAO

In the project there is a class called `baseDAO.java`, where I define a generic connection to the database.

So, it contains common functionalities among the DAO classes.

What contains a DAO class?

Every DAO class is defined as an extension of a baseDAO class.

```
public class PublisherDAO extends BaseDAO
```

The DAO class carries the implementation of the methods defined in the model and have the logic to interact with the objects.

Every time you interact with objects you must use DAO, that is an intermediary between object and logic.

The DAO class and the Database communication

The DAO class also contains all the operation with the database (that concerns the class of the DAO itself).

So, continuing the example of the publisher, the `publisherDAO.java` will contains all the methods for:

- Insert a new publisher in the database.
- Delete a publisher from the database.

- Get a publisher form the database given the `publisherID` and return it like a publisher object.
- ...

A few words about the auto commit

Especially when you deal with a RDBMS, it may happen that you have to temporarily turn off all the controls (*so temporarily leave the database in an inconsistent state*) to make some complex operations.

You have to turn off the auto commit option, in this way you can handle data through atomic transactions.

Indeed, if you turn off this option, you have to do the commit manually at the end of the transaction.

So, you can “commit manually” when you finished all the operations on the objects.

Code tips

PreparedStatement `stat` =

```
connection.prepareStatement(insertSQL.toString(),
    Statement.RETURN_GENERATED_KEYS);
```

`RETURN_GENERATED_KEYS` are useful when you insert a new record which the primary keys are auto increment, so you avoid making a query to get the just inserted row's id.

```
StringBuilder insertSQL = new StringBuilder();
insertSQL.append("insert into publisher(name, location) ");
insertSQL.append(" values (?,?)");
```

The query has the `?` because it is a parametrized query.

DTO - Data Transfer Object

The DTO is an object with attributes summarized because the author object can have a lot of attributes.

In general, DTO keeps separated model layer and view layer. Sometimes some information is derived from others, so DTO allows to return these derived and computed information.

The DTO generally contains derived information for speed-up operations.

Redis

Can be used like a key-value database, cache, message broker and streaming engine.

It offers atomic operations like incrementing values or calculate hash values.

The database is an in-memory database, it is a data structure storage, allows users to create data structures in memory, it retrieves speed etc, it can be used as a caching system, Redis provides different data types, provides keys that are strings, it supports atomic operations.

Working with a tool that stores data in memory have an issue, when the memory is flushed the data can be lost, but Redis provides a periodicity in which data are stored into the disk, you can set up a **time to live** for your keys. It also provides good scheming features.

For install Redis you go to its site.

Is available for Linux, but you use it in Windows with a Linux subsystem (*WSL2*).

Keys are unique in Redis, they are used to locate a specific value.

Empty string is also a Key.

Keys in Redis are the unique method to locate your files, so it is suggested to assign meaningful name for your keys, a good practice is to use a namespace to separate logical areas for your application, a good practice is to use for the key, the name of the entity, then use the column, then an identifier, then the information you are saving.

When you want to retrieve some information, you have to know the memory location in which it is memorized, you can use a hash algorithm, but the key must be short otherwise it takes a lot of time to compute the hashing.

Use a namespace to identify application keys:

Sales-app:invoice:10012:status = "PAID"

Very long keys are not a good choice, because they add latency.

Keys must be readable: "I need a key to store the quantity of followers of a user"

- Good $\square$ user:1000:followers
- Bad $\square$ u1000flw

Dots or dashes are often used for multi-word fields:

- comment:4321:reply.to
- shopping-cart:5431:updated-date

you have a maximum key value size, which is very large.

Redis Data Types

Redis support a certain set of data types:

hello world	String
011011010110111101101101	Bitmap
{23334}{6634-728}{916}	Bitfield
{a: "hello", b: "world"}	Hash
[A>B>C>C]	List
{A<B<C}	Set
{A:1, B:2, C:3}	Sorted set
{A: (50.1, 0,5)}	Geospatial
01101101 01101111 01101101	Hyperlog
{id1=time1.seq((a: "foo", a: "bar")) }	Stream

For our project we can just use strings, but if we act cool, we can use the other too.

Lists and Sets

In list can have duplicates while set can't.

```
jose@uss-defiant:~$ redis-cli
127.0.0.1:6379> set hello 'hello world from redis'
OK
127.0.0.1:6379> get hello
"hello world from redis"
127.0.0.1:6379> expire hello 10
(integer) 1
127.0.0.1:6379>
127.0.0.1:6379> get hello
(nil)
127.0.0.1:6379> █
```

Once you have installed the server, you can connect to the server from another terminal with **redis-cli**.

You can send command to Redis by using a command line interface.

- Set <key_name> <key_value> To create a new key.
- Get <key_name> To get the value from a key.

The Redis Server

```
saverio@Unum:~$ sudo service redis-server restart
```

```
saverio@Unum:~$ redis-cli
```

```
127.0.0.1:6379> ping // test connection.
```

```
// results
```

```
127.0.0.1:6379> exit
```

```
saverio@Unum:~$ sudo service redis-server stop
```

To define a new key-value pair:

```
set <key> <value>
```

To retrieve the value from a key:

```
get <key>
```

To define the expiration on a key:

```
expire<key> <seconds> NX
```

```
set <key> <value> EX <seconds>
```

To see the time to live of a key:

```
ttl<key>
```

To delete a key:

```
del<key>
```

Transactions in Redis

Execution of a group of command in a single step, so in an atomic way.

All the command in a transaction is serialized and executed sequentially in an atomic way (*o tutte o nessuna*).

Using the EXEC command, the actions in a transaction are put in a queue and executed one by one (serialized).

Redis do not support rollbacks!! But I can discard a previous sent action using the command DISCARD.

Keys involved in a transaction could be modified, so a transaction could influence future transaction.

Let's suppose that we have 2 clients: C1 and C2.

1. C2 send a set of command and execute them first.
2. C2 modified the key K in K'.
3. After C2 transaction, C1 try to execute its commands.
4. Since K has been modified, C1's transaction will be simply aborted.

Round-trip time

Redis is based on client-server architecture and uses TCP connections.

Every command is blocking: any time you issue a command in your client, it will be sent a connection request to the server.

The client program is stopped until a response to the server returns to the client.

The time waited by the client between the instant where the request has been sended and the instant where the server response returns is the round-trip time.

Pipelining

Without pipelining:

Client:INCR X

Server:1

Client:INCR X

Server:2

Client:INCR X

Server:3

Client:INCR X

Server:4

Pipelining is used to pack all commands together and then receive an answer in a batch.

With pipelining:

Client:INCR X

Client:INCR X

Client:INCR X

Client:INCR X

Server:1

Server:2

Server:3

Server:4

If you have some operations, for example if you have a book and the author and when you create a book, also the author must be inserted, so in Redis all the operations must be atomic, the commands are not executed until you type the `exec` command which will be executed in a sequence in an atomic way.

If you have two terminals that are two clients and on both of them you want to do an atomic operation, and with `watch` command you want to see the same key in these two clients, then you start a transaction in both of the clients, then we execute the command in both of the clients, so both of the commands are queued in both of the clients, then we execute the command `exec` in both of the clients, client 1 done the `exec` command before client 2, then in one of the two terminals we try to retrieve the value of the key the operation in this case will be aborted.

```
jose@uss-defiant:~$ redis-cli
127.0.0.1:6379> watch my-key
OK
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379> set my-key 20
QUEUED
127.0.0.1:6379> set my-key "modified by client 1"
QUEUED
127.0.0.1:6379> EXEC
(nil)
127.0.0.1:6379> get my-key
"modified by client 2"
127.0.0.1:6379> █

jose@uss-defiant:~$ redis-cli
127.0.0.1:6379> watch my-key
OK
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379> set my-key 12
QUEUED
127.0.0.1:6379> set my-key "modified by client 2"
QUEUED
127.0.0.1:6379> EXEC
1) OK
2) OK
127.0.0.1:6379> get my-key
"modified by client 2"
127.0.0.1:6379> █
```

When should you use Redis?

When the application must deal with many volatile data.

Redis is used by many big apps like GitHub, snapchat etc...

Let's see some scenario: Shop Cart, Online Auction.

Jedis

Jedis is a Java library for connecting to Redis and for use it.

All commands are implemented in the class `Jedis`.

```
import redis.client.jedis.jedis;

String redisHost = "localhost";

Integer redisPort = 6379; // the protocol port of Redis.

Jedis jed = new Jedis( redisHost , redisPort);

// Insert your stuff here.

Jed.close(); // Always close the Jedis object.
```

We must add this dependency to use redis in our Java application.

```
<dependency>
<groupid>redis.clients</groupid>
<artifactid>jedis</artifactid>
<version>4.3.0</version>
</dependency>
```

Connecting to the Redis server

```
import redis.clients.jedis.Jedis;

String redisHost= "localhost";

Integer redisPort= 6379;

//obtaining a single connection to Redis.

Jedis jedis= new Jedis(redisHost, redisPort);

// do your stuff here.

// remember to close all the resources you allocate.

jedis.close();
```

Connecting to Redis server (connection pool)

```
import redis.clients.jedis.jedis;

String redisHost= "localhost";

Integer redisPort= 6379;

//using a pool improves the performance of Redis operations

try (JedisPool pool = new JedisPool(redisHost, redisPort);

    Jedis jedis= pool.getResource())

// do your stuff here.
```

Once you get an instance of a Jedis object, you can execute many commands

```
String key = "my-app:some-key";

//SET command.

jedis.set(key, "Hello world!");

//EXPIRE command.

jedis.expire(key, 10);

//GET command.

String value = jedis.get(key);

//TTL command

long ttl= jedis.ttl(key);
```

We can send multiple commands within a transaction and abort it in case watched keys have been modified.

```
String key1 = "my-app:some-key";

String key2 = "my-app:key-2";
```

Jedis Commands

Watch

If this key is modified, the transaction is aborted.

```
jedis.watch(key1);
```

Multi

```
Transaction transaction= jedis.multi();
```

```
transaction.set(key1, "New value for key1");
```

```
transaction.set(key2, "New value for key2");
```

Exec

It will return a list with output values of each command sent in the transaction.

The execution of the transaction returns null when keys marked with WATCH have been modified

```
List<Object> result = transaction.exec();
```

Connection to the database

[IMG]

Subscribe / Unsubscribe

[IMG]

Publisher / Subscriber

[IMG]

Scaling in Redis

Eviction policies

A configured eviction policy is activated when the max memory limit is reached.

There are different policies:

- NoEviction
 - New values aren't saved in memory.
 - If there is replication, this applies to the primary database.
- AllKeys-LRU
 - Keeps most recently used keys, removes least recently used keys.
- AllKeys-LFU
 - Keeps frequently used keys, removes less frequently used keys.
- Volatile-LRU
 - Remove least recently used keys with expire field (TTL) set to true.
- Volatile-LFU
 - Removes least frequently used keys with expired field set to true.
- AllKeys-Random
 - Extract a random key to delete.
- Volatile-Random
 - Extract at random a key with expired field set true.
- Volatile-TTL
 - Removes keys with expire field set to true and the shortest remaining time to live (TTL) value.

How to see/set the eviction policy

```
127.0.0.1:6379> CONFIG GET maxmemory
```

1) "maxmemory"

2) "0" // No limit.

```
127.0.0.1:6379> CONFIG GET maxmemory-policy
```

1) "maxmemory-policy"

2) "noeviction" // No eviction policy used.

Persistence

Snapshots (RDB)

I periodically perform snapshots of my database.

I can lose some write operations that are occurred during the creation of the snapshot.

Append-Only File (AOF)

A simple log file, where every write operation is reported.

Using this log, I could rebuild the entire database.

Hybrid (RDB + AOF)

The most expensive (*for performance*) solution, but it's the best in terms of persistency.

I am sure that I won't lose anything anytime.

Because operations that I lose during the snapshot will be saved in the AOF.

Replication

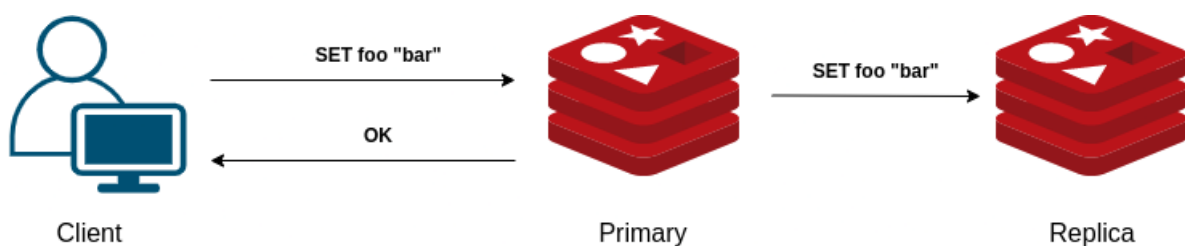
Used to assure availability.

By default, redis works in a master-slave system.

The primary database (master) takes care of keep updated the replicas by forwarding the same write operation to them.

The primary receives the client request, and after some time, will be sent to the replicas too.

When the primary server falls, one of the replicas takes its place.



The minimum number of replicas depends on the application.

Clustered database

It is the Redis solution for implementing the scaling.

Keys are distributed among cluster nodes.

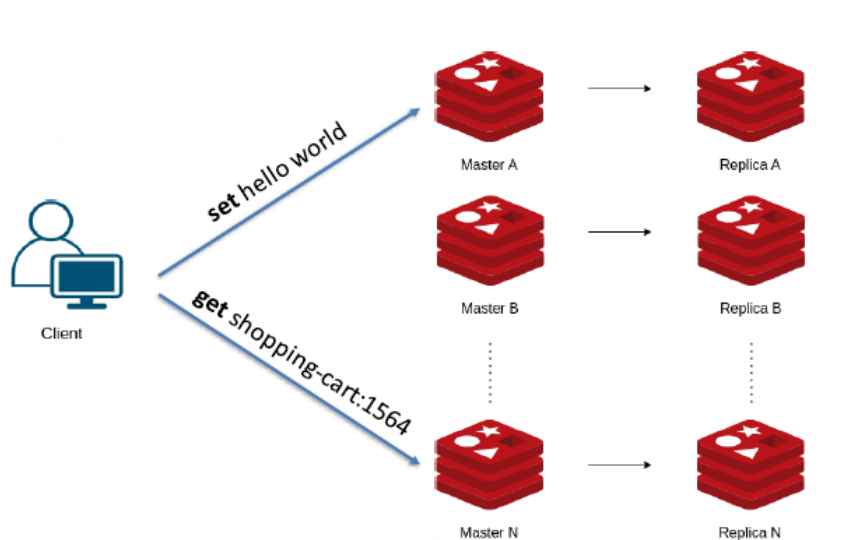
How determine where a certain key must go?

To determine in which node a key is stored must be computed a `HASH_SLOT`.

$\text{HASH_SLOT} = |\text{CRC16}(\text{Key})| 16384 \text{ // } \textit{Modulus}$.

There are 16384 hash slots.

Each node of the cluster has assigned a range of slots.



To configure this setup, we must use the `redis.conf` file, in this case we got two nodes, so we assign two ip addresses to them.

We specify that we don't want the protected mode.

Primary, IP: 173.18.15.2	Replica, IP: 172.18.15.3
bind 0.0.0.0 protected-mode no requirepass custompassword	bind 0.0.0.0 replicaof 173.18.15.2 6379 masterauth custompassword

Remember: Always set passwords!

Example of replication in redis

We want to create a database with three master nodes and three replica nodes.

```
jose@uss-defiant:~/repository/UNIP/LSMSD/redis-cluster$ pwd
/home/jose/repository/UNIP/LSMSD/redis-cluster
jose@uss-defiant:~/repository/UNIP/LSMSD/redis-cluster$ ls
7000 7001 7002 7003 7004 7005
jose@uss-defiant:~/repository/UNIP/LSMSD/redis-cluster$ █
```

Inside each folder (one folder for each node) we create the `redis.conf` file with the following content:

```
port 7000

cluster enabled yes

cluster config file nodes.conf

cluster node timeout 5000

append only yes
```

`cluster-node-timeout` is the maximum period in which a node can do not respond, if this time limit is reached, other nodes will consider it fallen.

Start each Redis instance by running the following command in each folder:

```
redis server ./redis.conf
```

Finally, to create the cluster, run the following command:

```
redis - - cli cluster replicas 1 - - cluster create 127.0.0.1: 7000 127.0.0.1: 7001 127.0.0.1:
7002 127.0.0.1: 7003 127.0.0.1: 7004 127.0.0.1: 7005
```

With the command in bold we specify that we want a replica for every master created.

Test your configuration by creating some keys:

```
jose@uss-defiant:~$ redis-cli -c -p 7000
127.0.0.1:7000> set foo bar
-> Redirected to slot [12182] located at 127.0.0.1:7002
OK
127.0.0.1:7002> set hello world
-> Redirected to slot [866] located at 127.0.0.1:7000
OK
127.0.0.1:7000> get foo
-> Redirected to slot [12182] located at 127.0.0.1:7002
"bar"
127.0.0.1:7002> get hello
-> Redirected to slot [866] located at 127.0.0.1:7000
"world"
```

To stop the cluster run this command:

```
create cluster stop
```

Jedis and Clusters

```
import redis.clients.jedis.*;
import java.util.HashSet;
import java.util.Set;

Set<HostAndPort> jedisClusterNodes = new HashSet<HostAndPort>();
jedisClusterNodes.add(new HostAndPort("127.0.0.1", 7001));
jedisClusterNodes.add(new HostAndPort("127.0.0.1", 7002));
jedisClusterNodes.add(new HostAndPort("127.0.0.1", 7003));

try (JedisCluster jedis = new JedisCluster(jedisClusterNodes)){
    /*
     do stuff here
    */
}
```

Notes on Redis Replication

- A master node can theoretically have infinite replicas.
- Redis application is non-blocking on the master side.
- Replication is also largely non-blocking on the replica-side.
- Replication can be used for:
 - *Scalability* □ Have multiple replicas for read-only queries.
 - For improving data safety and availability.

Keyspace Notification

Redis permits to monitor changes on keys and values in real time.

Clients can receive notification about the expiration of a key by subscribe to the related event channel.

1. When the client creates a new key with a certain TTL.
2. When the key expires, a related event occurs.
3. All the clients subscribed to this event will receive the notification.

This can be useful for some applications.

I can do this thing with every event related to the keys.

Keyspace notification in Jedis

Receiving key space notifications:

```
JedisPubSub jedisPubSub = new JedisPubSub() {  
    @Override  
    public void onPMessage(String pattern, String channel, String message) {  
        System.out.println("Pattern: " + pattern + ", channel: " + channel + ", message: " + message);  
    }  
};  
....  
try (Jedis jedis = pool.getResource()) {  
    jedis.psubscribe(jedisPubSub, "__keyevent@0__:expired");  
}
```

Do not forget to activate the event notification when a key expires. Run this command in Redis:

Publisher – Subscriber

A client can perform the `subscribe` command to a certain channel.

A publication on a channel (performed with the `publish` command) will be notified to every subscribed client.

To stop receive messages, the client can perform the `unsubscribe` command.

To define a subscriber run the following code:

```
JedisPubSub jedisPubSub = new JedisPubSub() {  
  
    @Override  
    public void onMessage(String channel, String message) {  
        System.out.println("Channel " + channel + " has sent a message : " + message );  
    }  
  
    @Override  
    public void onSubscribe(String channel, int subscribedChannels) {  
        System.out.println("Client is Subscribed to channel : "+ channel);  
        System.out.println("Client is Subscribed to "+ subscribedChannels + " no. of channels");  
    }  
  
    @Override  
    public void onUnsubscribe(String channel, int subscribedChannels) {  
        System.out.println("Client is Unsubscribed from channel : "+ channel);  
        System.out.println("Client is Subscribed to "+ subscribedChannels + " no. of channels");  
    }  
  
};  
....  
try (Jedis jedis = pool.getResource()) {  
    jedis.subscribe(jedisPubSub, "Channel1", "Channel2");  
}
```

To publish a message run the following code:

```
try (Jedis jedis = pool.getResource()) {  
  
    /* Publishing message to channel Channel1*/  
    jedis.publish("Channel1", "First message to channel Channel1");  
  
    /* Publishing message to channel Channel2*/  
    jedis.publish("Channel2", "First message to channel Channel2");  
  
    /* Publishing message to channel Channel1*/  
    jedis.publish("Channel1", "Second message to channel Channel1");  
  
    /* Publishing message to channel Channel2*/  
    jedis.publish("Channel2", "Second message to channel Channel2");  
  
}
```

Task 3 : Ecommerce

I will have an instance of MySQL (*with replication*) and an instance of Redis (*without replication*).

The application server (*where the java code is executed*) is a single point of failure.

The redis server is a SPoF too, because is a unique instance.

So, the new software architecture has:

- Multiple instances of the application server.
- A replica of the Redis server.

The MySQL is essential?

No, we can use other solutions.

Could a columnar database perform better?

According to the requirements, a columnar database can be good.

Software Layered Architecture

Is the architectural pattern of MVC.

- **Model**
 - It manages the data and the business logic.
- **View**
 - It's the frontend part, so, what the user will see.
- **Controller**
 - It determines what action a user wants to execute.

MVC is a way to organize the code, for make it more readable.

Which is the path followed by a user's request?

1. Presentation layer
 - a. ?
2. Controller layer
 - a. Intercept the request, understand the job to do and then dispatch the request to the proper handler.
3. Service Logic Layer
 - a. Often coincide with the business logic of the application.
4. Data Access Layer (Model Layer)
 - a. Layer where entities are implemented.
 - b. DAO objects are intermediary between the Service Logic layer and the Data access layer.

Obviously, we can also have more layers.

Like “*Intermediation Layer*”, where I put all the API of external components.

Package Organization

The project’s code is organized in packages.

Each class is contained in a well-defined package.

Model Package

It contains the implementation of entities involved in the app.

An entity in this package only has attributes, constructors, destructors, and Get & Set methods.

DAO Package

Part of the model layer, where I got DAO objects.

There is one DAO for each model entity.

Base DAO Package

Inside the DAO package there is the Base Package.

It contains the `baseDAO.java` file, so the java class which contains methods that every DAO class have in common.

DTO Package

Part of the model layer, where I got DTO objects.

Some entities could not have a DTO object.

Controller Package

Here is contained the code that handles the input of the user.

Service Layer Package

Here is contained the actual business logic of the app.

If we consider the Ecommerce example: here I can find operation like the “*checkout*”.

Redis Exercises

Black Friday

Black Friday, sales increases, CPU utilization in the redis master server was 100%.

How to improve the situation?

We reconfigure the redis architecture to ?? replicas.

Good practices in developing java application

Best practice is a good title, but there are not absolute best practices.

So “good” instead of “best” is a better title.

IA document

Implementation **A**nalysis document is the document that the client will sign with the software house.

The document contains what the software house must develop.

Package structure and naming conventions

How should we organize the hierarchy of directories.

Why should we do all of this?

- Readability
 - Well organized code improves the understanding of the code by someone which didn't write the code.
 - Learning curve of new joiner will be more long.
- Maintainability
 - Consume less time for maintain the code.
- Scalability
 - Code scalability, which means “How easy it is to add or change features?”.

Naming convention of packages

The only best practice about this is the naming convention of packages (and classes).

- Package names is expected to be in lower case (*for avoid conflicts with classes*).
- Companies use their reverse domain name as package prefix:
 - www.iongroup.com → com.iongroup.package_name
 - If the name contains some java keyword, then we append the underscore “_” after the keyword:
 - example.int → int_example.
- For internal libraries a similar rule is applied:
 - [reverse domain][division][lib name]
-

How can we organize the hierarchy

There are two or three ways, we will see only two of them.

By Layers (and/or by Type)

Each package contains all the objects that are part of a specific layer of the application.

It.unipi.dii.inginf.lsdb.library

.bean

.cache

.persistence

.factory

.model

.service

.local

.remote

...

By Feature

Each package contains all the object required to implement a specific feature.

It.unipi.dii.inginf.lsdb.library

.admin

.login

.registration

.user

.reserve

.search

...

Package by Features

Why should we care about hierarchy?

- High modularity, minimal coupling
 - A good test: what if an entire feature must be deleted, other features will be influenced by this?
- Easy to read and understand how features are implemented
 - If all the code of a feature is in only one package, then retrieve the code is easier to get.
 - Relationships among features are easier to identify.
 - For example: the relation between the user and the login functionalities.
- Package scope should be exploited
 - If a given object is used only within a package, why should it be visible outside it?

Defining a hierarchy does NOT mean a given package cannot use items defined in other packages.

We are simply reducing the scope of objects, to prevent abuses.

But if other packages require an object of another package, then we will increase the scope of that object.

Extract API if necessary.

Code duplication and the rule of three

There are programmers that prefer to duplicate the code rather than inheritance.

If we need to duplicate a not complex class, then do it.

But if, then don't do it.

Rule of three: you are allowed to copy and paste the code once, but that when the same code is replicated three times, it should be extracted into a new procedure.

Code duplication is useful when we want to avoid programming many subcases, so avoid writing methods with many parameters.

Only because I don't want to duplicate the code and write only one code for all.

Where place a new piece of code?

Choose the correct package is not an easy task.

- Pay attention to the size of a package.
- Pay attention to package circular dependencies.
 - Feature in a package A depends on a feature in package B which depends on a feature in package C which depends on another feature in package A.
 - This thing makes the code harder to debug.

Package by layers

Pros of package by layers

The more natural way to organize applications.

- Item doing the same job are in the same package
 - “All the caches are in the same package, so when I am looking for a cache, I go in the cache package”.
- Easy to change technology for a single layer.
 - Old and new can even coexist (*in different packages*) while the refactoring is in process.
- Easy to build reusable framework/libraries.
 - Identify the commonalities among different items of the same layer/type and extract an abstract framework for that.

Cons of package by layers

- Hard to understand the implementation of a given feature
 - This approach tends programmers to reuse generic and complex code.
 - Changes can easily break other use cases and the impact is hard to identify.
- Low modularity and high package coupling.
 - Package coupling is here by design, if I structure by layer for sure I will have to include several packages.
- Editing a feature will require changes among several packages and could break other features.
- Difficult to reduce the scope of objects that are supposed to be used un different packages.

Mix-up Techniques

For example, we can organize the first level of hierarchy for features and inside the package of a specific feature I can organize the package for layers.

Don't stick to a solution if you understand that is the wrong choice.

Git and the collaborative work

Pros of Git

- Distributed
 - Each developer has a local copy of the full history.
- Version control
 - History of all changes.
 - Branching
 - Traceability of changes.

How does Git work?

Git works in layers.

Git has many areas that you can call “saving areas”.

Working directory

The actual copy you have access to edit.

So, the file that you see in the file system when you open the folder.

Staging area

Buffer of your changes ready to commit.

The commit is “store to remote from the local”.

Git add Myclass.java

Git add .

Staging is a concept common in computer science.

Local repository

A full copy of what is available in the remote.

Include all the commits and branches created until the last pull.

So, with the following command I send all the changes in the staging area to the local repository.

```
Git commit -m "A commit message to show."
```

Remote repository

Centralized copy of your project, hosted in a server.

When we got something committed to the local, after a review all the commits are pushed all together (*in the same order*) in the remote.

Then colleagues can pull the remote copy to their local and see your work.

Git push

Stash

Parallel buffer you can use to temporary save your changes to the working copy.

Useful when you need to perform operations like merge/rebase from another copy.

Works like a stack, so with pop and push commands.

Git stash // *push*

Git stash pop

With a push I can put aside your changes to the working copy.

Then with pop I can re-apply my changes.

Branch

A parallel code thread that is created from another code thread.

The branch has his own commits.

A branch surely can re-merge with the master copy.

A branch is created when we want to fix a bug or add a feature.

How to structure applications with Maven

How to structure the application and how to split the API and the SPI.

What is Maven?

Maven is a building tool for managing java applications:

- Maven helps you making build process easy.
 -
- Providing uniform build system.
 - It is a big benefit, once you become familiar with the `pom` file syntax and with the most commons plugins that you can use during the maven build phase, you will be able to understand how maven organize a Java project.
- Providing quality project information.
 - I can run programs that evaluate the “*quality*” of my code (*and other statistics*).
 - I can create a set of rules for my code, that the program will check (a company can create its set of rules that every program written by employees must follow).
 - Some examples of rules:
 - Max length of a function.
 - Type of parameters.
 - ...
- Encouraging better development practices.
 - This point especially refers to the testing part.
 - Maven gives us the opportunity to create a separate directory where to put the code to test.
 - So, we can separate the code used for testing (*under test code*) from the code that is being written by employees (*production code*).

POM file - How Maven works?

Maven is based on the pom (*Project Object Model*) file.

There is a `pom.xml` file containing information about the project and configuration details used by Maven to build the project.

Some POM tags

- **`modelVersion`**: the version of the maven model
- **`groupId`**: unique Id of the organization/group that created the project
- **`artifactId`**: unique base name of the artifact generated by the project
- **`version`**: version of the generated artifact
- **`name`**: display name of this project
- **`url`**: where this project can be found (useful for documentation)
- **`properties`**: values whose scope is the current pom
- **`dependencies`**: external libraries to be used in the project
- **`build`**: contains build technical stuff, including the managing of plugins

Properties tags

You can think about `properties` like variables that are visible inside the `pom.xml` file.

You can use properties in other section of the file like variables in the code.

Properties are useful to avoid hard coding in the pom file!

Dependencies tags

In these tags you can specify external libraries that we want to import in the project.

Build tags

Contains build technical stuff, so:

- Plugins used in the project (*and their configuration*).
- Source template.
- Profile of building (useful to custom the building process, so to avoid building the entire project every time).

How to create multi-module projects – the parent POM

The parent pom is a normal POM file that we can refer to use it in other applications.

In brief, our POM (also called “child POM”) will inherit some tags from the parent POM.

The parent POM is also called “Super POM”.

You can see like a class (parent POM) and its sub-classes (all the child POM's).



[tion-to-the-pom.html](#)

If we got a parent POM, we could avoid writing some tags in the child POM (like the *project tag*).

However, in the child POM I can still overwrite tags that are contained in the parent POM.

How to use a parent POM?

A parent POM is useful for aggregate modules of the project.

<pre><project xmlns="..." xmlns:xsi="..." xsi:schemaLocation="..."> <modelVersion>4.0.0</modelVersion> <groupId>it.unipi.dii</groupId> <artifactId>library</artifactId> <version>1.0-SNAPSHOT</version> <packaging>pom</packaging> <modules> <module>library-main</module> <module>library-common</module> <module>library-books</module> <module>library-music</module> </modules> <dependencyManagement> <dependencies> <dependency> <groupId>org.junit.jupiter</groupId> <artifactId>junit-jupiter-api</artifactId> <version>5.6.2</version> <scope>test</scope> </dependency> </dependencies> </dependencyManagement> ... </project></pre>	<pre><project xmlns="..." xmlns:xsi="..." xsi:schemaLocation="..."> <modelVersion>4.0.0</modelVersion> <parent> <groupId>it.unipi.dii</groupId> <artifactId>library</artifactId> <version>1.0-SNAPSHOT</version> </parent> <groupId>it.unipi.dii</groupId> <artifactId>library-main</artifactId> <version>1.0-SNAPSHOT</version> <dependencies> <dependency> <groupId>it.unipi.dii</groupId> <artifactId>library-common</artifactId> <version>1.0-SNAPSHOT</version> </dependency> <dependency> <groupId>it.unipi.dii</groupId> <artifactId>library-music</artifactId> <version>1.0-SNAPSHOT</version> </dependency> <dependency> <groupId>junit</groupId> <artifactId>junit-jupiter-api</artifactId> </dependency> </dependencies> ... </project></pre>
---	---

it section

Figure 9 At the left we got the parent POM; at the right we got the child POM.

Why should we do this?

The code base, during the production, can significantly grow.

The longer the application lasts, the huger is the amount of code, so the more complex will be the maintenance.

How can we link the child POM and the parent POM?

In the example on the right (*the child POM*) we can see the tags `<parent></parent>` , with these tags I can specify which is the parent POM of this child POM.

You can also specify the path of the parent POM.

```
<relativePath>../baseapp/pom.xml</relativePath>
```

In the example on the left (*the parent POM*) we can see the tags `<packaging>pom</packaging>`.

With these tags I can specify that the project have multiple modules.

How can I include modules?

In the example on the left (*the parent Pom*) we can see a sequence of `<module></module>` tags.

With these tags I can specify the modules that I want to be included.

Mind that every submodule represents a specific concern of the application.

Then in every child pom I can specify (*with <dependencies>*) the modules that I want to use in the project (*related to the child POM*), like a library.

Can I avoid specifying common dependencies in the child POM's?

Obviously yes!

If in the parent POM I specify some dependencies (*inside the tag <dependencyManagement></dependencyManagement>*), every child POM linked to the parent POM have only to specify the `<groupId>` and the `<artifactId>` in the tag `<dependency>`.

Other information about the dependencies will be taken from the parent pom.

Version control of dependencies

Let's suppose that you ran a java application related to a child POM.

And let's suppose that you have specified some modules in the parent POM.

Obviously, the child POM will inherit the tags.

The java compiler will try to find the module.

If in the child POM you don't specify a version: the first model that it finds, it is the module that will be loaded.

For this reason, you must take care about older versions of the libraries.

But if you specify the `<version>1.0-SNAPSHOT</version>` in the child POM, then it will be loaded the `<version></version>` specified in the parent POM in the dependency's tags.

How to build a framework

A framework is a shared set of libraries (like code for memory cache or math functions).

In the parent POM (the image on the left):

ing-inf-parent is the framework name, and stuff in the <module> tags are the libraries included in the framework.

Each module is a library.

Then in the child POM (the image on the right):

We specify with <parent> the POM file, with the version of the pom file.

In the child POM we specify the libraries (offered by the framework) that I want to use.

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>it.unipi.dii.inginf</groupId>
    <artifactId>ing-inf-parent</artifactId>
    <version>2.0.4</version>
  </parent>

  <groupId>it.unipi.dii</groupId>
  <artifactId>library</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>pom</packaging>

  <modules>
    <module>library-main</module>
    <module>library-common</module>
    <module>library-book</module>
    <module>library-music</module>
  </modules>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.junit</groupId>
        <artifactId>junit</artifactId>
        <version>5.6.2</version>
        <scope>test</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>
</project>
```

Structured Databases

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>it.unipi.dii</groupId>
    <artifactId>library</artifactId>
    <version>1.0-SNAPSHOT</version>
  </parent>

  <groupId>it.unipi.dii</groupId>
  <artifactId>library-main</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <dependency>
      <groupId>it.unipi.dii</groupId>
      <artifactId>library-common</artifactId>
      <version>1.0-SNAPSHOT</version>
    </dependency>
    <dependency>
      <groupId>it.unipi.dii</groupId>
      <artifactId>cache</artifactId>
    </dependency>
    <dependency>
      <groupId>org.junit</groupId>
      <artifactId>junit-jupiter-api</artifactId>
    </dependency>
  </dependencies>
</project>
```



In this way, we are sure that we always use the latest version of the libraries because in the child POM we have specified that we use the version specified in the parent POM.

API & SPI

API (Application Programming Interface)

Why should we care about this?

The API of a library/module is the set of publicly accessible set of items provided by an external system.

API, give you access to functionalities required by an application.

Moreover, API separate public items (*interface, shared beans*) from implementation details.

In poor words, you are forcing the programmer to use the API and not to use the entire implementation (*which remain hid*).

In the code, you specify the definition of the objects that we want to use.

The actual implementation is in another directory.

In API addition is not a problem, removal is.

SPI (Service Provider Interface)

Publicly accessible set of items that can be implemented or extended to achieve goals.

The application is responsible for the implementation of a specific interface invoked by the library.

This is useful, because some part of the library must be application-specific, but we still want to use a library (*which is a set of general-use code*).

The application-specific part is the SPI.

In SPI removal is not a problem, addition is.

Code Obfuscation

It is always possible to decompile the java bytecode and get the source code.

To avoid this, companies often adopt java code obfuscation.

Code obfuscation consists of:

- Files, classes, methods, and fields are renamed with random identifiers.
- Comments removed.

We can use some tool for this, like ProGuard.

The problem about this technique is that we can have problems with libraries and multi-module maven applications.

Because code obfuscation also changes the name of the function of the libraries.

How can we solve this problem?

We can simply specify which modules don't need to be obfuscated.

I can do this thing with the keyword @KeepName.

```
// File: PriceGridServiceImpl.java
package it.unipi.dii.inginf.lsd.b.library.grid;
import ...
class PriceGridServiceImpl implements GridService {
    PriceGridServiceImpl() {}
    public List<String> getColumnNames() {
        return Lists.newArrayList("ItemId", "Price");
    }
}
```



```
// File: b.java
package it.unipi.dii.inginf.lsd.b.e.a;
import ...
class b implements c {
    public b() {}
    List<String> d() {
        return Lists.newArrayList("ItemId", "Price");
    }
}
```

url: itarabscot

```
// File: PriceGridServiceImpl.java
package it.unipi.dii.inginf.lsd.b.library.grid;
import ...
class PriceGridServiceImpl implements GridService {
    PriceGridServiceImpl() {}
    public List<String> getColumnNames() {
        return Lists.newArrayList("ItemId", "Price");
    }
}
```



```
// File: b.java
package it.unipi.dii.inginf.lsd.b.e.a;
import ...
class b implements GridService {
    public b() {}
    List<String> getColumnNames() {
        return Lists.newArrayList("ItemId", "Price");
    }
}
```

```
package it.unipi.dii.inginf.lsd.b.e.a;
import ...;

@KeepName
@KeepPublicClassMemberNames
public interface GridService {
    List<String> getColumnNames();
}
```

Javadoc – Document your stuff!

It is a good practice to document public API using Javadoc.

When you are using IntelliJ and you go over a function with the pointer of the mouse, you can see a little description that explains parameters and other stuff; that thing is very useful, and it is made with Javadoc.

A documentation of Javadoc is made with HTML tags.

```

/**
 * Manager for Multimedia objects.
 * @since 2.4.1
 * @see {@link Multimedia}
 */
public class MultimediaManager {
    //... private part

    /**
     * Processes a new Multimedia and its price.
     * <p>
     * Depending on the price value:
     * <ul>
     * <li>p>0: we do this and not that</li>
     * <li>p=0: we do that and not this</li>
     * <li>p<0: we do both this and that</li>
     * </ul>
     *
     * @param m the Multimedia to be processed, not null
     * @param p the Multimedia price
     * @throws {@link MultimediaException} if m is null or invalid
     */
    public void process(Multimedia m, double p) throws MultimediaException {
        //...
    }

    /**
     * Reads a Multimedia provided its unique identifier
     *
     * @param id the unique identifier of the Multimedia
     * @return the Multimedia corresponding to the id, null if not found
     */
    public Multimedia getMultimedia(String id) {
        //...
    }
}

```

Javadoc has some interesting tags like:

- @param
- @return
- @throws
- @link
- @code
- @see
- @since
- ...

Testing Java applications

Smoke testing

“Smoke testing” □ Manually test the application after a few changes.

Smoke testing exists and we can use it (*the classic printf test*), but it is not enough!

Unit test

What is a Unit test?

A unit test is a procedure to verify that a particular detail of a module is working properly.

A unit test does not to test the whole application!

It is written by developers (often the same developers which wrote the code to test).

Unit tests must be documented (like normal code) and written along with the code (tests must be done during the code production and not at the end).

Moreover, when you add a new functionality, it's a good practice to test the old functionalities too (*for be sure that aren't conflicts*).

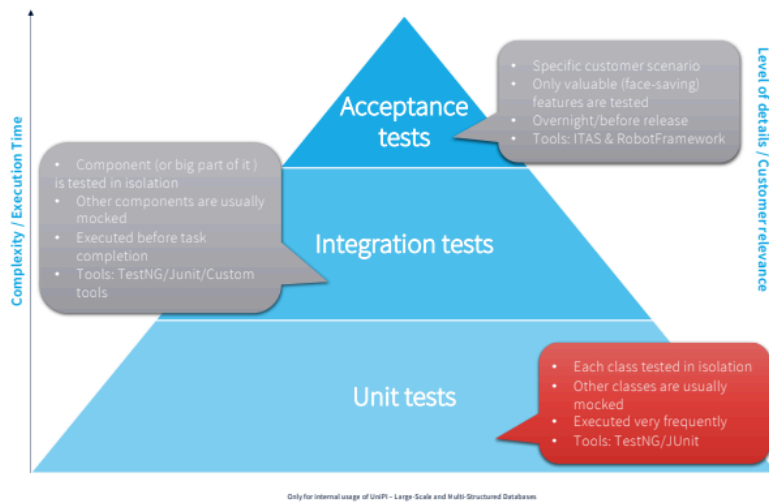
A test is not a unit test if:

- It communicates across the network.
 - We want to test our code (*behaviors*), not the third-party role!
- Touches the file system.
- Somehow talks to the database (*read or write*).
- Runs threads.
- Forces you to do special things to the environment (*such as editing config files*) to run it.

A test is a unit test if:

- It is quick, a unit test does not take ages to be performed.
- It is part of the build in Maven.

Testing Phases



Acceptance test

The most time-consuming ones.

For this reason, it is usually done overnight or before the official release.

Tests a specific customer scenario, so it tests many modules, classes, third-party components, and the interactions between them.

But only valuable features are tested.

Tools: ITAS and RobotFramework.

Integration test

Component (*or big part of it*) is tested in isolation.

Other components are usually mocked.

It is usually executed before a task completion.

Tools: TestNG, JUnit and other custom tools.

Unit test

It is the cheapest one.

Each class (*module*) is tested (*in an automatic way*) in isolation, and other classes are usually mocked.

A unit test is executed very frequently.

Usually, a unit test is very simple, almost “*stupid*” (*because it must be very fast*).

A unit test is useful for immediately discover little or big bugs.

Tools: TestNG and JUnit.

Best practices for unit tests

One class, (at least one fixture)

Add a fixture for each class that you add to the project.

For different scenarios, use different fixture.

One test, one assert.

For the “*single responsibility principle*” every test should contain only one assert.

So, you should write many tests against the same class method.

Each test must be short

Max 10 lines of code for each unit test.

Test name must clearly specify the goal of the test.

No generic names, better a long and verbose name.

Test1 ☐ NOT OK!

GIVEN_author_names_available_WHEN_search_by_prefix ☒ OK!

Test must be easy to maintain

DRY Principle ☐ “Don’t Repeat Yourself”.

KISS Principle ☐ “Keep It Simple and Stupid”.

Test must be simple

Avoid complex logic (if, switch, for ...).

Avoid dependencies between tests

A test should be able to stand on its own.

Its result must not depend on the result of another test or on the order which tests are performed.

Tests must be fast and automatic

Avoid special configurations.

Test boundary cases for each types of data

Numbers (0, positive or negative), Strings (length, empty...), Dates, Collections (empty, first, last...), exemptions thrown...

Write a test for a bug

If a test make you discover a bug, then you should write a specific test for that bug.

Cover the code properly

Don't leave the under-production code uncovered.

You break it, you fix it

Who breaks a test is also responsible to fix it, as soon as possible.

A test is broken if after a change in the code, a test that before was successful performed now it fails.

This thing is called "*regression*".

Tests must be easily readable

The most important one.

A test is a documentation about a specific feature.

Common approaches to testing

Noobs approach

Code for days, test for days.

This approach is expensive and hard.

Better approach

Code a little, test a little.

This approach permits to:

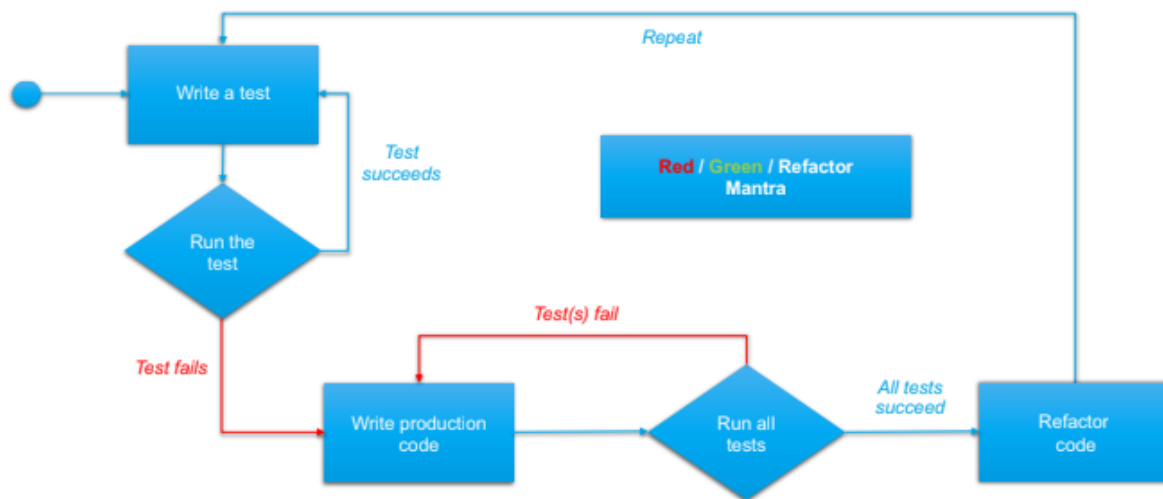
- Spot immediately design problems.
- Recognize early unused or useless code.
- Intercept errors as soon as possible.

Optimal approach

You use the TDD (Test Driven Development).

Write a test first, and then write the minimum amount of production code to let the test passes; then repeat.

Write the test, write the code that pass that test.



In this way, I will always have total coverage of all the production code!

Remember: No new code shall be written without associated test.

JUnit

It is an open-source framework for perform testing in java applications.

Provides annotation to identify test methods:

- `@Test` (this function is a test) .
- `@BeforeEach` (executed before every test) .
- `@BeforeAll`.
- `@AfterEach` (executed after every test) .
- `@AfterAll`.

JUnit provides assertion for checking expected results.

`Assertion.assertEquals( a , b );`

The assertion (and so the test) fails if a and b are different.

A test with JUnit can be run automatically and provides immediate feedback after checking the expectations.

How to import JUnit:



`@VisibleForTesting` □ Annotation for increase the scope of the function.

Mocking

A mock is a substitute implementation of a class to emulate the code.

A mock must be used when a class is:

- Expensive or impractical to instantiate.
- Belongs to a third-party library.

A mock object must be:

- Simple.
- Shouldn't contain any kind of logic.
- Allow to setup private state.
- Be stupid!

```
public interface IEngine {
    void start();
}

public class EngineImpl implements IEngine {
    public void start() {
        Engine.getInstance().start();
    }
}

public class MockEngine implements IEngine {
    boolean started = false;

    public void start() {
        started = true;
    }

    public boolean wasStarted() {
        return started;
    }
}
```

There are some frameworks for mocking (*or I can do this manually*).

Mocking Frameworks (Mockito)

Sometimes manual mocking is not possible.

Mockito is a framework that allows us to automatically write mocks that emulates interfaces or classes.

```
<properties>
  <junit.platform.version>1.6.2</junit.platform.version>
  <junit.version>5.6.2</junit.version>
  <mockito.version>3.0.0</mockito.version>
</properties>

<dependencyManagement>
  <dependencies>
    ... other dependencies not scoped for tests, e.g. guava
    ... other dependencies scoped for tests, e.g. junit
  </dependencies>
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <version>${mockito.version}</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-junit-jupiter</artifactId>
    <version>${mockito.version}</version>
    <scope>test</scope>
  </dependency>
</dependencies>
</dependencyManagement>
```

Parent POM

```
<dependencies>
  <dependency>
    <groupId>com.google.guava</groupId>
    <artifactId>guava</artifactId>
  </dependency>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
  </dependency>
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
  </dependency>
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-junit-jupiter</artifactId>
  </dependency>
</dependencies>
```

Module POM

But a mocking framework has some cons:

- Tests become fragile.

- It requires more knowledge of the code under test.
- Lower readability.
- Does not encourage the sharing of mocks in the team (*everyone tends to have their own mocks*).

Java & MongoDB

Install the community edition of MongoDB.

MongoClient

Example

This example demonstrates specifying a `ConnectionString`:

```

MongoClient mongoClient = MongoClient.create(
    MongoClientSettings.builder()
        .applyConnectionString(new ConnectionString("<your connection string>"))
        .build());

```

MongoClient provides an interface for the MongoDB instance.

The MongoClient object represent a pool of collections to the database.

We will only need one instance of class MongoClient, even with multiple threads.

The MongoClient object is thread-safe, multiple access by different threads to the single instance is managed by the class itself.

MongoClientSettings

To control the behaviour of the MongoClient I can use an object called MongoClientSettings.

That object is returned by the create method.

close()

Method used at the end of the application.

It cleans up resources.

Access to the database

`getDatabase()`

Method used to access the database.

`MongoDatabase MDB = MongoClient.getDatabase( "testDB" );`

If the database does not exist, it will be created (*confirmed by the MongoDB documentation*).

Access and handling a collection

getCollection()

To access a collection, you must use:

```
MongoCollection<Document> MC = MDB.getCollection( "users" );
```

If the collection does not exist, it will be created.

listCollectionName()

Get the list of existing collections:

```
for(String name : MDB.listCollectionName() ) {  
    System.out.println( name );  
}
```

drop()

It drops the collection.

```
MC.drop();
```

Create and insert a document

Document class

To create a document, you can use the class Document:

```
Document doc = new Document( "name" , "MongoDB" )  
.append( "count" , 1 ).append( "version" , 2 )  
.append( "Name" , "Sergio" ).append( "Surname" , "Arrighi" )  
.append( "State" , "Italy" ).append( "City" , "Pisa" )  
...  
.append( "Friends" , Arrays.list( "Paolo", "Luca" );
```

insertOne()

Then, just insert it into the collection:

```
MC.insertOne(doc);
```

insertMany()

Then for insert it into a collection you will use:

```
List<Document> docs = new ArrayList<Document>();  
/* populate docs */  
MC.insertMany(docs);
```


Query a collection

find()

You can retrieve a single document in a collection (*on a MongoClient object*).

You can pass a query filter to the find() method to query for and return documents that match the filter in the collection.

If you do not include a filter, MongoDB returns all the documents in the collection (*it means no filters installed*).

```
MC.find( and( gt( "i" , 50 ) , lte( "i" , 100 ) ) );
```

"Document with $50 < i \leq 100$ "

The ' , ' means 'and'.

first()

The first() method returns only the first matching document.

Find and print documents

```
try (MongoCursor<Document> cursor = myColl.find().iterator())
{
    while (cursor.hasNext())
    {
        System.out.println(cursor.next().toJson());
    }
}
```

Consumer Class

Used to iterate through query results.

```
import java.util.function.Consumer;

Consumer<Document> printDocuments = doc ->
    {System.out.println(doc.toJson());};

myColl.find().forEach(printDocuments);
```

Collect results in a list

```
List<Document> results =
    myColl.find().into(new ArrayList<>());
```


Update a document

UpdateResult class

```
UpdateResult updaters = Collection.updateFunction(  
<filter condition>,  
<update>  
);
```

The update function returns an object `UpdateResult`, which gives us some information about the operation, included the number of documents modified.

`updateOne()`

```
MC.updateOne(eq( "name" , "Alessio" ), set( "age" , 28 ));
```

`updateMany()`

```
MC.updateMany(lt( "age" , 28 ), set( "name" , "Alessio"));
```

Delete a document

DeleteResult class

```
DeleteResult deleters = Collection.deleteFunction(  
<filter condition>  
);
```

The delete function returns an object `DeleteResult`, which gives us some information about the operation, included the number of documents deleted.

`deleteOne()`

```
MC.deleteOne(eq( "name" , "Alessio"));
```

`findOneAndDelete()`

```
MC.findOneAndDelete(eq("name" , "Alessio"));
```

This command deletes the document, but also returns it.

`deleteMany()`

```
MC.deleteMany(gt( "age" , 90));
```

Pipeline

match()

I can pipe together several matches, to combine them and use them in a unique query.

```
Bson match1 = match(eq("categories" , "Bakery"));
```

```
Bson match2 = match(lt("year" , 1980));
```

...

```
Bson matchN = match(eq( "City" , "Rome" ));
```

group()

```
Bson g1 = group("$city" , sum( "totPop" , "$pop" ));
```

project()

How to specify the subset of field to return?

```
Bson p1 = project(fields(include( "city" )));
```

```
Bson p2 = project(fields(exclude( "_id" )));
```

...

```
Bson p1_2 = project(fields(
exclude( "_id" ),include( "city" , "state" )));
```

sort()

```
Bson s1 = sort(descending( "pop" ));
```

```
Bson s2 = sort(ascending( "avgAge" ));
```

limit()

```
Bson l1 = limit( 2 );
```

skip()

```
Bson sp1 = skip( 2 );
```

unwind()

It deconstructs the array field and returns several documents having:

- As fields the same fields of the original document.
- Instead of the array field, the document will have a new one which contain one of the values inside the array.

In other words: Let's suppose that the array has 3 elements.

The `unwind` will return 3 different documents, each with one of the 3 elements of the array.

```
Bson uw1 = unwind( "grades" );
```

How to use the pipeline?

```
coll.aggregate(Arrays.asList(matchN,g1,p1_2,s1,l1,uw1))  
.forEach(printDocuments());
```

MongoDB Replication

The primary node

We can have only one primary node.

The primary node is the node which receive the write and read operations by the clients.

When a write operation occurs, also the replicas will be updated following a certain consistency policy.

The secondary nodes and replicas

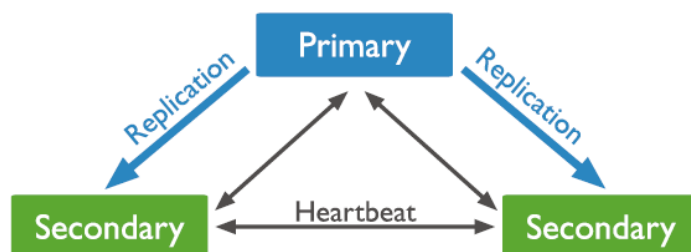
We can have as many secondary nodes as we need.

Some of the secondary nodes can be act like replicas of the primary node.

The Heartbeat approach

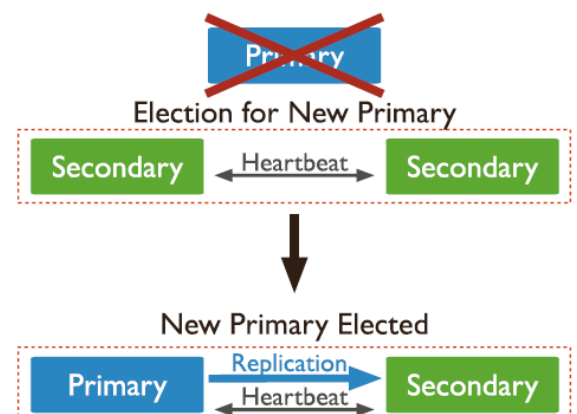
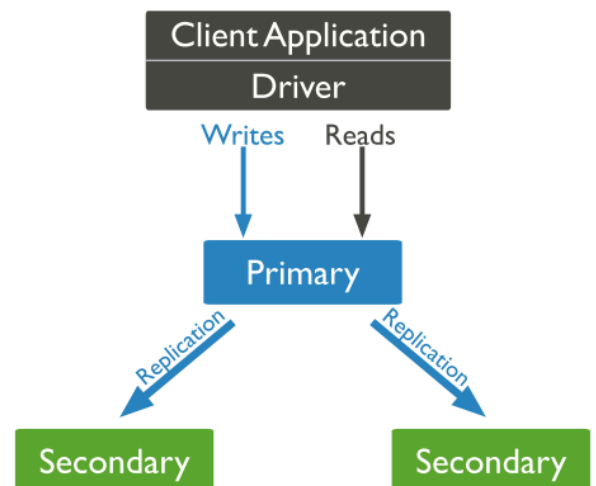
It is a method to check if nodes are alive.

The heartbeat is the time that nodes wait before realizing that a certain node is dead.



If the primary node is negative to the heartbeat, it means that it's dead, so other nodes will elect a new primary node.

The data inside the primary node are fully recovered thanks to replicas.



Write concern specification

Write concern can include the following fields:

```
{ w: <value>, j: <boolean>, wtimeout: <number> }
```



- the `w` option to request acknowledgment that the write operation has propagated to a specified number of `mongod` instances or to `mongod` instances with specified tags.
- the `j` option to request acknowledgment that the write operation has been written to the on-disk journal, and
- the `wtimeout` option to specify a time limit to prevent write operations from blocking indefinitely.

J Journal log

The journal is the log of all writes operations.

If a write operation includes `j : true`, then it will be registered in the journal.

Acknowledgment behaviour

The `w` option and the `j` option determine when `mongod` instances acknowledge write operations.

Standalone

A standalone `mongod` acknowledges a write operation either after applying the write in memory or after writing to the on-disk journal. The following table lists the acknowledgment behavior for a standalone and the relevant write concerns:

	<code>j</code> is unspecified	<code>j:true</code>	<code>j:false</code>
<code>w: 1</code>	In memory	On-disk journal	In memory
<code>w: "majority"</code>	On-disk journal <i>if running with journaling</i>	On-disk journal	In memory

Replica sets

The value specified to `w` determines the number of replica set members that must acknowledge the write before returning success. For each eligible replica set member, the `j` option determines whether the member acknowledges writes after applying the write operation in memory or after writing to the on-disk journal.

`w: "majority"`

Any data-bearing voting member of the replica set can contribute to write acknowledgment of `"majority"` write operations.

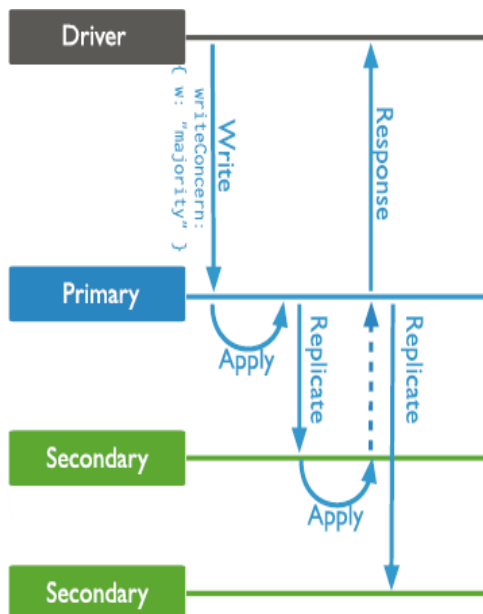
The following table lists when the member can acknowledge the write based on the `j` value:

<code>j</code> is unspecified	Acknowledgment depends on the value of <code>writeConcernMajorityJournalDefault</code> : - If <code>true</code>, acknowledgment requires writing operation to on-disk journal (<code>j: true</code>). <code>writeConcernMajorityJournalDefault</code> defaults to <code>true</code> - If <code>false</code>, acknowledgment requires writing operation in memory (<code>j: false</code>).
<code>j: true</code>	Acknowledgment requires writing operation to on-disk journal.
<code>j: false</code>	Acknowledgment requires writing operation in memory.

Write Concern Type	Behavior
ACKNOWLEDGED	Use the default Write Concern from the server
JOURNALED	Wait for the server to group commit to the journal file on disk.
MAJORITY	Wait on a majority of servers for the write operation.
UNACKNOWLEDGED	Return as soon as the message is written to the socket.
W1	Wait for acknowledgement from a single member.
W2	Wait for acknowledgement from two members
W3	Write operations that use this write concern will wait for acknowledgement from three members

Replication update mechanism

At the right we can see how the replica are updated.



Hierarchy of the write concerns

By default, write concern is organized in a hierarchy, it is global, we can overwrite by specifying a more stringent write concern, both for a given database or for a single collection.

```
// Write concern at client level
MongoClient mongoClient = MongoClient.create(
    "mongodb://localhost:27018,localhost:27019,localhost:27020/" +
    "?w=2&wtimeout=5000");

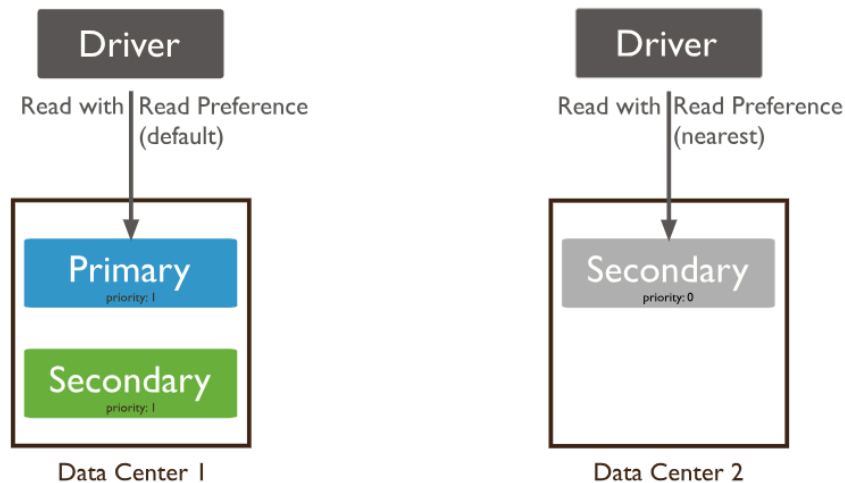
// Write concern at DB level
MongoDatabase db = mongoClient.getDatabase("LSMDB")
    .withWriteConcern(WriteConcern.W2);

// Write concern at collection level
MongoCollection<Document> myColl = db.getCollection("students")
    .withWriteConcern(WriteConcern.W2);
```

By default, the read preference is configured as primary value, together with write operations are forwarded to the primary node of the replica set.

Read preference

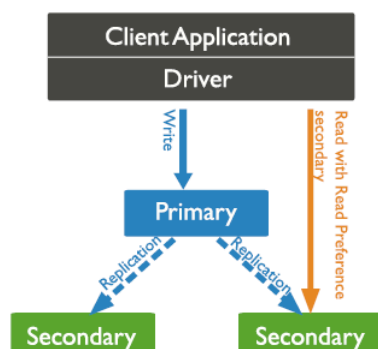
Read preference describes how MongoDB clients route read operations to the members of a [replica set](#).



Behavior of the read preference

All read preference modes except [primary](#) may return stale data because [secondaries](#) replicate operations from the primary in an asynchronous process. [1] Ensure that your application can tolerate stale data if you choose to use a non-[primary](#) mode.

A **read preference** provides client applications with control over which nodes of a replica set are used for reads



Read Preferences	Behavior
PRIMARY	The default read mode. Read from primary only
PRIMARY PREFERRED	Read from primary if available, otherwise a secondary
SECONDARY	Read from a secondary node if available, otherwise error.
SECONDARY PREFERRED	Read from a secondary if available, otherwise read from the primary.
NEAREST	Read from any member node from the set of nodes which respond the fastest. (Responsiveness measured in pings)

Hierarchy of the read preference

```
// Read Preferences at client level
MongoClient mongoClient = MongoClient.create(
    "mongodb://localhost:27018,localhost:27019,localhost:27020/" +
    "?readPreference=secondary");

// Read Preferences at DB level
MongoDatabase db = mongoClient.getDatabase(s: "LSMDB")
    .withReadPreference(ReadPreference.secondary());

// Read Preferences at collection level
MongoCollection<Document> myColl = db.getCollection(s: "students")
    .withReadPreference(ReadPreference.secondary());

// Tagged read preferences
Tag west_tag = new Tag( name: "dc", value: "west");
ReadPreference tagged_pref =
    ReadPreference.secondaryPreferred(new TagSet(west_tag));
MongoCollection<Document> myColl = db.getCollection(s: "students")
    .withReadPreference(tagged_pref);

myColl.find().forEach(printDocuments());
```

Replication on MongoDB - shell commands

Let's suppose that we want to create 3 replicas.

Create directories

The first thing to do is to create 3 directories, one for each replica.

```
# Ubuntu
mkdir ~/data/r{1,2,3}

# Windows (PowerShell)
mkdir C:\MongoDB\data\r1
mkdir C:\MongoDB\data\r2
mkdir C:\MongoDB\data\r3
```

Start multiple MongoDB Servers

Then you must start the 3 server processes.

```
# Windows (For Ubuntu it's the same)
mongod --replSet rs0 --dbpath c:\MongoDB\data\r1
--port 27018 --bind_ip localhost --oplogSize 200

mongod --replSet rs0 --dbpath c:\MongoDB\data\r1
--port 27019 --bind_ip localhost --oplogSize 200

mongod --replSet rs0 --dbpath c:\MongoDB\data\r1
--port 27020 --bind_ip localhost --oplogSize 200
```

With `--replSet`, i say that these server processes are replicas.

Connect with mongosh

```
# Windows (For Ubuntu it's the same)
mongosh --port 27017
```

Configure replicas

Define the following configuration (copy-paste the following code within the mongo shell).

```
rsconf = {  
  _id: "rs0",  
  members: [  
    {_id: 0, host: "localhost:27018"},  
    {_id: 1, host: "localhost:27019"},  
    {_id: 2, host: "localhost:27020"}]};
```

Finally

5. Initiate the replica set (only ONCE)

```
rs.initiate(rsconf);
```

6. Check the status:

```
rs.status();
```

7. Change configuration :

```
rs.reconfig(rsconfig);
```

- Set different priorities to replica set members in order to change the default behavior during the primary election

```
{_id: 0, host: "localhost:27018", priority: 0.5}
{_id: 1, host: "localhost:27019", priority: 1},
{_id: 2, host: "localhost:27020", priority: 3}]
```

- Set tags for different members:

```
{_id:0, host:"localhost:27018", tags:{dc:"east"}},
{_id:1, host:"localhost:27019", tags:{dc:"mid"}},
{_id:2, host:"localhost:27020", tags:{dc:"west"}}]
```

```
rsconf = {
  _id: "lsmdb",
  members: [...],
  settings: {
    setDefaultRWConcern :{w: 2, wtimeout : 5000}}};
```

- **W** is the minimum *number of writes* that the application waits until it regains the flow of execution. The default value is *majority*, but it can be any integer ≥ 1 .
- **Wtimeout** is the maximum *waiting time* (in *ms*) before regaining the flow of execution. If set to 0 the application will wait indefinitely or until “w” is satisfied.

```
// Method 1
MongoClient mongoClient = MongoClient.create(
    "mongodb://localhost:27018,localhost:27019,localhost:27020/" +
    "?retryWrites=true&w=majority&wtimeout=10000");

// Method 2
// You can specify one or more nodes in the replica set.
// MongoDB will automatically discover PRIMARY and SECONDARY nodes within the cluster
uri = new ConnectionString("mongodb://localhost:27018");
MongoClientSettings mcs = MongoClientSettings.builder()
    .applyConnectionString(uri)
    .readPreference(ReadPreference.nearest())
    .retryWrites(true)
    .writeConcern(WriteConcern.ACKNOWLEDGED).build();
MongoClient mongoClient2 = MongoClient.create(mcs);
```

- In this tutorial we have, at our disposal, the following VMs

VM	IP address	OS
Replica-0	10.1.1.10	Ubuntu 20.04.04
Replica-1	10.1.1.11	Ubuntu 20.04.04
Replica-2	10.1.1.12	Ubuntu 20.04.04

- First, we need to connect through a VPN
- Install mongoDB on each VM
- Start and configure the servers in a similar fashion as before

Follow these steps to install MongoDB Community Edition using the `apt` package manager.

1 Import the public key used by the package management system

From a terminal, install `gnupg` and `curl` if they are not already available:

```
sudo apt-get install gnupg curl
```

To import the MongoDB public GPG key from <https://pgp.mongodb.com/server-7.0.asc>, run the following command:

```
curl -fsSL https://pgp.mongodb.com/server-7.0.asc | \
  sudo gpg -o /usr/share/keyrings/mongodb-server-7.0.gpg \
  --dearmor
```


2 Create a list file for MongoDB

Create the list file `/etc/apt/sources.list.d/mongodb-org-7.0.list` for your version of Ubuntu.

Ubuntu 22.04 (Jammy)

Ubuntu 20.04 (Focal)

Create the `/etc/apt/sources.list.d/mongodb-org-7.0.list` file for Ubuntu 20.04 (Focal):

```
echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-7.0
```

3 Reload local package database

Issue the following command to reload the local package database:

```
sudo apt-get update
```

4 Install the MongoDB packages

You can install either the latest stable version of MongoDB or a specific version of MongoDB.

Install the latest version of MongoDB

Install a specific release of MongoDB

To install the latest stable version, issue the following

```
sudo apt-get install -y mongodb-org
```

Large-Scale VPN

1. Install OpenVPN from <https://openvpn.net/community-downloads/>
2. Download the *largescale.ovpn* file containing all the parameters for the VPN.
3. Start OpenVPN and select “Import from file”
4. Select the *largescale.ovpn* file and start the connection

If everything goes well, you should be able to connect through ssh via user=**root** and pass=**root**.

```
ssh root@10.1.1.10
```

- Open 3 different terminals and issues the following commands

```
#Terminal 1
ssh root@10.1.1.10
mkdir data
mongod --replSet lsmdb --dbpath ~/data --port
27020 --bind_ip localhost,172.16.4.99 --oplogSize
200
```

```
#Terminal 2
ssh root@10.1.1.11
mkdir data
mongod --replSet lsmdb --dbpath ~/data --port
27020 --bind_ip localhost,172.16.4.100 --oplogSize
200
```

```
#Terminal 3
ssh root@10.1.1.12
mkdir data
mongod --replSet lsmdb --dbpath ~/data --port
27020 --bind_ip localhost,172.16.4.101 --oplogSize
200
```

- Connect to one of the VM through ssh (open a 4th terminal)

```
#Terminal 4
ssh root@172.16.4.99
mongo --port 27020

rsconf = {
  _id: "lsmdb",
  members: [
    { _id: 0, host: "10.1.1.10:27020", priority:1},
    { _id: 1, host: "10.1.1.11:27020", priority:2},
    { _id: 2, host: "10.1.1.12:27020", priority:5}]
};

rs.initiate(rsconf);
```

Exam Questions (Italian)

Spiega quali sono i bisogni delle aziende moderne che hanno portato allo sviluppo dei database NoSQL.

Le aziende moderne si sono sviluppate su larga scala e devono servire a volte anche miliardi di utenti in tutto il globo.

Necessitano di un paradigma che permetta loro di gestire i BigData:

- **Disponibilità:** Il servizio deve sempre essere attivo, per questo serve replicazione e una gestione efficiente degli aggiornamenti delle repliche.
- **Flessibilità:** Le fonti di informazioni sono tante e variegate, che possono quindi restituire valori o file in formato molto eterogeneo, la natura schematica di un RDBMS non permette una gestione ottimale di tutti questi casi.
- **Scalabilità orizzontale:** Un RDBMS di base scala difficilmente poiché le operazioni SQL, a causa della normalizzazione (*join su tabelle che possono trovarsi sparse per il mondo*) e delle operazioni ACID vanno a consumare molto tempo.

Spesso le applicazioni sono eseguite su un datacenter, e quindi su molte macchine: Il paradigma relazionale non andava bene perché: rende difficoltosa la scalabilità orizzontale a causa delle transazioni ACIDE e della consistenza.

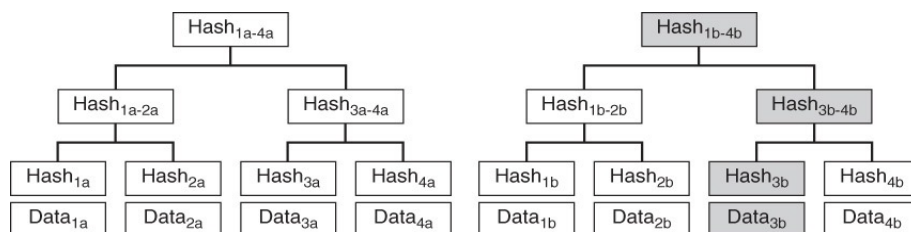
Spiega il processo di Anti-entropia.

Metodo per capire se una replica è aggiornata oppure no.

Ogni partizione possiede un hash tree, dove le foglie sono il valore hash di una determinata sezione dei dati di una partizione.

Ogni nodo dell'albero è l'hash dei suoi nodi figli concatenati.

L'hash tree viene scambiato tra le repliche (*molto meglio scambiare qualche hash che tutto il dataset di una partizione*).



Se al momento dello scambio, una replica si accorge che l'altra replica ha una root diversa, allora una delle due non è aggiornata: Quindi si naviga l'albero seguendo gli hash diversi fino a trovare la sezione diversa, la replica meno aggiornata si fa inviare dall'altra la partizione aggiornata.

In questo modo è facile tenere aggiornate le repliche scambiando il minor numero di dati.

Il processo di aggiornamento non è immediato, ma prima o poi il cluster converge verso la totale consistenza delle repliche.

Spiega i compound indexes in MongoDB.

Indici definiti su più campi. Il funzionamento è lo stesso degli indici normali ma abbiamo qualche dettaglio in più:

Ognuno dei campi può avere un suo ordinamento:

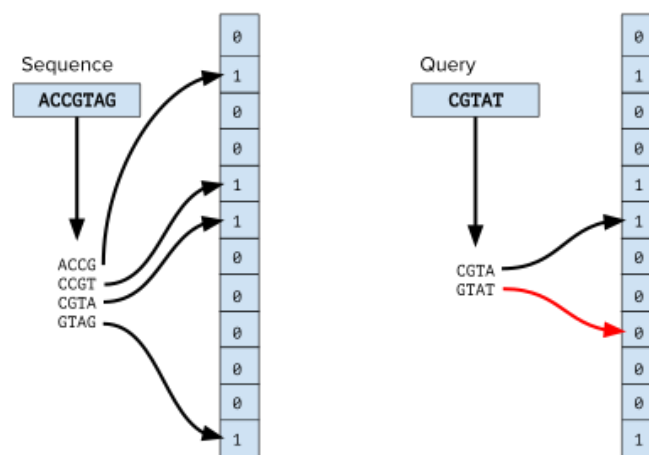
- **ASC** □ 1
- **DESC** □ -1

L'ordine con cui sono stati specificati ha importanza: Se nei criteri di match della query appaiono quei campi ma in diverso ordine, l'indice non verrà utilizzato.

Spiega come funziona un bloom filter.

Il bloom filter è una struttura dati probabilistica (*utilizzabile con ogni tipo di database*) che permette di scoprire se una partizione contiene un certo dato oppure no.

Può dare dei falsi positivi, ma mai dei falsi negativi.



Nel pratico un bloom filter è un array di N posizioni, inizializzato a 0; ogni partizione ha un bloom filter associato.

Si definiscono $K \ll N$ funzioni di hash che restituiscono un numero da 0 a $N-1$.

Data una chiave che deve essere inserita in una partizione:

1. Si eseguono le K funzioni hash.
2. Le K funzioni di hash daranno K valori compresi tra 0 e $N-1$
3. Si settano a 1 tutte le posizioni dell'array indicate dalle funzioni di hash.

Quando si cerca una chiave nella partizione:

1. Si eseguono le K funzioni hash.
2. Le K funzioni di hash daranno K valori compresi tra 0 e $N-1$
3. Si controllano tutte le posizioni dell'array indicate dalle funzioni di hash, se almeno una è a 0, allora sono sicuro che la chiave non è nella partizione.

Un bloom filter conviene quando si hanno pochissime cancellazioni, poiché ad ogni cancellazione occorre ricalcolare tutto il bloom filter, altrimenti potrebbero capitare dei falsi negativi.

I problemi dati dalla proprietà schemaless.

Non avere uno schema da molti vantaggi in termini di flessibilità, ma allo stesso tempo perdiamo alcuni vantaggi molto importanti che invece un database relazionale ci dà:

Controllo sui valori dei campi (Constraints).

Controllo sull'integrità referenziale.

In un database senza schema, se li vogliamo, devono essere realizzati a mano dal programmatore.

Document DB: Quando posso mettere due entità diverse nella stessa collezione?

Quando sono molto spesso acceduti assieme (*poiché i documenti di una collezione tendono a stare vicini nel disco*) e se a livello di attributi sono molto simili (*altrimenti l'utilizzo di indici diventerebbe scomodo*).

Contesti in cui è meglio usare un KVDB.

Quando si necessita di velocità, semplicità e non si ha bisogno di conservare le relazioni tra entità.

Quando si hanno query di lettura e scrittura semplici su entità semplici ma anche molto frequenti.

Viene anche usato come cache per database relazionali o per memorizzare file di configurazione.

Contesti in cui è meglio usare un documentDB.

Quando necessitiamo di un database flessibile in merito alla struttura delle entità, ma abbiamo anche query complesse che necessitano di indici più complessi e filtraggio avanzato.

Perché i KVDB sono più veloci?

I KVDB come Redis spesso sono InMemory, ossia sono database che vengono caricati in memoria RAM; ciò li rende velocissimi.

Quale architettura DB è meglio usare per programmare ad oggetti?

La programmazione ad oggetti apprezza molto un database che memorizzi le entità seguendo lo stesso formato di una classe, alcuni esempi:

Un documentDB come MongoDB memorizza le entità come file JSON i quali sono molto adatti ad essere convertiti in oggetti e viceversa.

Un KVDB come Redis che memorizza i singoli campi come stringhe in un gigantesco array associativo.

Ricorda la frustrazione dei programmatori ad oggetti: la rappresentazione di un oggetto in un RDBMS non coincide con la visione data da una classe.

Che vantaggi propone la proprietà “schemaless”?

Permette avere entità eterogenee senza modificare la struttura del database.

In un RDBMS, per aggiungere o togliere un attributo occorre modificare la struttura e poi anche il codice, mentre in un database NoSQL basta solo modificare il codice e semplicemente inserire la nuova entità senza modificare le altre.

Caratteristiche principali dei Big Data.

Volume: I dati che arrivano sono tanti e spesso appartenenti ad un unico grande file.

Veracità: I dati possono variare nel tempo, nel senso che possono “scadere” o comunque non essere più validi.

Velocità: Ogni sorgente di dati ha una velocità di invio differente.

Varietà: Ogni sorgente di dati può avere un formato differente.

Valore: I dati in sé non hanno valore, ma la conoscenza sì, per estrarla servono procedure che operano in questi dati.

Su applicazioni molto grandi, perché preferiamo Cassandra a MongoDB?

La natura di Cassandra la rende più adatta ad applicazioni altamente distribuite.

Cassandra dispone di un sistema di replicazione privo di single-point-of-failure (*masterless*).

Cassandra inoltre dispone di un protocollo di comunicazione tra i nodi del cluster che la rende particolarmente resistente ai guasti e quindi altamente disponibile.

Differenze tra le operazioni di lettura e scrittura tra MongoDB e Cassandra.

MongoDB utilizza un protocollo di replicazione di tipo master-slave, mentre Cassandra uno di tipo masterless.

Riguardo le scritture:

In MongoDB le scritture vengono fatte sul primary server e poi propagate sulle repliche.

Cassandra invece ha un sistema che permette di inviare la scrittura ad un qualsiasi server del cluster, poi sarà lui ad agire da proxy e inoltrare la scrittura al server target.

Riguardo le letture:

In MongoDB la lettura può essere fatta da una qualsiasi replica dello shard target (*lo shard in cui si trova il dato che vogliamo leggere*), il server in cui si effettua la lettura è scelto mediante una semplice politica decisa dal progettista al fine di implementare del load balancing.

In Cassandra la lettura può essere ricevuta da un qualsiasi server, il quale si comporterà da proxy; quindi, inoltra la lettura al server target, riceve la risposta e a quel punto il proxy la inoltra al client.

Cosa sono gli indici?

Gli indici sono una struttura dati di tipo “*chiave-valore*” che data una chiave associa una serie di puntatori al disco; permettono di scoprire velocemente senza scorrere il database dove sono localizzati tutti i valori associati a quella chiave.

Permettono un operazione di lettura fatta sull’attributo indicizzato più veloce.

Rallentano un operazione di modifica o inserimento fatta sull’attributo indicizzato; poiché l’indice deve essere aggiornato.

Cosa è una network partition e come si gestisce?

Una network partition è uno dei problemi introdotti dal teorema CAP.

Accade quando per motivi di guasto o di progettazione (*come l’introduzione di subnet separate*), alcuni nodi non possono comunicare con altri.

Un sistema è detto “*Partition tolerant*” se nonostante accada una cosa del genere, esso continua a funzionare; in particolare possiamo trovare due approcci:

CP: Il sistema in caso di partition potrebbe non essere disponibile per qualcuno, ma si garantisce che a tutti quelli che funziona verranno prodotti risultati consistenti (*ad esempio l’ATM funziona in una banca ma in un’altra no*).

AP: Il sistema in caso di partition rimane disponibile tutti, ma a qualcuno potrebbero essere forniti dei dati inconsistenti.

Cosa è la eventual consistency?

La eventual consistency è un insieme di tecniche per gestire la consistenza delle repliche nel contesto dei database NoSQL.

Contrariamente alla strict consistency, che prevede che le repliche vengano aggiornate tutte prima della conferma di avvenuta operazione (*two-phase-commit*); durante la replicazione il sistema non è disponibile: non può accettare altre richieste di lettura o scrittura.

Le tecniche di eventual consistency fanno in modo che non tutte le repliche siano aggiornate prima della conferma, ma siano aggiornate in un secondo momento; con la garanzia che prima o poi (*eventually*) verranno aggiornate.

Questo permette al sistema di essere più veloce e quindi più disponibile, al prezzo della possibilità di mostrare dati non aggiornati all’utente.

Quali tipi di eventual consistency ci sono?

Read-your-write consistency

Si garantisce che, nel caso un utente generi una scrittura su un oggetto X, allora ogni sua lettura di X restituirà sempre il valore aggiornato (*strategia per assicurare l'utente riguardo l'avvenuta scrittura*).

Nel caso in cui l'utente leggesse X da un altro server, allora non è garantito che legga il valore aggiornato.

Session consistency

Stessa filosofia della read-your-write consistency ma applicata ad una sessione e non ad un singolo server.

Per sessione si intende una zona, ad esempio tutti gli utenti del nord Italia potrebbero far parte di un'unica sessione.

Si garantisce che tutte le modifiche fatte da un utente vengano subito effettuate all'interno della sua sessione.

Nelle altre sessioni, gli aggiornamenti arriveranno dopo mantenendo lo stesso ordine.

Monotonic-read consistency

Un utente che legge una variabile X; nel caso in cui la leggerà successivamente, non vedrà mai un valore più vecchio di quello che ha già letto.

Monotonic-write consistency

Si garantisce che l'ordine delle scritture effettuate anche da client diversi venga sempre mantenuto.

Ricorda l'esempio dello sconto e dell'acquisto, se cambio l'ordine il risultato finale può essere incoerente.

Causal-consistency

Si garantisce che, nel caso sussista una relazione di causalità tra due scritture (*ad esempio un post e le sue risposte*), la relazione venga rispettata.

Non rispettarla comporterebbe comportamenti inaspettati (*ad esempio vedo solo le risposte ma non il post*).

In caso di repliche ho problemi in scrittura o in lettura?

In caso di repliche problemi in entrambi i casi.

In lettura, poiché (*in caso di eventual consistency*) potrei leggere da una replica non aggiornata e quindi leggere un valore vecchio.

In scrittura, poiché ho un degrado di prestazioni e una maggiore complessità nel caso voglia creare un sistema di eventual consistency.

Va bene scegliere solo una architettura di database per una applicazione?

Certamente, tutto dipende dai requisiti di sistema che uno ha.

In caso di applicazioni piccole e generalmente non distribuite va bene scegliere una sola architettura.

In caso di database che devono avere a che fare con grandi moli di dati, distribuzione su larga scala e query analitiche molto complesse, non è consigliato.

Dato che ogni tipologia di database ha dei pro e dei contro si cerca di sfruttare il meglio di ognuno: Si cerca sempre di avere due lati (*OLAP* e *OLTP*) uno impostato per massimizzare le performance delle operazioni classiche (*Transazioni ACID e query semplici*) e l'altro per massimizzare le performance di query analitiche (*BASE o denormalizzato*).

Che problemi incontro quando uso due tipologie di database differenti nella stessa applicazione?

Il trasferimento dei dati tra i due database diversi che potrebbe aver bisogno di numerose e costose manipolazioni via software.

Ad esempio, trasferire dati da un documentDB ad un RDBMS richiede numerose operazioni SQL.

Un esempio di applicazione di questo tipo è un e-commerce: i dati persistenti (*prodotti, ordini confermati, utenti...*) li memorizzo su un documentDB come MongoDB. Le informazioni riguardo al carrello (*quindi informazioni temporanee, accedute spesso*) le mantengo in un KVDB come Redis impostando come scadenza per le chiavi 48h.

In quali casi è meglio la strict consistency rispetto alla eventual consistency?

La strict consistency va scelta quando la possibilità che l'utente incontri dati non consistenti è inaccettabile (*come, ad esempio, un sistema bancario*) e quindi possiamo fare a meno delle prestazioni.

La eventual consistency può essere scelta quando possiamo tollerare l'imprevisto spiegato sopra e quindi possiamo ottenere prestazioni migliori.

Differenze tra un database relazionale e un columnarDB?

La differenza principale tra le due soluzioni è il modo con cui le informazioni vengono memorizzate sul disco e quindi anche il modo in cui vengono accedute.

In un database relazionale i record sono memorizzati per riga (*tuple*) ciò migliora molto le performance in caso di inserimento, cancellazione e lettura di un intero record. Poiché un blocco del disco spesso contiene interi record.

Un columnarDB segue la logica NoSQL, quindi segue la proprietà schemaless: Due row che comprendono le stesse column family potrebbero avere alcune colonne avvalorate e altre no.

I valori vuoti (*null*) in un columnarDB non occupano spazio.

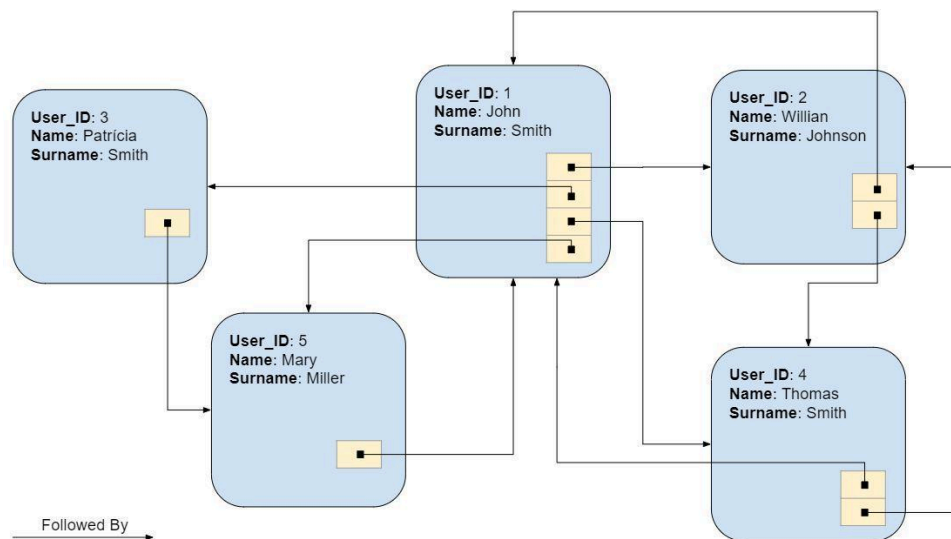
graphDB: Cosa è la Index-free Adjacency?

Tecnica che permette di evitare l'utilizzo di un indice globale per trovare tutte le relazioni di un nodo.

Gli indici in un graphDB servono per evitare una traversata del grafo in cerca delle relazioni che stiamo cercando.

Le relazioni tra i nodi vengono direttamente memorizzare all'intero dei nodi stessi, come se fosse un mini-indice.

Questo permette di evitare una graph traversal o l'utilizzo (*e mantenimento*) di un indice globale (*da ciò il nome index-free*) nel momento in cui devo trovare tutte le relazioni di un certo tipo di un nodo di partenza.



Come si traduce una relazione uno-a-molti in document DB

Ci sono due possibili metodologie, considerando le entità Customer e NumeroTelefono (*un customer può avere molti numeri*).

Document embedding

Le informazioni necessarie al documento (*NumeroTelefono*) del documento (*Customer*) vengono inserite all'interno del NumeroTelefono.

Questo approccio è molto comodo perché permette di eseguire operazioni che in RDBMS richiederebbero una join semplicemente accedendo ad un'unica collezione NumeroTelefono.

Document linking

Quando le informazioni da embeddare sono veramente troppe allora si inserisce solo il campo `_id` del documento Customer.

Questo approccio conviene usarlo quando le “operazioni di join” sono poche e quindi si occupa meno spazio.

La differenza tra una replica e uno shard

Uno shard è una partizione del database che viene inserita in un altro server fisico. L'obiettivo principale dello sharding è quello di fornire load balancing.

Una replica è una copia dell'intero dataset (*o di uno shard*) inserita in un altro server fisico. La replica ha lo scopo di fornire availability nel caso in cui il server con la replica principale (*primary replica*) abbia dei problemi.

Low-level Branching: Posso mettere due entità diverse nella stessa collezione di MongoDB?

Nel pratico è possibile, grazie alla proprietà schemeless dei documentDB, nessuno effettuerà dei controlli sui documenti.

Questa pratica è consigliata solo quando le due entità hanno attributi molto simili e sono accedute spesso assieme (*come, ad esempio, due tipologie di prodotti*).

Descrivi il sistema di replicazione masterless.

Tipologia di replicazione contrapposta alla master-slave.

Non ci sono master, quando un server riceve una richiesta di scrittura la esegue e invia gli updates ai suoi vicini logici (*ogni server ha una piccola lista di altri server che considera suoi vicini*).

I vicini ricevono l'aggiornamento e lo propagano a loro volta ai loro vicini (*per vicini si intendono due o più server designati come tali, di solito i server di un cluster sono interconnessi*).

Questa architettura è ottima nel caso in cui il nostro sistema ha un numero di scritture piuttosto alto, perché in tal caso la master-slave avrebbe un problema di hotspotting nel master.

Questa architettura viene usata per soluzioni altamente distribuite come Cassandra.

Protocollo di comunicazione di Cassandra (lettura e scrittura) e hinted handoff.

Cassandra ha un sistema di comunicazione tra pari: quando un server riceve una richiesta si comporta da proxy e la inoltra al server target:

Lettura: Il proxy inoltra la richiesta al target, riceve la risposta e la inoltra al client.

Scrittura: Il proxy riceve la scrittura (*non la esegue*) e prova ad inoltrarla al target.

Hinted Handoff: Nel caso in cui il target sia spento, si inviano gli update ad un nodo designato il quale conserva gli update e periodicamente controlla che i nodi target siano accesi, appena accesi il designated node invia gli update in modo che possano aggiornarsi.

Descrivi le proprietà BASE.

Basically Available: Il sistema punta ad essere sempre disponibile ad accettare nuove richieste.

Soft State: I dati all'interno del database cambiano nel tempo anche senza operazioni da parte dell'utente.

Eventually consistent: I dati all'intero del sistema non sempre sono in stato consistente, ma so che prima o poi convergerà allo stato consistente.

OLTP, OLAP quale tipologia di database si adatta meglio?

OLTP

- Alto volume di transazioni.
- Necessita transazioni ACID e di dati normalizzati.
- Query semplici del tipo "Mostra gli utenti che hanno comprato questo oggetto".
- Necessità di scritture veloci.

Quindi per un sistema OLTP un database relazionale è una buona scelta.

OLAP

- Necessità di eseguire query complesse, lente e analitiche.
- Molti dati.

Quindi per un sistema OLAP un database columnar è una buona scelta.

Descrivi le proprietà ACID.

Atomicità: Ogni operazione deve essere atomica, quindi non può essere interrotta.

Consistenza: Il database deve rimanere consistente dopo la transazione.

Isolamento: Una transazione eseguita insieme ad un'altra non deve poter vedere gli stati temporanei del'altra.

Durabilità: Gli effetti di una transazioni devono essere permanenti.

Scrivi le proprietà di un database Schemaless.

Un database senza schema permette alle entità di non seguire uno schema predefinito. Ciò permette al Sistema di essere estremamente flessibile di fronte alla varietà data dai big data.

Un RDBMS ha uno schema che va definito a priori, questa cosa gli permette di essere preciso e di fornire controllo automatico dei valori ma allo stesso tempo non gli permette di adattarsi a tutte le possibili situazioni.

Ovviamente ha anche degli svantaggi: non si ha typing-check dei valori (*controllo formato o range*) e non si ha il controllo sull'integrità.

Descrivi il problema dell'hotspotting

Quando in caso di cattivo smistamento del carico, un server del cluster (*o solamente una parte del cluster*) viene sovraccaricato di tutto il lavoro.

Si risolve appunto cambiando lo smistamento delle richieste, magari utilizzando una funzione di hash sequenziale che distribuisca il modo uniforme tutte le richieste ai server del cluster.

Cosa è TimesTen e come funziona?

TimesTen è una soluzione InMemory per database relazionali: L'intero database (*le tabelle*) viene caricato in memoria RAM e ovviamente le operazioni SQL vengono risolte in RAM. La persistenza è ottenuta in due modi:

Snapshots: Periodicamente, il database caricato in memoria viene salvato sul disco. Inoltre, l'ultimo snapshot è la copia che viene caricata in memoria inizialmente.

Transaction log: Tutte le transazioni di inserimento o modifica che hanno effettuato commit vengono elencate (*si fa dopo ogni commit*) in questo file append-only conservato da qualche parte nel disco. Molto utile in caso di ricostruzione, in quanto partendo dall'ultimo snapshot è possibile ricostruire l'intero database prima del problema.

Per entrambe le soluzioni, il caricamento dello snapshot e la modifica del log devono essere fatti in maniera atomica, a costo di perdere performance.

Quando conviene utilizzare un InMemory database?

Quando il database è piuttosto piccolo (*meno di un Terabyte*) .

Quando si hanno molte operazioni che non hanno bisogno di scrivere su una memoria persistente (*ad esempio molte query in lettura*).

File XML e JSON: i pro e i contro di entrambi.

Sia XML che JSON sono human-readable, ossia sono leggibili da un essere umano.

XML è il primo prototipo di file di questo tipo e si presenta come un file HTML e ha questi vantaggi:

- Altamente modulare, quindi parti di documenti XML possono essere usati in altri documenti semplicemente copiandoli e incollandoli.
- Ci sono molti software che permettono di modificare un documento XML tramite una GUI.

XML ha questi svantaggi:

- Anche se per piccole informazioni, un documento XML risulta molto verboso e quindi occupa molto spazio e banda in caso di trasmissione.

JSON ha una struttura di tipo chiave-valore e ha questi vantaggi:

- Altamente modulare, quindi parti di documenti JSON possono essere usati in altri documenti semplicemente copiandoli e incollandoli.
- Molto poco verboso e quindi molto leggero.
- Molto semplice.
- Molti linguaggi possiedono svariate API per maneggiarli.
- Può includere array e altri file json.

Che cosa è il principio della data locality?

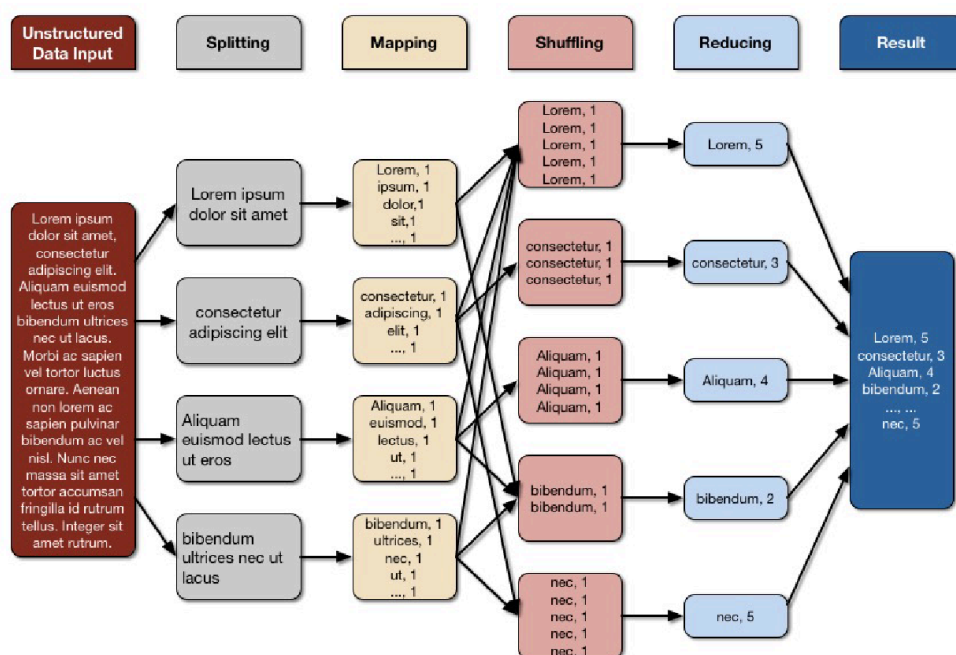
La data locality è un principio che recita: Spostare il calcolo verso i nodi in cui risiedono i dati è molto più economico che spostare i dati verso il calcolo.

Su questo principio si basano tutte le tecniche di calcolo distribuito come MapReduce.

MapReduce, a cosa serve e descrivi le fasi.

MapReduce è un framework per realizzare calcolo distribuito su un cluster di molti nodi e segue la logica della data locality. Si basa sul concetto di Divide et Impera:

1. **Splitting**: Si divide l'input in chunks e ogni chunk viene dato ad un nodo del cluster.
2. **Map**: Ogni nodo esegue un determinato calcolo sul suo chunk.
3. **Shuffle**: Gli output dei mapper vengono raggruppati, secondo un certo criterio; ogni gruppo contiene svariate esecuzioni parziali.
4. **Reduce**: I gruppi vengono ridotti ad un singolo risultato per gruppo.
5. **Result**: I gruppi vengono uniti nell'output finale.



KVDB, estrazione di un valore data la chiave in presenza di molteplici server, elenca tutte le fasi.

Data una chiave si localizza la partizione in cui è stata inserito il valore tramite la soluzione decisa dai progettisti (*funzione modulo o di hash*).

Le chiavi di hash sono divise tra i server tramite un metodo come il consistent hashing.

Questa funzione prende in ingresso la chiave e in uscita in modo deterministico viene restituito il server di destinazione.

Dopodiché si esegue un normale accesso dentro al server target alla ricerca della chiave.

Poi sempre usando una funzione di hash, si individua l'indirizzo fisico; in caso di collisioni si utilizza una tabella di hash con liste di trabocco.

La tabella di hash restituisce l'indirizzo di memoria in cui risiede il valore relativo alla chiave.

Graph DB spiega il funzionamento e per cosa conviene utilizzarlo.

Sfruttiamo la teoria dei grafi per realizzare un database: Le istanze di entità sono i vertici del grafo e le relazioni gli archi tra i vertici.

Le query vengono eseguite facendo delle traversate del grafo, ossia percorrere gli archi da vertice a vertice.

Fare una cosa del genere in un database relazionale richiederebbe moltissime join a catena, ma un graphDB possiede procedure apposta per semplificare questa cosa.

Conviene utilizzarlo quando l'applicazione intende sfruttare molto le relazioni tra entità più che le entità stesse (*ad esempio in caso di un social network*).

Nota che un graphDB difficilmente può essere distribuito, infatti la maggior parte dei DBMS (come Neo4j) non supportano la distribuzione.

I pro e i contro della master-slave replication.

La master-slave replication è una tecnica che prevede un primary server che riceve richieste di lettura e scrittura.

Il primary server quando riceve una scrittura aggiorna (*subito o poco dopo*) le repliche.

Le repliche possono solo rispondere a richieste di lettura.

- **Pro:** la semplicità nel realizzarlo e nel gestire la consistenza tra le repliche, dato che a tutto ci pensa il master.
- **Contro:** Il master è un single-point-of failure, in caso di caduta il sistema non può più accettare richieste di scrittura. In tal caso si esegue un processo di elezione seguendo una policy decisa dal progettista (*periodicamente si verifica l'attività del master, se gli slave non ottengono risposta se ne elegge uno nuovo*).

In un columnarDB, una stringa in formato JSON può essere messa come nome della colonna?

Si può fare, ma non è consigliato poiché i columnarDB sono ottimizzati per l'accesso rapido ai dati; quindi utilizzare nomi complessi può rallentare le operazioni di lettura e scrittura.

Inoltre, renderebbe il database molto meno leggibile.

In Cassandra, ogni colonna viene indicizzata per il nome, quindi nomi complessi possono aumentare l'overhead degli indici. In BigTable abbiamo ragioni simili.

columnarDB: Cosa è il key space, row key, column family e colonna.

Key space: Equivalente del database per i relazionali, è un contenitore di tutte le row key utilizzate nel database.

Row: Descrive un'entità; ossia individua un set di valori appartenenti ad una (o più) column families.

Row key: Identifica una row all'interno del key space.

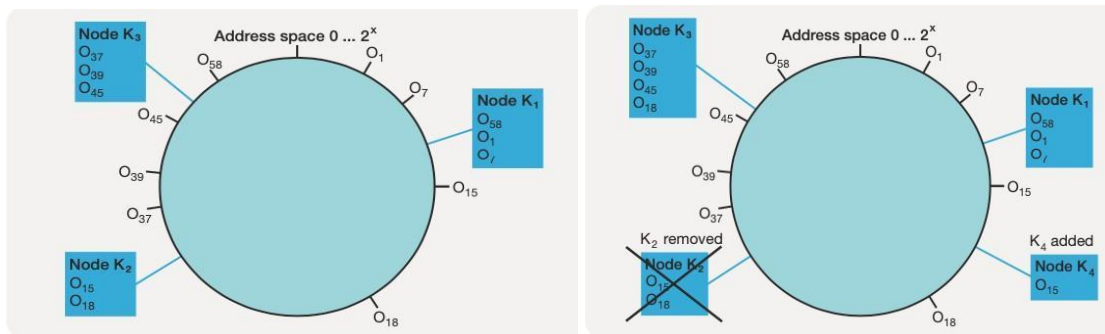
Column family: Gruppo di colonne che sono spesso accedute assieme; infatti, le colonne di una column family sono molto spesso memorizzate vicine nel disco.

Colonna: Set di valori, ognuno associato ad un timestamp (*versione*).

Spiega cosa è il consistent hashing.

Tecnica usata quando in presenza di partizionamento del database, dobbiamo aggiungere o rimuovere un server.

In tal caso il partizionamento di prima non va più bene e quindi dobbiamo ripetere il partizionamento da 0. In un contesto dove vengono create e distrutte macchine virtuali di continuo questa cosa non è accettabile, perché non mentre il sistema non è disponibile.



Nel caso in cui si aggiunga un nodo nuovo: si individua lo spazio delle chiavi che desideriamo assegnargli e riallochiamo solo quelle; quindi, alla peggio coinvolgo solo due server invece che tutti.

Nell'esempio sopra, il nuovo nodo K4 gli assegniamo l'oggetto O15.

Caratteristiche di HANA DB?

Hana DB è una soluzione InMemory per database relazionali ma che utilizza anche una soluzione columnar per velocizzare le query analitiche.

Ha a disposizione una zona del disco per gli snapshot e un transaction log per garantire la persistenza.

La zona di memoria destinata ad Hana è divisa in quattro parti:

- **Row store:** Il database principale dove vengono effettuate operazioni di lettura e scrittura destinate alla parte a righe.
- **L1 Delta Store:** Memoria a righe dove vengono effettuate le scritture destinate alla parte columnar.
- **L2 Delta store:** Le scritture conservate in L1 Delta store vengono successivamente passate in questo store con una leggera compressione a colonne, in attesa di venir trasferite nello store columnar.
- **Store columnar:** La parte altamente compressa in modo colonnare.

Le analitiche vengono eseguite considerando L1, L2 e store columnar.

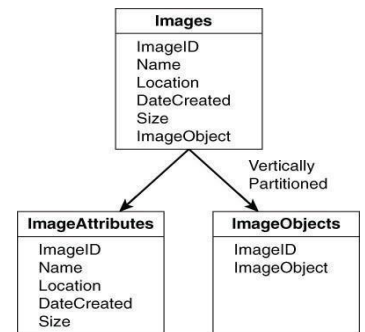
Spiega cosa è lo sharding e il vertical partitioning.

Vertical partitioning (*partitioning*)

“La tabella utenti viene divisa in due gruppi di colonne: una va nel server 1 mentre l'altra nel server 2”.

Lo scopo delle partizioni è quello di rendere più facile la gestione di un database.

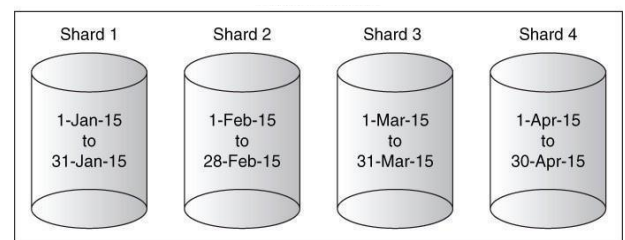
Le partition spesso sono gestite dai DBMS in modo trasparente dal punto di vista dell'applicazione.



Horizontal partitioning (*sharding*)

Consiste dividere le righe di un database in N porzioni più piccole e dividerli in N server fisici.

Ovviamente ogni shard può essere replicata tramite un sistema arbitrario.



“Le righe della tabella utenti con ID < 1000 vanno nello shard 1”.

Lo scopo dello sharding è quello di fornire load balancing (*ogni shard è un server fisico indipendente*) e tolleranza ai guasti (*gli shard sono indipendenti, un guasto di uno non influenza l'altro*).

Le politiche di sharding:

- **Range:** Se una chiave appartiene ad un certo range di valori allora andrà in un certa shard: *“Gli utenti con ID da 1 a 10000 vanno nella partizione 1”.*
- **Hash:** La shard dipende dal valore della funzione hash con input il valore target (*rowID, chiave, etc...*).
- **Lista di valori:** Tutte le chiavi che appartengono a questo insieme vanno in un certa shard: *“Gli utenti con residenza a Firenze e Roma vanno nella partizione 1”.*

Perché lo shard opera su più database? Perché lo sharding è la tecnica per distribuire geograficamente una applicazione in tutto il mondo per diminuire la latenza: Nei server posizionati in America non serve che ci siano i dati degli utenti francesi. Ovviamente un applicazione può avere molteplici database anche su architetture diverse.

Il vantaggio dello sharding rispetto al partitioning:

- Meno costoso.
- Facilmente adattabile al numero di nodi del cluster.

Esercizi Pratici DocumentDB

1. MongoDB: Realizzare una query date le specifiche (*può anche servire una aggregation pipeline*).

La query di base è questa: restituisce dalla collezione `inventory` tutti i documenti con `item` uguale a 10.

```
.find({ item : 10 })
```

La query di base è questa: restituisce dalla collezione `inventory` tutti i documenti con `item` maggiore a 10.

```
.find({ item : { $gt : 10 } })
```

Restituisce dalla collezione `inventory` tutti i documenti che hanno il campo `item`.

```
.find({ item : { $exists : true } } )
```

Restituisce tutti i documenti con `age` uguale ad uno dei valori compresi nell'array.

```
.find({ age : { $in: [ 25 , 26 ] } } )
```

La query di base è questa: restituisce dalla collezione `inventory` tutti i documenti con `item` maggiore a dieci **E ANCHE** con nome "`phone`".

```
.find({ item : { $gt : 10 } , name : "Phone" })
```

La query di base è questa: restituisce dalla collezione `inventory` tutti i documenti con `item` maggiore a dieci **OPPURE** con nome "`phone`".

```
.find({ $or[ { item : { $gt : 10 } , name : "Phone" } ] })
```

Per ordinare il result set:

```
.find({}).sort({ year : 1 , qty : -1 })
```

Per limitare a N risultati.

```
.find({}).limit(n)
```

Per evitare i primi N risultati.

```
.find({}).skip(n)
```

Per contare I risultati usciti:

```
.find({}).count()
```

Per fare una proiezione:

```
db.inventory.find(  
  
{status : 'A' } , // Filter.
```

```
{ item : 1 , status : 1 , _id : 0}) // Projection.
```

Per quanto riguarda l'aggiornamento:

```
.updateOne( <filters> , <update> , <options>)  
  
.updateOne( { name : "Computer"},  
  
{  
  
  $set: { "size.uom" : "cm" , status : "P"} // field to  
change  
  
  $currentDate:{lastModified : true}  
  
}))
```

Aggregation pipeline

Una aggregation pipeline funziona in modo semplice: è divisa in stage e ogni stage valore sull'output dello stage precedente.

Ogni stage può essere una normale query con condizioni di match oppure operazioni di raggruppamento o altro.

Nota per l'esame: se non ti ricordi la sintassi, scrivi la query con uno pseudocodice e spiega a parole gli stage che vorresti usare.

```
> db.zipcodes.aggregate( [  
...   { $group: { _id: "$state", totalPop: { $sum: "$pop" } } },  
...   { $match: { totalPop: { $gte: 10*1000*1000 } } }  
... ] )
```

La prima query: raggruppa tutti gli stati `_id: "$state"` e per ogni stato facciamo la somma della popolazione delle loro città `totalPop: { $gte : 10 * 1000 * 1000 }`.

```
> db.zipcodes.aggregate( [  
...   { $group: { _id: { state: "$state", city: "$city" }, pop: { $sum: "$pop" } } },  
...   { $group: { _id: "$_id.state", avgCityPop: { $avg: "$pop" } } }  
[... ] )
```

In genere si utilizzano tre keyword: `$match`, `$group` e `$sort`.

```

> db.zipcodes.aggregate( [
...   { $group:
...     {
...       _id: { state: "$state", city: "$city" },
...       pop: { $sum: "$pop" }
...     }
...   },
...   { $sort: { pop: 1 } },
...   { $group:
...     {
...       _id : "$_id.state",
...       biggestCity: { $last: "$_id.city" },
...       biggestPop: { $last: "$pop" },
...       smallestCity: { $first: "$_id.city" },
...       smallestPop: { $first: "$pop" }
...     }
...   }
... ]])

```

2. **Progettare un documentDB che sia in grado di soddisfare le due query della applicazione:**
 - Lista dei corsi sostenuti dallo studente.
 - Media dei voti dello studente.
 - La tua guida sono sempre le due query: Ogni query deve sempre poter essere eseguita considerando solo una collezione.
 - Se c'è scritto che la query ha alta frequenza, allora puoi inserire ridondanze per ottimizzarla; altrimenti la ridondanza viene considerata spreco di spazio.
3. **MongoDB: Differenze tra queste due query.**
4. **Convertire una soluzione graphDB in una documentDB a partire da un grafo.**
 - Usa le strategie di document embedding per ricreare le relazioni.
5. **Convertire una soluzione relazionale in una documentDB a partire da un UML.**

Esercizi Pratici KVDB

1. Operazioni fondamentali.

- Ricavare un valore tramite la chiave `val = AppNameSpace[v_key]`
- Settare un valore tramite la chiave `AppNameSpace[v_key]= val`
- Eliminare un valore tramite la chiave

2. Pseudocodice dei metodi Set e Get.

```

define getCustAttr(p_id, p_attrName)
  v_key = 'cust' + ':' + p_id + ':' + p_attrName;
  return (AppNameSpace[v_key]);

define setCustAttr(p_id, p_attrName, p_value)
  v_key = 'cust' + ':' + p_id + ':' + p_attrName
  AppNameSpace[v_key] = p_value

```

Con `p_id` l'identificatore univoco del customer e `p_attrName` il nome dell'attributo che vogliamo prelevare.

3. Pseudocodice per recuperare un oggetto dal database.

Nel metodo costruttore metti tutti i metodi `getCustAttr()`.

4. Pseudocodice per una ricerca dato un attributo.

```
define findCustomerWithAttr(p_startID, p_endID, p_attr): begin
    returnList = ();

    for id in p_startID to p_endID:
        attr = getCustAttr(id , p_attr);

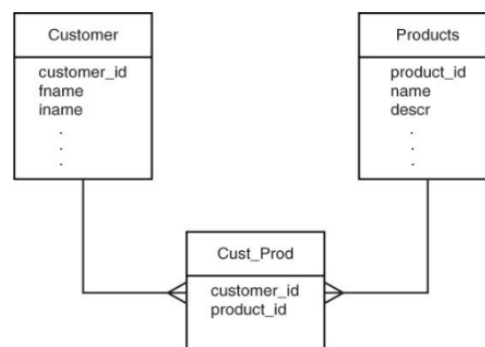
        if inString(p_attr , attr):
            addToList(attr , returnList);

    return(returnList);
end
```

end

Esercizi Pratici columnarDB

1. Come tradurre una relazione molti a molti in un columnarDB.



For every customer I create a redundant structure which contains the list of the object IDs.

The redundant structure is a column family, where every column is a value itself (as can be seen in the image below, the name of the column can store information too).

Cust_Prod			
Column Name →	38383	48282	59595 ← Product ID
Column Value →	Dell Laptop	Apple iPhone	Galaxy Tab S ← Product Name

This column family is linked to the same row key of the customer column family, so using the key `customer1` we can also access the column family of all the products ordered by that customer.

Row key: `customer_id`

Column Family: `customer`

Column: `fname = "Alice"`

Column: `lname = "Rossi"`

...

Column Family: `Cust_Prod`

Column: `38383 = "Dell Laptop"`

Column: `48282 = "Apple iPhone"`

...

Row key: `product_id`

Column Family: `product`

Column: `name = "Dell Laptop"`

...

Column Family: `Prod_Cust`

Column: `customer_id = "Alice"`

...

2. Definire come strutturaresti un ColumnDB date delle specifiche.

Il tuo obbiettivo è quello di definire le column families.

Le colonne che sono spesso accedute assieme sono arte di un'unica column family.

Le query più eseguite sono la guida principale per quanto riguarda la formazione delle column family.

Nota che, in non tutti i DBMS si può includere più di una column family per row, in BigTable è possibile mentre in Cassandra no.

Row key: `user_id`

Column Family: `user`

Column: `name = "Alice"`

Column: `age = 30`

Column Family: address

Column: city = "Roma"

Column Family: other_info

Column: driving_license = "2018-01-06"